Karlsruher Institut für Technologie

Scientific Computing Center

Modelparallelismus in Adaptiv Fourier Neural Operator Transformern für Skalierbare Daten-Getriebene Wettervorhersagen

Masterthesis von

Robin Alexander Maurer

an der Fakultät für Informatik
Scientific Computing Center

Erstgutachter:	Prof. Dr. Achim Streit
Zweitgutachter:	Prof. Dr. Veit Hagenmeyer
Betreuender Mitarbeiter:	Dr. Charlotte Debus

02. Juli 2024 – 02. Januar 2025

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

Ich versichere wahrheitsgemäß, die Arbeit selbstständig angefertigt, alle benutzten Hilfsmittel vollständig und genau angegeben und alles kenntlich gemacht zu haben, was aus Arbeiten anderer unverändert oder mit Änderungen entnommen wurde.

Deißlingen-Lauffen, 02.01.2025



(Robin Alexander Maurer)

Verwendete Hilfsmittel

Generative KI

Während der Entwicklung in Visual Studio Code war Github Copilot aktiv und wurde genutzt, um bestehenden Code zu erläutern und eigenen Code zu simplifizieren. Weiterhin wurde Copilot zur Fehlerbehebung bei NCCL oder anderen Kommunikationsproblemen befragt.

Korrektur von Satzbau, Kommasetzung und Rechtschreibung durch Gratisversion von Writefull in Overleaf.
Weiterhin wurde ChatGPT mit Scholar GPT verwendet, um eigene Formulierungen zu präzisieren. Hierbei wurden keine gesamten Texte übernommen, sondern diese als Anstoß genutzt. Siehe bspw. `https://chatgpt.com/share/676e8b81-facc-800d-8cba-4dd6738b166e` vgl. mit Einleitung des Grundlagenkapitels oder `https://chatgpt.com/share/677284e5-b860-800d-a3ee-87614ce0665c`

Weitere Hilfe

Bei Problemen im Umgang mit Horeka wurden Mitglieder der Forschungsgruppe sowie Stack Overflow zu Rate gezogen.

Weiterhin wurde Code aus HeAT (Helmholz AI Tools) verwendet, um die Kommunikation durch Torch.distributed einfacher aufsetzen zu können: HeAT Snippet Comm.py

Abstract

In this master's thesis, approaches for model-parallel Adaptive Fourier Neural Operators (AFNO) were developed and investigated. The aim was to extend an existing model with model parallelism, to evaluate the scalability of the resulting architectures and to analyze the resulting overhead. For evaluation purposes, several parallelization approaches were implemented and examined under realistic test conditions.

The results show that the model-parallel architectures did not exhibit any significant scaling effects under the given framework conditions. However, fine-grained parallelization approaches achieved better results in terms of model accuracy compared to global parallelization strategies. At the same time, pure data parallelism was found to be the most efficient solution for stand-alone AFNONet applications.

This work provides important negative insights into the limitations and challenges of scaling the AFNO using model parallelism and provides a basis for further investigations, for example the implementation due to time restrictions, discarded approaches, the use in another application field or the investigation of the successor model Spherical Harmonics Neural Operator (SNO).

Zusammenfassung

In dieser Masterarbeit wurden Ansätze für modellparallele Adaptive Fourier Neural Operators (AFNO) entwickelt und untersucht. Ziel war es, ein bestehendes Modell um Modellparallität zu erweitern, die Skalierbarkeit der resultierenden Architekturen zu bewerten und den entstehenden Overhead zu analysieren. Zur Evaluation wurden mehrere Parallelisierungsansätze implementiert und unter realistischen Testbedingungen untersucht.

Die Ergebnisse zeigen, dass die modellparallelen Architekturen unter den gegebenen Rahmenbedingungen keine signifikanten Skalierungseffekte aufwiesen. Feingranulare Parallelisierungsansätze erzielten im Vergleich zu globalen Parallelisierungsstrategien jedoch bessere Ergebnisse hinsichtlich der Modellgenauigkeit. Gleichzeitig stellte sich heraus, dass reiner Datenparallelismus für Stand-Alone-Anwendungen des AFNONets die effizienteste Lösung darstellen.

Diese Arbeit liefert wichtige negative Erkenntnisse zu Grenzen und Herausforderungen bei der Skalierung des AFNO mittels Modellparallelismus und bietet eine Grundlage für weiterführende Untersuchungen, beispielsweise die Umsetzung, auf Grund von zeitlichen Restriktionen, verworfener Ansätze, der Nutzung in einem anderen Anwendungsfeld oder der Untersuchung des Nachfolgemodells Spherical Harmonics Neural Operator (SNO).

Inhaltsverzeichnis

Verwendete Hilfsmittel i

Abstract iii

Zusammenfassung v

1. Einleitung 1
1.1. Zielsetzung . . . 2
1.2. Aufbau der Arbeit . . . 2

2. Grundlagen 3
2.1. Neuronale Netze und Transformer . . . 3
2.1.1. Transformer Architekturen . . . 4
2.1.2. Encoder-Decoder-Struktur . . . 5
2.1.3. Visual Transformer . . . 5
2.2. Tiefe Neuronale Netze und Parallelisierungsansätze . . . 5
2.3. Datengetriebene Wettermodelle . . . 7
2.3.1. Herausforderungen datengetriebener Wettermodelle . . . 7
2.3.2. Beispiele datengetriebener Wettermodelle . . . 8
2.4. Adaptive Fourier Neural Operators und FourCastNet . . . 8
2.4.1. FourCastNet Architektur . . . 9
2.4.2. Mathematische Grundlagen der Adaptive Fourier Neural Operators 11
2.4.3. Parallelisierung in FourCastNet . . . 13

3. Forschungsmethoden und Versuchsaufbau 15
3.1. Rahmenbedingungen der Experimente . . . 15
3.2. Kommunikation und Speicherlayouts . . . 16
3.3. Metriken . . . 18
3.4. Datensatz . . . 20
3.5. Überlegungen zur Modellparallelisierung . . . 21
3.5.1. Strategien zur Modellparallelisierung . . . 21
3.5.2. Lokale und globale Parallelisierungsansätze . . . 23
3.5.3. Kommunikationsaufwand und Implementierungsdetails . . . 24

4. Implementation 29
4.1. Experimente . . . 29
4.2. Anpassung des Trainingscodes . . . 30
4.3. Implementation des Modellparallelismusses . . . 31

5. Evaluation **33**
5.1. Baseline . . . 33
5.2. MP-Modelle . . . 34
5.2.1. Skalierbarkeit . . . 35
5.2.2. Overhead . . . 41
5.2.3. Loss . . . 41
5.3. Herausforderungen und Verworfene Ansätze . . . 44
5.3.1. Verworfene Ansätze . . . 44
5.3.2. Aufgetretene Probleme . . . 47

6. Fazit **51**

7. Ausblick **53**

Literatur **55**

A. Anhang **63**

Abbildungsverzeichnis

2.1. Eigene Darstellung der Architektur von FourCastNet, basierend auf dem Code des Originalpapers [46], erstellt in Miro. Auf der linken Seite ist das Model nach den implementierten Modulen untergliedert: FourCastNet in gelb, das AFNONet-Backbone in grau, der Inhalt der Encoder-Blöcke des AFNONet in hellem Grün, dem AFNO für das Spatial-Mixing in limettengrün, mit Fourier-Schicht in minzgrün und das MLP, zuständig für das Channel-Mixing, in dunkelgrün. Auf der rechten Seite erfolgt die Unterteilung nach Layer-Typen, wie Convolutional-Layern in petrol oder Aktivierungsfunktionen in lila. 10

3.1. Darstellung von zusammenhängendem und nicht zusammenhängendem Speicher. Diese Abbildung ist zu finden in [21]. 17

3.2. Abbildungen übernommen aus [53]. Die Abbildungen zeigen die Umsetzung des Parallelismus in Megaton-LM. Abbildung A ist hierbei die grobgranulare Struktur, in der die Schichten des Transformers durch Pipelining und die einzelnen En- und Decoder über Tensorparallelismus verteilt werden. Abbildung B zeigt eine vergrößerte Ansicht der Encoder und Decoder. Weiterhin sind darin die Kommunikationsfunktionen `f` und `g` zu erkennen. 22

3.3. Visualisierung der Architekturen mit eingefügten Kommunikationsschichten (orange hervorgehoben). Auf der linken Seite ist die globale Parallelisierungsstrategie zu erkennen, bei der lediglich 2 Kommunikationsoperationen pro Forward und Backwardpass hinzukommen. Mittig sowie auf der rechten Seite sind die lokalen oder feingranularen Strategien abgebildet. Diese unterscheiden sich lediglich durch die Position sowie durch die Dimensionalität des zu kommunizierenden Tensors. 25

3.4. Eigene Darstellung eines Tree-Allgathers 26

5.1. Aufführung der durchschnittlichen Zeit in einer Epoche sowie dem Sampleduchsatz pro GPU, für 8, 16, 32 und 64 GPU, der verschiedenen Architekturen 36

5.2. Tabellen zur Darstellung der durchschnittlich benötigten Rechenzeit pro Epoche sowie dem Sampleduch- satz pro GPU, für 8, 16, 32 und 64 GPU, der verschiedenen Architekturen inklusive Baseline und Experimentkonfigurationen. 37

5.3. Diagramm zur Darstellung der relativen starken Skalierbarkeit des globalen Ansatzes. Neben der Ideallinie (100% Skalierbarkeit) werden die verschiedenen MP-Sizes in rot, grün und blau aufgeführt. Messwerte sind hier mit einem Kreuz gekennzeichnet, wobei die gepunkteten Linien lediglich der Visualisierung und Projektion dienen. 38
5.4. Diagramme zur Veranschaulichung der Metriken der schwachen Skalierung aus den Experimenten auf der globalen Parallelisierungsstrategie. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3. 39
5.5. Diagramm zur Darstellung der relativen starken Skalierbarkeit der lokalen Ansätze. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3. 39
5.6. Diagramme zur Veranschaulichung der Metriken der schwachen Skalierung aus den Experimenten der lokalen Parallelisierungsstrategie der Verteilung über die Hidden-Dimension. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3. 40
5.7. Diagramme zur Veranschaulichung der Metriken der schwachen Skalierung aus den Experimenten der lokalen Parallelisierungsstrategie der Verteilung über die Diagonalblöcke. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3. 40
5.8. Diagramme zur Veranschaulichung des Overheads der Modelle der verschiedenen Experimente, erstellt in W&B. Hierbei werden diese in Kommunikations und Datenvorverarbeitungs-Overhead aufgeteilt und ins Verhältnis gegenüber der Rechenzeit gestellt. Dies geschieht für die Verwendung von 8 bzw. 64 GPUs auf dem globalen Ansatz mit MP-Sizes 2 und 8. . . . 42
5.9. Tabellarische Abbildungen der Ergebnisse aller Experimente auf 64 GPUS. 43
5.10. Verlauf des Loss entlang der Epochen des Trainings. In dieser Abbildung wurden aus allen Experimenten mit 64 GPUs, zehn Modelle mit den besten Losswerten bis Epoche 5 ausgewählt und den Trainingsverlauf in diesem Diagram dargestellt. 44
5.11. Beispielhafter Parallelisierungsplan des AFNONet für torch.distributed tensor.parallel. 46

A.1. Baseline Output bei 64 GPU in Epoche 50. 64
A.2. Output Baseline 8GPU Ep50 64
A.3. Output bei 64 GPU Global Weak MP8 in Epoche 50. 65
A.4. Output bei 64 GPU Global Strong MP8 in Epoche 50. 65
A.5. Output bei 64 GPU Global Weak MP2 in Epoche 50. 66
A.6. Output bei 64 GPU Global Strong MP2 in Epoche 50. 66
A.7. Output bei 64 GPU Hidden Strong MP8 in Epoche 40. 67
A.8. Output bei 64 GPU Diagonal Weak MP2 in Epoche 40. 67
A.9. Output bei 64 GPU Diagonal Strong MP2 in Epoche 40. 68
A.10. Output bei 64 GPU Hidden Weak MP2 in Epoche 50. 68

A.11. Output bei 64 GPU Hidden Strong MP2 in Epoche 50. 69
A.12. Mogliche Implementation eines Block up Layers für den verworfen Ansatz 70
A.13. Darstellung der Modulusmodelle nach vorbild von Abbildung 2.1 71
A.14. Fehler beim verteilen von nicht Kontigeren Dimensionen sowie Timings 72
A.15. Barebone MPI Commtime pro epoche 72
A.16. Nvidia zu Cuda-aware MPI mit NCCL 72
A.17. Modulus Init Shinanigans 73

Tabellenverzeichnis

3.1. Auflistung der verwendeten Variablen aus ERA5. Die Abkürzungen bedeuten das folgende: $U_{10}(V_{10})$: zonale (meridonale) Windgeschwindigkeit in $10m$ Höhe über der Oberfläche; T_{2m}: Temparatur $2m$ über der Oberfläche; T,T(sic!)[U],V,Z.(sic!)[,]RH: Temperatur, zonale Geschwindigkeit, meridonale Geschwindigkeit, Geopotenzial, relative Feuchtigkeit in der jeweiligen vertikalen Ebene; $TCWV$: Gesamter Wasserdampf der Vertikalen. Diese Tabelle und ihre Beschreibung ist aus [46] übernommen. 20
3.2. Zusammenfassung der Kommunikationskomplexitäten der verschiedenen Ansätze 27

5.1. Vergleich der Baseline Experimente mit dem Ursprungsmodell. Verglichen wird über die Anzahl an verwendeten GPUs anhand des Validierungsfehlers, der Laufzeit sowie der Gesamtnutzung von GPU Ressourcen. 34

1. Einleitung

In den letzten Jahren haben Durchbrüche im Bereich der Künstlichen Intelligenz (KI) dazu geführt, dass die Komplexität von KI-Modellen stetig zunimmt. Die Modelle werden zunehmend größer und erfordern immense Rechenressourcen, was die praktische Nutzung in realen Szenarien erschwert.[41]

Große KI-Modelle haben neben ihrer beeindruckenden Leistungsfähigkeit auch Nachteile. Sie sind nicht nur äußerst speicherintensiv, sondern führen unter anderem auch durch längeren Rechenzeiten zu erhöhtem Energieverbrauch[30]. Diese Rechenzeiten werden durch paralleles Training mitigiert, bei dem mehrere Instanzen eines Modells gleichzeitig trainiert und anschließend synchonisiert werden. Insbesondere wenn die Größe der Modelle den Speicher einzelner Recheneinheiten übersteigt, stoßen klassische Berechnungsmethoden an ihre Grenzen. An dieser Stelle wird auf Modellparallelismus zurückgegriffen, eine Technik, die es ermöglicht, die Berechnungen eines Modells über mehrere Recheneinheiten hinweg zu verteilen. Parallelisierte Modelle ermöglichen eine signifikante Reduzierung der Berechnungszeit und erlauben die Verarbeitung von Modellen und Daten, die die Kapazitäten einzelner Recheneinheiten übersteigen. Durch die Verteilung der Rechenlast auf mehrere Einheiten können selbst äußerst komplexe Probleme gelöst werden. Diese Eigenschaften machen Parallelisierungstechniken unverzichtbar für moderne Anwendungen in Wissenschaft und Industrie, auch wenn diese oft mit einem höheren Ressourcenverbrauch einhergehen können.[4]

Ein Anwendungsbereich, in dem diese Modelle eine zentrale Rolle spielen, ist die Wettervorhersage. Klassische Wetterberechnungen basieren auf physikalischen Modellen und numerischen Simulationen, die besonders rechenintensiv und zeitaufwendig sind[40]. In den letzten Jahren haben sich Neurale Operatoren als eine vielversprechende Technologie etabliert, die traditionelle numerische Simulationen in verschiedenen Bereichen, einschließlich der Wettervorhersage, revolutionieren können. Diese KI-gestützten Ansätze bieten eine neue Methode zur Approximation partieller Differentialgleichungen (PDE), die herkömmlicherweise die Grundlage für Wettermodelle bilden.[37, 46]

Der entscheidende Vorteil Neuraler Operatoren liegt in ihrer Fähigkeit, Vorhersagedaten deutlich schneller als klassische numerische Methoden zu generieren. Dies wird erreicht ohne auf teure Gitterverfeinerung oder explizite physikalische Modellierungen angewiesen zu sein. Trotz der verkürzten Rechenzeit bleiben die Genauigkeitsverluste im Vergleich zu traditionellen Verfahren minimal, was sie für kurzfristige Vorhersagen besonders attraktiv macht.[46]

Die Bedeutung schneller Vorhersagen zeigt sich in Anwendungsbereichen wie der Luftfahrt und der Energieplanung. Schnelle und präzise Wettervorhersagen ermöglichen es, Flugrouten zu optimieren, die Sicherheit zu erhöhen und Kosten zu senken. Ebenso profitiert die Gesellschaft durch verbesserte Katastrophenwarnsysteme, die Leben retten und wirtschaftliche Schäden reduzieren können.

1.1. Zielsetzung

In dieser Masterarbeit liegt der Fokus auf der Entwicklung modellparalleler Architekturen für Adaptive Fourier Neural Operator (AFNO). Das Hauptziel besteht darin verschiedene Ansätze zu implementieren, die das Ursprungsmodell um die Option des modellparallelen Trainings und der Inferenz zu erweitern. Dabei soll sichergestellt werden, dass die ursprüngliche Genauigkeit des Modells erhalten bleibt oder idealerweise verbessert wird.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Untersuchung der entwickelten Ansätze hinsichtlich ihrer Skalierbarkeit und Effizienz. Zudem wird der durch die Parallelisierung entstehenden Overhead bewertet und die Laufzeiten zwischen der ursprünglichen und den modellparallelen Architekturen verglichen. Dabei wird die Frage adressiert, ob und unter welchen Bedingungen der Einsatz von Modellparallelismus einen substanziellen Mehrwert bietet.

1.2. Aufbau der Arbeit

Die vorliegende Masterarbeit gliedert sich wie folgt: In Kapitel 2 werden die theoretischen Grundlagen dieser Arbeit vorgestellt. Zunächst erfolgt eine kurze Einführung in Neuronale Netze und Transformer-Architekturen, gefolgt von einer Erläuterung der Prinzipien des Parallelismus in tiefen neuronalen Netzen. Darüber hinaus werden datengetriebene Wettermodelle wie GraphCast, PanguWeather und FourCastNet beschrieben, sowie einen Überblick über relevante Datensätze, die in der Wetterforschung verwendet werden gegeben. Im Anschluss wird die Funktionsweise sowie die mathematischen Grundlagen von Fourier Forecasting Neural Network (FourCastNet) und AFNO näher erläutert.

In Kapitel 3 werden die methodischen Ansätze der Arbeit beschrieben. Dies umfasst die Vorüberlegungen zur Implementierung des Modells sowie zur Planung und Durchführung der Experimente, wie etwa Rahmenbedingungen und gewählte Metriken.

Kapitel 4 widmet sich der detaillierten Implementierung des Modells. Es werden die vorgenommenen Architekturänderungen sowie die technischen Herausforderungen beschrieben, die während der Umsetzung aufgetreten sind.

In Kapitel 5 erfolgt die Evaluation der durchgeführten Skalierungs-Experimente. Die Ergebnisse werden analysiert, und die entwickelten Modelle werden hinsichtlich ihrer starken und schwachen Skalierbarkeit sowie weiteren Metriken bewertet. Dabei wird insbesondere untersucht, wie effizient die Ansätze bei unterschiedlichen Ressourcenanforderungen und Skalierungsniveaus funktionieren. Zusätzlich werden Konzepte und Ansätze diskutiert, die im Laufe der Arbeit verworfen wurden, um den methodischen Entscheidungsprozess transparent darzustellen.

Den Abschluss bildet Kapitel 6, das die wichtigsten Erkenntnisse der Arbeit zusammenfasst. Darüber hinaus wird ein Ausblick auf mögliche Weiterentwicklungen und zukünftige Forschungsschwerpunkte gegeben, die sich auf Basis dieser Arbeit ergeben.

2. Grundlagen

Die Grundlagen dieses Kapitels legen die theoretischen Bausteine dar, die für das Verständnis der nachfolgenden Abschnitte erforderlich sind. Ziel ist es, ein Fundament für die behandelten Themen zu schaffen, indem wesentliche Konzepte, Technologien und Herausforderungen aufgezeigt werden. Hierbei werden zunächst Neural Networks (NNs) (dt. Neuronale Netze) erläutert und die bedeutendsten Durchbrüche im Laufe der Zeit dargestellt mit Fokus auf die Transformer-Architektur. Weiterhin werden Parallelisierungsansätze aufgezeigt und datengetriebene Wettermodelle und deren einhergehenden Herausforderungen thematisiert. Abschließend werden die mathematischen Grundlagen von Adaptive Fourier Neural Operators (AFNOs) (dt. Adaprive Fourier Neural Operatoren) sowie die Architektur des FourCastNet (*Four*ier Fore*Cast*ing Neural *Net*work) erläutert.

2.1. Neuronale Netze und Transformer

NN sind ein Grundpfeiler der Künstlichen Intelligenz (KI) und sind inspiriert durch die Funktionsweise biologischer neuronaler Systeme. Die ersten neuronalen Modelle stammen von Frank Rosenblatt (1958), der das Perzeptron entwickelte. Dieses einfache Netzwerk bestand aus einem Input-Layer (dt. Eingabeschicht), einer Gewichtungsfunktion und einem Output-Layer (dt. Ausgabeschicht). Es war in der Lage, einfache lineare Zusammenhänge zu erlernen, scheiterte jedoch bei der Lösung komplexerer Probleme wie dem XOR-Problem. [48]

Ein bedeutender Fortschritt erfolgte mit der Einführung von Aktivierungsfunktionen, mehrschichtigen Netzwerken, den sogenannten Multilayer Perceptrons (MLPs), und dem Backpropagation-Algorithmus. Durch das Anwenden nicht-linearer Aktivierungsfunktionen, wie die Sigmoid-Funktion, auf die Ausgaben der Layer, konnten nicht-lineare Zusammenhänge gelernt werden [13]. Backpropagation ermöglichte es, Gewichte in tieferen Netzwerken durch Gradientenabstieg effizient zu aktualisieren.[49] Diese Methode bildete die Grundlage für viele spätere Erfolge im Bereich der KI.

Breite Netzwerke bestehen aus wenigen Layern mit vielen Neuronen und erfordern hohe Rechen- und Speicherressourcen [50]. Deep Neural Networks (dt. Tiefe Neuronale Netzwerke, DNNs) sind Neural Networks mit vielen Hidden-Layern (dt. versteckten oder Zwischen-Schichten) mit geringer Neuronendichte und können mit weniger Parametern ähnliche Ergebnisse wie breite Netzwerke erzielen, sind jedoch anfällig für Herausforderungen wie das Saturieren von Gradienten, wenn das Netz zu tief wird [35, 50, 13]. Das Trainieren solcher DNN wird auch Deep Learning genannt.

In den 1990er-Jahren kamen Convolutional Neural Networks (CNNs) auf, die durch die Arbeit von Yann LeCun bekannt wurden. CNNs revolutionierten die Verarbeitung visueller Daten, indem sie lokale Muster und hierarchische Merkmale mittels Convolutional-Layern

(dt. Faltungsschichten) extrahierten. Ihre Architektur ermöglichte es, Bilddaten effizienter zu analysieren als klassische vollvernetzte NN.[34]

Für sequenzielle Daten wie Sprache oder Zeitreihen wurden Recurrent Neural Networks (RNNs) entwickelt, bei denen der Output des Netzes zusammen mit dem nächsten Teil der Inputsequenz kombiniert und wieder in das Netz zurückgeleitet wird. Allerdings litten RNNs unter dem Problem der Gradientensaturierung, wodurch sie Schwierigkeiten hatten, langfristige Abhängigkeiten zu modellieren.[20] Dieses Problem wurde durch die Einführung von Long-Short Term Memory (LSTM)-Netzwerken gelöst, die ein explizites Langzeitgedächtnis integrierten [29].

2.1.1. Transformer Architekturen

Ein fundamentaler Wandel, im Bereich der sequenziellen Datenverarbeitung, kam mit der Veröffentlichung des Papers „Attention Is All You Need“ von Vaswani et al. (2017). Die Transformer-Modelle des Papers verzichteten auf rekurrente Strukturen und nutzten stattdessen den Self-Attention-Mechanismus, der es ermöglicht, Abhängigkeiten zwischen beliebigen Positionen in einer Sequenz gleichzeitig zu modellieren. Diese Architektur führte zu deutlichen Verbesserungen sowohl in der Trainingsgeschwindigkeit als auch in der Qualität der Ergebnisse.[55]

Transformer-Modelle wie GPT (Generative Pre-trained Transformer), BERT (Bidirectional Encoder Representations from Transformers) und ViT (Visual Transformer) haben zahlreiche KI-Anwendungen revolutioniert:

- **GPT:** Erzeugung von Texten, maschinelle Übersetzung und Frage-Antwort-Systeme [44].
- **BERT:** Textklassifikation, Sentiment-Analyse und Named Entity Recognition [16].
- **ViT:** Bildklassifikation, Objekterkennung und Segmentierung [17].

Self-Attention und Multi-Head-Attention. Self-Attention ist das zentrale Konzept der Transformer-Architektur [55]. Dabei wird erst für jedes Eingabeelement eine Gewichtung der Beziehung zu anderen Eingabeelementen berechnet. Der Mechanismus basiert auf drei Matrizen:

- **Query (Q)**: Repräsentiert die aktuellen Eingaben, deren Relevanz berechnet werden soll.
- **Key (K)**: Dient zur Berechnung der Gewichtungen für die Queries.
- **Value (V)**: Enthält die Informationen, die entsprechend der berechneten Gewichte skaliert werden.

Der Self-Attention-Score wird durch ein skaliertes Skalarprodukt berechnet:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{2.1}$$

wobei d_k die Dimension der Key-Vektoren ist. Der Softmax-Operator erzeugt die Gewichtungen für die Values.[55]

Um mehrere Aspekte der Daten simultan zu erfassen, wird der Multi-Head-Attention-Mechanismus verwendet. Dabei werden die Eingabedaten parallel in mehrere „Attention-Heads“ (ein Attention-Head besteht aus je einer Gruppe Q-,V-,K- Matrizen) projiziert, die unabhängig voneinander unterschiedliche Gewichtungen berechnen. Die Ergebnisse der einzelnen Heads werden anschließend zusammengeführt und weiterverarbeitet.[55]

Token-Mixing. Die Basiseinheit numerischer Darstellungen des Inputs werden als Token bezeichnet. Im Self-Attention-Mechanismus wird die Beziehung zwischen verschiedenen Positionen der Token modelliert. Dies ermöglicht ein sogenanntes Token-Mixing, bei dem Informationen über die gesamte Eingabesequenz global ausgetauscht werden.[55] Dadurch können Transformer kontextabhängige Repräsentationen erstellen, die sowohl lokale als auch globale Abhängigkeiten erfassen.

2.1.2. Encoder-Decoder-Struktur

Die Transformer-Architektur besteht aus einer Encoder-Decoder-Struktur. Der Encoder nimmt Eingabedaten (z.B. Texte oder Bilder) auf und erzeugt latente Repräsentationen. Der Decoder verwendet diese Repräsentationen, um die Ausgabe zu generieren.[55]

Eingaben werden zunächst durch Embedding-Layer (dt. Einbettungsschichten)in kontinuierliche Wertevektoren transformiert und mit Positionsinformationen ergänzt, da Transformer keine inhärente Reihenfolge der Daten berücksichtigen. Diese Positionsinformationen werden meist durch sinus- und cosinus-basierte Positional Embeddings eingebettet.[55]

2.1.3. Visual Transformer

Mit den Visual Transformers (ViT), erstmals von Dosovitskiy et al. (2020) vorgestellt, wurde die Transformer-Architektur erfolgreich auf Bildverarbeitungsaufgaben übertragen. ViTs zerlegen Bilder in kleinere, gleichgroße Patches und behandeln diese wie Wörter in einem Satz. Durch Self-Attention werden die Beziehungen zwischen den Patches modelliert, wodurch sowohl lokale als auch globale Bildinformationen erfasst werden können.[17]

2.2. Tiefe Neuronale Netze und Parallelisierungsansätze

DNNs sind eine Klasse von künstlichen NN, die aus vielen Layern mit wenigen Neuronen bestehen und in der Lage sind, komplexe Muster und Zusammenhänge in den Eingabedaten zu lernen [36]. Sie ermöglichen es, hierarchische Merkmale auf verschiedenen Abstraktionsebenen zu erfassen, was sie besonders geeignet für Probleme macht, bei denen Daten komplexe Strukturen aufweisen, wie beispielsweise bei Bildern, Sprache oder wissenschaftlichen Simulationen.[4]

Um die Herausforderungen der Skalierbarkeit und Rechenkomplexität zu adressieren, spielen Parallelisierungsansätze beim Training von DNNs eine entscheidende Rolle. Einer

der wichtigsten Ansätze ist der Datenparallelismus, bei dem das gleiche Netzwerk auf mehrere Geräte repliziert wird, während der Datensatz in unabhängige Batches (dt. Partitionen) aufgeteilt wird [4]. Eine gängige Variante davon ist der Batch-Parallelismus. Hier werden die Eingabedaten in Mini-Batches aufgeteilt, die parallel auf unterschiedlichen Geräten verarbeitet werden. Nach jedem Berechnungsschritt erfolgt eine Synchronisation durch das Mitteln der Gradienten. Alternativ können auch spezifische Teile der Eingabedaten parallelisiert werden, wie beim Channel-Parallelismus, bei dem die Kanäle (z.B. RGB-Kanäle) auf verschiedene Geräte verteilt werden. Für Modelle, die auf kleine, hoch-dimensionale Datensätze angewendet werden, eignet sich der Feature-Parallelismus, bei dem einzelne Features aus den Samples unabhängig voneinander verarbeitet werden.[4]

Ein weiterer zentraler Ansatz ist der Modellparallelismus, bei dem das Netzwerk selbst in mehrere Teile aufgeteilt wird. Dies ist besonders nützlich, wenn die Größe des Modells die Kapazitäten eines einzelnen Geräts überschreitet. Obwohl dies die Speicheranforderungen reduziert, steigen die Kommunikationskosten zwischen den Geräten. Eine mögliche Aufteilung erfolgt schichtweise, wobei die Layer des Netzwerks auf verschiedene Geräte verteilt werden. Hierbei entstehen Leerlaufzeiten in den Geräten. Alternativ können auch Teile einer einzelnen Schicht parallelisiert werden, etwa durch die Verteilung von Neuronen oder Sub-Modulen auf mehrere Geräte. Für besonders große Matrizenoperationen, wie sie in modernen Architekturen wie Transformern häufig vorkommen, bietet der Tensorparallelismus eine effiziente Lösung. Hier wird ein einzelnes Tensorobjekt, beispielsweise eine Gewichtsmatrix oder ein Aktivierungstensor, auf mehrere Geräte aufgeteilt, wodurch sich die Rechenzeit erheblich reduzieren lässt.[4]

Eine weitere Möglichkeit zur Optimierung der Trainingszeit stellt das Pipelining dar. Dabei wird das Training in aufeinanderfolgende Verarbeitungsschritte unterteilt, sodass die Geräte gestaffelt arbeiten und die Leerlaufzeiten der Geräte bei schichtweiser Aufteilung des Netzwerks reduziert werden. Ein Beispiel hierfür ist der Fall, in dem ein Gerät die erste Schicht für einen bestimmten Input verarbeitet, während ein anderes Gerät zeitgleich die zweite Schicht für einen vorherigen Input berechnet. Pipelining erfordert jedoch ein komplexes Synchronisationsmanagement, um Verzögerungen zu minimieren.[4] Ein bekanntes Framework für Pipelining ist z. B. DreamPipe.

Ein fortschrittlicher Optimierungsansatz für verteiltes Training ist der Zero Redundancy Optimizer (ZeRO). ZeRO minimiert die Speicheranforderungen, indem Modellparameter, Gradienten und Optimizer-Zustände über mehrere Geräte aufgeteilt werden. Dadurch entfällt die Notwendigkeit der vollständigen Duplizierung dieser Elemente auf jedem Gerät. Dies ermöglicht nicht nur erhebliche Einsparungen bei der Speicherbelegung, sondern reduziert auch die Kommunikationskosten während der Gradientenberechnung. ZeRO ist besonders skalierbar und erlaubt das Training von Modellen mit mehreren Milliarden Parametern auf einer großen Anzahl von GPUs.

Darüber hinaus stellt das Ensemble-Learning einen Ansatz dar, bei dem mehrere Modelle unabhängig voneinander trainiert und ihre Vorhersagen kombiniert werden, um die Gesamtleistung zu verbessern. Durch die parallele Ausführung des Trainings der einzelnen Modelle kann Ensemble-Learning effizient umgesetzt werden, was insbesondere bei rechenintensiven Aufgaben vorteilhaft ist. Wobei Ensemble-Learning eher dem Meta-Learning zugeschrieben, jedoch zur Vollständigkeit erwähnt wird.[4]

2.3. Datengetriebene Wettermodelle

Traditionelle Wettervorhersagemodelle, wie numerische Wettervorhersagemodelle (Numerical Weather Prediction, NWP), basieren auf physikalischen Gleichungen, die die Dynamik der Atmosphäre beschreiben. Diese Modelle simulieren komplexe Prozesse durch die Lösung von partiellen Differentialgleichungen (partial differential equations, PDE) auf Basis physikalischer Gesetze wie der Navier-Stokes-Gleichungen [10]. Derartige Berechnungen sind jedoch extrem rechenintensiv, erfordern leistungsstarke Hochleistungsrechner und können aufgrund notwendiger Approximationen physikalischer Prozesse systematische Fehler aufweisen.[40, 47]

In den letzten Jahren haben datengetriebene Wettermodelle aufgrund von Fortschritten in Deep Learning erheblich an Bedeutung gewonnen. Diese Modelle verzichten weitgehend auf explizite physikalische Gleichungen und stützen sich stattdessen auf die Analyse großer Mengen historischer Wetterdaten, um Muster zu erlernen und Vorhersagen zu treffen [57]. Durch diese datenbasierte Herangehensweise können sie Wettervorhersagen erheblich schneller berechnen und haben das Potenzial, verborgene Zusammenhänge aufzudecken, die in physikbasierten Modellen nicht direkt abgebildet sind. Wichtige Datensätze, die häufig in datengetriebenen Modellen verwendet werden, sind beispielsweise ERA5 (European Centre for Medium-Range Weather Forecasts Reanalysis v5) [27], GFS (Global Forecast System) [22] und CFSR (Climate Forecast System Reanalysis) [15, 51].

2.3.1. Herausforderungen datengetriebener Wettermodelle

Trotz beeindruckender Fortschritte stehen datengetriebene Wettermodelle vor einigen Herausforderungen, die ihre Leistungsfähigkeit einschränken können:

- **Extremwetterereignisse:** Künstliche Intelligenz (KI)-Modelle haben Schwierigkeiten, seltene oder extreme Wetterereignisse präzise vorherzusagen, da diese in Trainingsdaten häufig unterrepräsentiert sind [43].

- **Extrapolation:** Die Modelle sind anfällig für Unsicherheiten bei der Vorhersage von Bedingungen, die stark von den Trainingsdaten abweichen, etwa in Klimawandel-Szenarien [7].

- **Räumliche Auflösung:** Obwohl datengetriebene Modelle schnell sind, erreichen sie derzeit noch nicht die räumliche Auflösung von klassischen numerischen Modellen [18].

- **Physikalische Konsistenz:** Die Gewährleistung der Konsistenz zwischen vorhergesagten physikalischen Variablen stellt eine Herausforderung dar, da KI-Modelle nicht explizit physikalische Gesetze erzwingen [8].

- **Rechenaufwand:** Trotz ihrer Effizienz beim Inferenzprozess erfordert das Training großer KI-Modelle erhebliche Rechenressourcen.

2.3.2. Beispiele datengetriebener Wettermodelle

Im Folgenden werden drei führende datengetriebene Wettermodelle vorgestellt, die unterschiedliche Architekturen und Ansätze zur Wettervorhersage nutzen: GraphCast, Pangu-Weather und Fourier Forecasting Neural Network (FourCastNet).

GraphCast (Google DeepMind). GraphCast nutzt eine auf graphbasierte Architektur zur Wettervorhersage. Das Modell verarbeitet Daten von 37 atmosphärischen Ebenen und erstellt präzise 10-Tage-Vorhersagen mit einer räumlichen Auflösung von 0,25°. Durch die graphbasierte Struktur kann das Modell die komplexen Zusammenhänge zwischen verschiedenen atmosphärischen Parametern effizient erfassen. In umfangreichen Tests übertrifft GraphCast das hochauflösende Integrated Forecasting System (IFS) des Europäischen Zentrums für mittelfristige Wettervorhersage (ECMWF) in vielen Regionen und Vorhersagezeiträumen.[32, 33]

Pangu-Weather (Huawei). Pangu-Weather basiert auf einer Vision-Transformer- Architektur und verwendet Daten von 13 atmosphärischen Ebenen. Auch dieses Modell erstellt 10-Tage-Vorhersagen mit einer Auflösung von 0,25°. Eine Besonderheit von Pangu-Weather ist seine Transparenz und Zugänglichkeit: Das Modell ist auf GitHub verfügbar, und Huawei teilt regelmäßig Live-Vorhersagen, was zur Weiterentwicklung und breiteren Anwendung beiträgt.[6, 5, 58]

FourCastNet (NVIDIA). FourCastNet kombiniert die Leistungsfähigkeit von Transformern mit spektralen Techniken wie den Adaptive Fourier Neural Operatorn. Das Modell verarbeitet Wetterdaten von 20 atmosphärischen Ebenen und generiert ebenfalls 10-Tage-Vorhersagen bei einer Auflösung von 0,25°. Durch die Nutzung von AFNOs ist FourCastNet in der Lage, sowohl großskalige atmosphärische Dynamiken wie Jetstreams als auch kleinräumige Strukturen wie Wirbel zu erfassen und vorherzusagen.[31, 46]

2.4. Adaptive Fourier Neural Operators und FourCastNet

FourCastNet ist ein leistungsstarkes, datengetriebenes Modell für Wettervorhersagen, das ohne die explizite Lösung physikalischer Gleichungen auskommt. Stattdessen kombiniert es moderne NN mit spektralen Techniken, um hochdimensionale meteorologische Daten effizient zu verarbeiten und präzise Vorhersagen zu erzeugen. FourCastNet basiert auf den Prinzipien der Fourier Neural Operators (FNOs), die durch die Transformation von Daten in den Frequenzraum räumliche Muster erkennen. Dies ermöglicht eine Modellierung kontinuierlicher Felder, wie sie in Wettervorhersagen üblich sind. Durch die Fähigkeit, großskalige Wetterphänomene wie Jetstreams und kleinräumige Ereignisse wie Wirbel abzubilden, bietet FourCastNet eine vielversprechende Alternative zu traditionellen numerischen Wettermodellen.[31, 46]

2.4.1. FourCastNet Architektur

Die Architektur von FourCastNet basiert auf einer Vision-Transformer-Struktur, wobei anstelle der klassischen Attention-Heads AFNO eingesetzt werden. Die in Abbildung 2.1 dargestellte Implementierung folgt dem Originalpaper [46, 45], während Nvidia in ihrer Plattform Modulus eine leicht modifizierte Variante vorstellt [31, 11], die im Kapitel 5 erläutert wird.

Der Datenfluss im FourCastNet beginnt mit der Codierung der Eingabedaten durch Patch- und Positional-Embeddings. Das Patch-Embedding teilt die Eingabedaten X mithilfe von Faltungen in kleinere Unterregionen, sogenannte „Patches“, auf. Da bei diesem Prozess die ursprüngliche räumliche Struktur der Daten verloren gehen könnte, fügt das Positional-Embedding explizite Positionsinformationen hinzu, um die Anordnung der Patches im Netzwerk zu bewahren. Ein nachgeschaltetes Dropout verhindert Overfitting (dt. Überanpassung), indem es während des Trainings zufällig Neuronen deaktiviert.

Im Kern der Architektur durchlaufen die Daten mehrere AFNO- bzw. Encoder-Blöcke, die spektrale und nichtlineare Transformationen kombinieren. Zunächst werden die Eingaben normalisiert, um Stabilität in den Berechnungen zu gewährleisten. Anschließend erfolgt im AFNO-Layer das Token-Mixing mittels der Fast Fourier Transformation (FFT), die die Eingabedaten in den Frequenzraum überträgt. Aufgrund der hermiteschen Symmetrie der FFT, der Eigenschaft einer komplexen quadratischen Matrix gleich ihrer Adjungierten zu sein [26], reicht es aus, nur eine Hälfte der symmetrischen Paare zu verarbeiten, wodurch der Rechenaufwand signifikant reduziert wird.

Im Frequenzraum findet das Spatial-Mixing (dt. die räumliche Durchmischung) der Daten durch das Fourier-Layer statt. Hierbei werden die Informationen durch Matrixmultiplikationen und Aktivierungsfunktionen verarbeitet, bevor sie im Token Demixing-Schritt mittels der inversen FFT zurück in den geometrischen Raum transformiert werden. Die Ausgabe des Spatial-Mixings wird erneut normalisiert und an das Feed-Forward-Netzwerk des AFNO-Layers übergeben, das für das Channel-Mixing zuständig ist. Residualverbindungen, die die ursprünglichen Eingaben mit den transformierten Ausgaben addieren, sorgen für einen ungehinderten Fluss der Informationen durch das Netzwerk. Dies stabilisiert das Training tiefer Netzwerke, da der Gradientenfluss verbessert und das Problem der saturierenden Gradienten reduziert wird.

Nach den Encoder-Blöcken folgt der Decoder, der die Ausgabedaten mittels eines linearen Layers (im Code „Head“ genannt) auf die gewünschte Dimensionsgröße verdichtet.

Um die globale Kohärenz der Daten zu bewahren, fügt das FourCastNet ein periodisches Padding hinzu. Dies stellt sicher, dass die Ränder der Daten zyklisch behandelt werden, was insbesondere für meteorologische Vorhersagen von Bedeutung ist, um die räumliche Kontinuität über geografische Grenzen hinweg zu erhalten. Schließlich werden die berechneten Features durch eine Aktivierungsschicht geführt, um die finalen Vorhersagen für meteorologische Größen wie Windgeschwindigkeit und Niederschlag zu erzeugen.

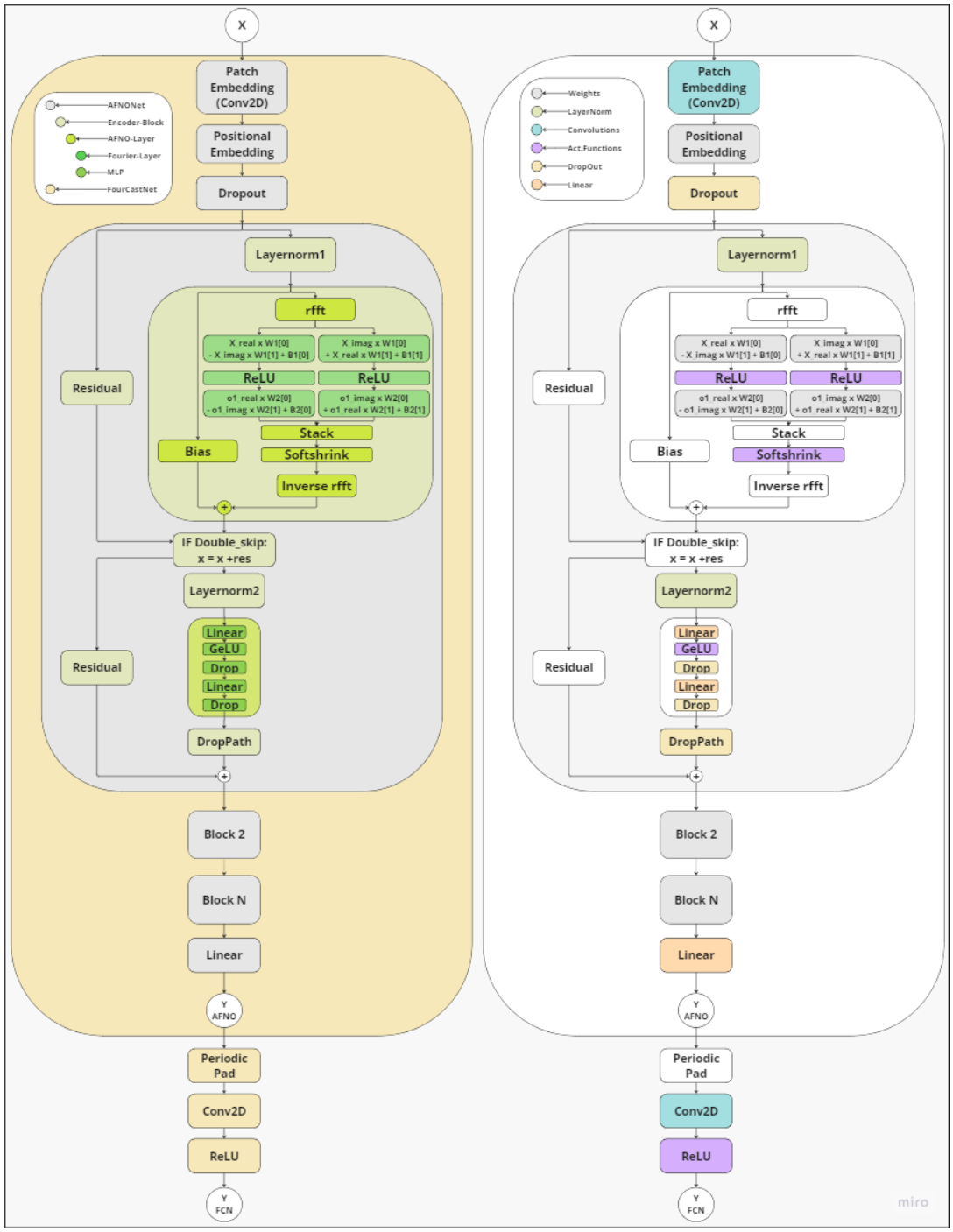


Abb. 2.1.: Eigene Darstellung der Architektur von FourCastNet, basierend auf dem Code des Originalpapers [46], erstellt in Miro. Auf der linken Seite ist das Model nach den implementierten Modulen untergliedert: FourCastNet in gelb, das AFNONet-Backbone in grau, der Inhalt der Encoder-Blöcke des AFNONet in hellem Grün, dem AFNO für das Spatial-Mixing in limettengrün, mit Fourier-Schicht in minzgrün und das MLP, zuständig für das Channel-Mixing, in dunkelgrün. Auf der rechten Seite erfolgt die Unterteilung nach Layer-Typen, wie Convolutional-Layern in petrol oder Aktivierungsfunktionen in lila.

2.4.2. Mathematische Grundlagen der Adaptive Fourier Neural Operators

Adaptive Fourier Neural Operator sind eine Weiterentwicklung der Fourier Neural Operator (FNO), die ursprünglich zur effizienten Lösung von PDEs entwickelt wurden. Bevor auf die mathematischen Details der AFNO eingegangen wird, folgt zunächst eine Erläuterung zu Neural Operators (dt. Neuralen Operatoren).

Neurale Operatoren. Neurale Operatoren ermöglichen es, Abbildungen zwischen unendlichdimensionalen Funktionalräumen, auch Banachräumen, zu approximieren.[37]

Banachräume sind vollständig normierte Räume $(\mathbf{X}, ||\cdot||)$, also ein normierter Vektorraum $\mathbf{X}$ über einem Körper, etwa den reellen oder komplexen Zahlen, in dem jede Fundamentalfolge aus Elementen des Raums einen Grenzwert besitzt der ebenfalls im Raum liegt.[3]

Im Gegensatz zu klassischen neuronalen Netzen, die auf Abbildungen zwischen endlichdimensionalen Vektorräumen arbeiten, eignen sich Neural Operators insbesondere zur Lösung von PDEs. Dabei approximieren sie Lösungsoperatoren, ohne explizit auf die zugrunde liegenden Gleichungen angewiesen zu sein. Dies bedeutet, dass Neural Operators eine universelle Parametrisierung von Operatoren erzeugen, die unabhängig von der Auflösung oder Diskretisierung der Daten funktioniert .[39, 37]

Sei $D \subset \mathbb{R}^d$ eine offene, begrenzte Menge und $\mathcal{A} = \mathcal{A}(D; \mathbb{R}^{d_a})$ sowie $\mathcal{U} = \mathcal{U}(D; \mathbb{R}^{d_u})$ separierbare Banachräume von Funktionen. Weiterhin sei v_j für $j = 0, 1, ..., T-1$ eine Sequenz von Funktionen $v_0 \mapsto v_1 \mapsto ... \mapsto v_T$ welche Werte in $\mathbb{R}^{d_v}$ annehmen. Die gesuchte Abbildung $G^\dagger : \mathcal{A} \mapsto \mathcal{U}$ stellt den Lösungsoperator einer parametrisierten PDE dar.[37] Neural Operators approximieren $G^\dagger$ durch eine parametrisierte Funktion $G_\theta : \mathcal{A} \to \mathcal{U}$ mit $\theta \in \Theta$ [38]:

$$G_\theta \approx G^\dagger. \tag{2.2}$$

Hierbei wird ein Input $a \in \mathcal{A}$ in eine höhere Repräsentation v_0 durch ein Netz P transformiert [38]:

$$v_0(x) = P(a(x)). \tag{2.3}$$

Die Repräsentation wird iterativ durch Kernel-Operatoren v_t aktualisiert [38, 37]:

$$v_{t+1}(x) := \sigma\left(W v_t(x) + (\mathcal{K}(a;\phi)v_t)(x)\right), \forall x \in D, \tag{2.4}$$

wobei $\mathcal{K}$ ein Kerneloperator ist, W eine lineare Transformation und σ eine nicht-lineare Aktivierungsfunktion. Die Kernelintegraltransformation ist gegeben durch [37]:

$$(\mathcal{K}(a;\phi)v_t)(x) := \int_D \kappa(x, y, a(x), a(y); \phi) v_t(y)\, dy, \forall x \in D, \tag{2.5}$$

wobei κ durch ein neuronales Netz parametrisiert wird. Die Ausgabe des Neural Operators $u(x) = Q(v_T(x))$ ist die Projektion von v_T durch die Transformation von einem weiteren Netz $Q : \mathbb{R}^{d_v} \to \mathbb{R}^{d_u}$.[37]

Fourier Neural Operators (FNO). FNOs basieren auf der Idee, die Daten in den Frequenzraum zu transformieren, um dort komplexe räumliche Interaktionen effizient darzustellen. Die Fouriertransformation $\mathcal{F}$ und ihre Inverse $\mathcal{F}^{-1}$ für eine Funktion $f : D \to \mathbb{R}^{d_v}$ sind gegeben durch [37]:

$$(\mathcal{F}f)_j(k) = \int_D f_j(x) e^{-2i\pi\langle x,k\rangle} dx, \quad (\mathcal{F}^{-1}f)_j(x) = \int_D f_j(k) e^{2i\pi\langle x,k\rangle} dk. \tag{2.6}$$

Unter Anwendung des Faltungstheorems [42] folgt der Kerneloperator im Fourierraum [37]:

$$(\mathcal{K}(\phi)v_t)(x) = \mathcal{F}^{-1}\left(\mathcal{F}(\kappa_\phi) \cdot \mathcal{F}(v_t)\right)(x), \forall x \in D. \tag{2.7}$$

Hierbei ist $\kappa_\phi : \mathbb{R}^{2(d+d_a)} \to \mathbb{R}^{d_v \times d_v}$ ein durch $\phi \in \Theta_\mathcal{K}$ parametrisiertes Netz.[37]

Adaptive Fourier Neural Operators (AFNO). Im Gegensatz zu klassischen FNOs werden AFNOs nicht zur Approximation von PDEs sondern in Transformern zur Bild- und Sequenzverarbeitung für das Spatial-Mixing genutzt. AFNO erweitert FNO durch mehrere Verbesserungen, um die Effizienz und Anpassungsfähigkeit zu steigern.[24]

Für einen Input-Token-Tensor $X \in \mathbb{R}^{h \times w \times d}$, der auf einem diskreten Gitter definiert ist (z.B. Bilddaten), wird der Gewichtstensor W durch die diskrete Fouriertransformation (DFT) parametrisiert $W := DFT(\kappa) \in \mathbb{C}^{h \times w \times d \times d}$, hierbei steht h für die Höhe, w für die Breite und d für die Anzahl der Kanäle bzw. Dimensionen der Daten.[24] In FNOs erfolgt das Token-Mixing in drei Schritten [24]:

1. Token-Mixing: Die Fouriertransformation der Eingabedaten [24]:

$$z_{m,n} = [DFT(X)]_{m,n}, \quad \forall (m,n) \in [h] \times [w]. \tag{2.8}$$

2. Channel-Mixing: Multiplikation der Fouriertransformierten mit einem Gewichtstensor W [24]:

$$\tilde{z}_{m,n} = W_{m,n} z_{m,n}. \tag{2.9}$$

3. Token-Demixing: Rücktransformation der transformierten Daten mittels inverser Fouriertransformation [24]:

$$y_{m,n} = [IDFT(\tilde{z})]_{m,n}. \tag{2.10}$$

Da die direkte Parametrisierung des Gewichtstensors W zu einer hohen Komplexität von $O(Nd^2)$ führt, verwenden AFNOs eine Blockdiagonalstruktur für die Gewichte. Diese Struktur teilt die Gewichte W in k kleinere Blöcke der Größe $d/k \times d/k$ auf und kann als Äquivalent zur Multi-Head-Self-Attention interpretiert werden.[24] Der Kernel arbeitet hierdurch unabhängig auf jedem Block [24]:

$$\tilde{z}^{(l)}_{m,n} = W^{(l)}_{m,n} z^{(l)}_{m,n}, \quad l = 1, \ldots, k. \tag{2.11}$$

Eine weitere Erweiterung durch AFNO ist das Weight-Sharing. Statt statischer Gewichte werden hier Gewichte durch ein Zwei-Layer-Perzeptron parametrisiert, dessen Gewichte W_1, W_2 und b über alle Token geteilt werden [24]:

$$\tilde{z}_{m,n} = MLP(z_{m,n}) = W_2 \sigma(W_1 z_{m,n}) + b. \tag{2.12}$$

Zuletzt setzt AFNO auf eine adaptive Auswahl der wichtigsten Frequenzbereiche im Fourierraum. Da Bilddaten im Frequenzraum typischerweise sparsity (dt. Dünn- oder Schwach-besetztheit) aufweisen (d.h., die meisten Informationen konzentrieren sich in den niedrigen Frequenzen), wird eine LASSO-Regularisierung $\min \|\tilde{z}_{m,n} - W_{m,n} z_{m,n}\|^2 + \lambda \|\tilde{z}_{m,n}\|_1$ [54] genutzt, um irrelevante Frequenzen zu sparsifizieren.[24]

Dieses Minimierungsproblem wird durch eine Soft-Thresholding-Operation gelöst [24]:

$$\tilde{z}_{m,n} = S_\lambda(W_{m,n} z_{m,n}), \quad S_\lambda(x) = \operatorname{sign}(x) \max\{|x| - \lambda, 0\}. \tag{2.13}$$

Hierbei ist λ ein Regularisierungsparameter, der die Sparsity reguliert.[24]

2.4.3. Parallelisierung in FourCastNet

Gegenwärtig existieren zwei verschiedene Implementationen von FourCastNet, zum einen der Code zum Originalpaper [46] aus 2022 und zum anderen eine überarbeitete Version zum Paper (FourCastNet: Accelerating Global High-Resolution Weather Forecasting using Adaptive Fourier Neural Operators) [31] aus Juni 2023, die seit der Veröffentlichung von Nvidias KI-Plattform Modulus im ersten Quartal 2024 öffentlich zugänglich ist. Im Originalpaper wurde lediglich Datenparallelismus verwendet [45], das zweite Paper nutzte „Modellparallelsimus [...] durch das Aufteilen der Channel (Feature-Abbildungen) über die GPUs“ [31], verteilte Matrixmultiplikation im MLP (Reihenweise in der ersten Schicht sowie Spaltenweise in der Zweiten), sowie Distributed Dataparallel (DDP) [11].

3. Forschungsmethoden und Versuchsaufbau

Dieses Kapitel beschreibt die Rahmenbedingungen des Experiments sowie die methodischen Überlegungen und Schritte zur Implementierung des modelparallelen AFNONets. Es werden die technischen Voraussetzungen, das experimentelle Vorgehen und die Überlegungen zur Anpassung des Modells detailliert dargestellt, um die Grundlage für die spätere Analyse der Ergebnisse zu schaffen.

3.1. Rahmenbedingungen der Experimente

Die Experimente werden auf dem Hochleistungsrechner „Horeka“ des Karlsruher Instituts für Technologie durchgeführt. Hierdurch entstehen die Einschränkungen und Eigenschaften der Rechenressourcen, die nachfolgend aufgeführt werden [52]:

- **GPU:** Die verfügbaren Grafikprozessoren umfassen NVIDIA A100 SXM mit 40 GB und H100 SXM mit 94 GB Grafikspeicher.
- **Rechenknoten:** Jeder Rechenknoten des Horeka-Systems ist mit maximal vier GPUs ausgestattet.
- **Kommunikation:**
  - **Intranode**: Die Kommunikation zwischen GPUs innerhalb eines Knotens erfolgt über NVLink, unterstützt durch NCCL (NVIDIA Collective Communication Library) mit einer Bandbreite von $600 GB/s$.
  - **Internode**: Die Kommunikation zwischen verschiedenen Knoten erfolgt über InfiniBand, gesteuert durch MPI (Message Passing Interface), mit einer maximalen Bandbreite von $200 GB/s$.
- **Lease-Zeit:** Die Nutzung der Rechenknoten ist auf eine maximale Laufzeit von 48 Stunden pro Anfrage beschränkt.

Zusätzlich zu den infrastrukturellen Rahmenbedingungen orientieren sich die Experimente methodisch an der Vorgehensweise des Originalpapers. Wie in Abschnitt 1.1 erläutert, liegt der Fokus der Untersuchung auf der Skalierbarkeit des AFNONets, das als Backbone für das FourCastNet dient. Daher wird auf ein Finetuning durch Two-Step-Training verzichtet, da das Ziel der Experimente primär die Analyse des Trainingsverhaltens ist.

Die wesentlichen Modell- und Trainingsparameter basieren auf den Vorgaben des Originalpapers [46, 45] und wurden wie folgt angepasst, um die Einschränkungen des Horeka-Clusters zu berücksichtigen:

- **Optimizer:** Der Adam-Optimizer wird für die Optimierung verwendet.
- **Loss:** Der L2-Loss dient als Optimierungsziel.
- **Epochen:** Aufgrund der maximalen Laufzeitbeschränkung auf 48 Stunden wird das Training auf 50 Epochen reduziert (im Vergleich zu den ursprünglich vorgesehenen 80 Epochen).
- **Input-Auflösung:** Die Eingabedaten werden auf eine Rasterauflösung von 720×1440 heruntergesampelt.
- **Input-Dimensionalität:** Die Eingabedaten umfassen 20 Kanäle, wie im Originalpaper und in Abschnitt 3.4 beschrieben.
- **Patch-Size:** Die Token werden auf eine Größe von 8×8 festgelegt.
- **Batch-Size:** Die Batch-Größe beträgt maximal 64, wodurch im DDP-Fall eine GPU pro Sample verwendet wird.
- **Diagonalblöcke:** Für die Fourier-Layer werden acht Diagonalblöcke verwendet, das Analog des Modells zu Attention-Heads.
- **AFNO-Layer:** Das Modell wird mit einer Tiefe von 12 Encoder-Blöcken initialisiert.

Alle Experimente basieren auf den NVIDIA A100-Grafikprozessoren, um eine Vergleichbarkeit mit der methodischen Grundlage des Originalpapers sicherzustellen.

3.2. Kommunikation und Speicherlayouts

Die effiziente Interprozesskommunikation spielt eine zentrale Rolle bei der Verteilung und dem Austausch von Daten zwischen verschiedenen Rechenknoten. Wie zuvor beschrieben, erfolgt die Kommunikation über zwei Haupttechnologien: MPI über InfiniBand für die Internode-Kommunikation (Kommunikation zwischen zwei Rechenknoten) und NCCL über NVLink für die Intranode-Kommunikation (Kommunikation innerhalb eines Rechenknotens). Eine fehlerhafte Konfiguration der genutzten Systemmodule kann zu erheblichen Problemen an der Schnittstelle zwischen diesen beiden Technologien führen, die sowohl die Stabilität als auch die Effizienz der Datenübertragung beeinträchtigen.

MPI und mpi4py. Die Internode-Kommunikation erfolgt mittels des Message Passing Interface (MPI), das speziell für hochperformante Datenübertragungen in verteilten Systemen entwickelt wurde. In der Implementierung wird die Python-Bibliothek mpi4py verwendet, die eine einfache und effektive Schnittstelle für die Nutzung von MPI bietet [14].

NCCL und torch.distributed. Für die Intranode-Kommunikation zwischen GPUs eines Rechenknotens wird die NVIDIA Collective Communications Library (NCCL) eingesetzt. Diese ermöglicht Datenübertragung zwischen GPUs über NVLink. In Kombination mit dem OpenMPI-Backend wird NCCL über die Python-Bibliothek torch.distributed [2] genutzt, um die Kommunikation innerhalb des Modells zu koordinieren.

Kommunikationsmuster und Herausforderungen. Die Anzahl und Art der Kommunikation hängt maßgeblich vom verwendeten Kommunikationsmuster sowie der Modell-Architektur ab. Wir verwenden One-to-All bzw. All-to-One-Kommunikation, bei der ein Prozess mit jedem anderen Prozess kommuniziert bzw. alle Prozesse Daten an einen Prozess senden. Diese Muster erfordern jeweils $P-1$ Kommunikationsvorgänge pro Operation (wobei P die Anzahl der Prozesse ist). Für eine optimale Leistung ist es entscheidend, dass die zu übertragenden Tensoren contiguous (dt. zusammenhängend, kontiger) im Speicher der GPU vorliegen [4]. Wenn Tensoren über non-contiguous Dimensionen verteilt werden sollen (z. B. bei Scatter- oder Gather-Operationen), führt dies zu zusätzlichen Speicheroperationen wie Reorganisation, was weiteren Overhead (dt. Zusatzaufwand) bedeutet [4]. Weiterhin können hierbei Probleme auftreten: Scatter-Operationen teilen Tensoren in Teilstücke auf, die an verschiedene Prozesse oder Geräte gesendet werden. Dabei greifen sie direkt auf die physische Speicherrepräsentation des Tensors zu. Bei non-contiguous Tensoren entspricht diese interne Darstellung jedoch oft nicht der logischen Anordnung der Dimensionen. Abbildung 3.1 dient der Verdeutlichung der Bedeutung contiguous.

Dies führt zu erhöhten Latenzzeiten, da zusätzliche Operationen zur Umwandlung der Tensoren in eine zusammenhängende Darstellung erforderlich sind. Sollten diese nicht stattfinden, wie es z.B. im nativen torch.distributed der Fall ist (hierzu muss die Funktion *.contiguous*() manuell aufgerufen werden), können Daten falsch verteilt werden oder Dimensionen eine falsche Menge an Werten erhalten.

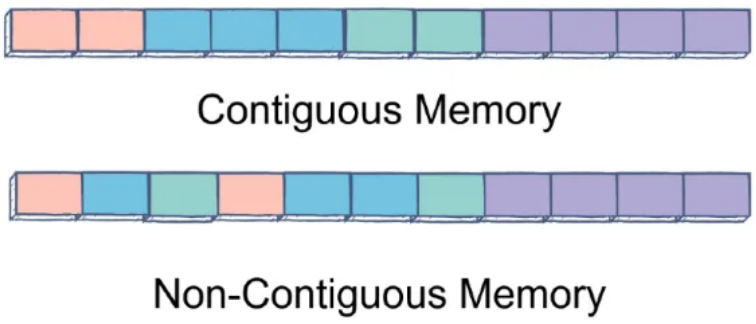


Abb. 3.1.: Darstellung von zusammenhängendem und nicht zusammenhängendem Speicher. Diese Abbildung ist zu finden in [21].

3.3. Metriken

In dieser Arbeit werden neben den üblichen Loss-Metriken spezifische Zielmetriken definiert, um die Leistung des Systems zu bewerten. Zu den wichtigsten Metriken zählen die starke und schwache Skalierung, Effizienz und der Kommunikations-Overhead.

Skalierung. Die Skalierbarkeit eines Systems bezeichnet die Fähigkeit, mit zunehmender Rechenkapazität Probleme proportional schneller zu lösen. Dabei wird zwischen starker Skalierung und schwacher Skalierung unterschieden. Die starke Skalierung beschreibt, wie gut ein Problem konstanter Größe mit zusätzlichen Ressourcen in kürzerer Zeit gelöst werden kann. Sie wird häufig durch den Speedup S definiert, der das Verhältnis der Ausführungszeiten eines sequentiellen Systems (T_1) und eines parallelen Systems mit P Prozessen (T_p) darstellt [28]:

$$S = \frac{T_1}{T_p}. \tag{3.1}$$

Weiterhin gilt mit Amdahls Gesetz [1], welches beschreibt, wie Programme durch Overhead wie Datenmanagement immer einen nicht parallelisierbaren Teil enthalten, dass der Speedup höchstens linear sein kann. Die Laufzeit eines Programms lässt sich wie folgt darstellen:

$$T_1 = T_{seriell} + T_{parallel} \tag{3.2}$$

In einem parallelen System mit Parallelisierungsgrad P ergibt sich:

$$T_p \geq T_{seriell} + T_{parallel}/P, \tag{3.3}$$

Im Gegensatz zur starken Skalierung beschreibt die schwache Skalierung die Fähigkeit eines Systems, ein zu den zusätzlichen Ressourcen proportional größeres Problem in gleicher Zeit zu lösen. Dies ist besonders relevant für datenintensive Probleme wie sie in wissenschaftlicher Forschung oder KI-Anwendungen auftreten. Hierbei vergleicht schwache Skalierung die Zeit, die ein nicht parallelisiertes System zur Lösung des größeren Problems hypothetisch benötigen würde, mit der Zeit die das parallele System benötigt. Vorgeschlagen von John L. Gustafson, wird der Speedup für die schwache Skalierung wie folgt berechnet [25]:

$$S_{skaliert} = \frac{T_{seriell} + P \cdot T_{parallel}}{T_p} \tag{3.4}$$

Ein weiteres Ziel der schwachen Skalierung ist die Effizienz, also die Bewertung der Leistungssteigerung im Verhältnis zu den eingesetzten Ressourcen. Die Effizienz wird definiert als [25]:

$$E = \frac{S_{skaliert}}{P} \tag{3.5}$$

Jedoch ist ein Training auf einer einzelnen GPU nicht möglich, geschuldet den Limitationen aus Abschnitt 3.1. Daher wird T_1 approximiert durch:

$$\frac{B \times K \times E}{D_{P_{min}}} = \frac{N}{D_{P_{min}}} \approx T_1, \tag{3.6}$$

wobei $D_{P_{min}}$ der Durchsatz des Basline-Modells mit 8 GPU ($P_{min} = 8$) und N die Anzahl aller Samples $N = B \times K \times E$ im Trainingsdatensatz, B die Batch-Size, K die Anzahl an Batches pro Epoche und E die Anzahl an Epochen ist. Definiert über den Durchsatz:

$$D\left[\frac{Samples}{s}\right] = \frac{B\left[\frac{Samples}{Batch}\right] \times K\left[\frac{Batches}{Epoche}\right] \times E\,[Epochen]}{T\,[s]}, \tag{3.7}$$

folgt:

$$D_{P_{min}} = \frac{N}{T_{P_{min}}}. \tag{3.8}$$

Daraus ergibt sich mit $T_1 \approx T_{P_{min}}$ in Gleichung 3.1 eine relative, starke Skalierung:

$$S' = \frac{T_{P_{min}}}{T_p}, \tag{3.9}$$

in Gleichung 3.4 eine relative, schwache Skalierung:

$$S'_{skaliert} = \frac{P}{P_{min}} \frac{T_{P_{min}}}{T_p}, \tag{3.10}$$

und in Gleichung 3.5 eine relative Effizienz von:

$$E' = \frac{T_{P_{min}/T_p}}{P/P_{min}} \tag{3.11}$$

Durch diesen Ansatz wird der Durchsatz als eigene Metrik ebenfalls interessant.

Loss. Die Bewertung des Modells erfolgt primär durch den `relative L2-Loss`, wie in Abschnitt 3.1 beschrieben. Diese Metrik misst die Differenz zwischen den vorhergesagten Tensoren $\hat{Y} = Model(X)$ und den tatsächlichen Tensoren Y und ist daher auch als (ein skalierter) mean squared error (MSE) bekannt:

$$L2\text{-}Loss = \frac{||\hat{Y} - Y||_2^2}{||Y||_2^2} \tag{3.12}$$

Overhead. Zusätzlich zur Skalierung wird der durch Modellparallelismus verursachte Overhead bewertet. Dieser beeinflusst direkt die Effizienz und Skalierbarkeit eines Systems und umfasst:

- **Anzahl der Kommunikationen**: Abhängig von der verwendeten Architektur und den durchgeführten Operationen.
- **Datenvorverarbeitung**: Zusätzliche Vorverarbeitungen, um Tensoren für die Übertragung bereitzustellen (z. B. Umwandlung in contiguous Memory-Layouts).
- **Verbrachte Zeit in Kommunikationsoperationen**: Die Zeitdauer der Datenübertragung.
- **Verbrachte Zeit in Barrieren**: Verzögerungen durch Synchronisationsbarrieren, bei denen Prozesse aufeinander warten müssen.

Vertikale Ebenen	Variablen
Oberfläche	$U_{10}, V_{10}, T_{2m}, sp, mslp$
1000hPA	U, V, Z
850hPA	T, U, V, Z, RH
500hPA	T, U, V, Z, RH
50hPA	Z
Integriert	$TCWV$

Tabelle 3.1.: Auflistung der verwendeten Variablen aus ERA5. Die Abkürzungen bedeuten das folgende: $U_{10}(V_{10})$: zonale (meridonale) Windgeschwindigkeit in $10m$ Höhe über der Oberfläche; T_{2m}: Temparatur $2m$ über der Oberfläche; T,T(sic!)[U],V,Z.(sic!)[,]RH: Temperatur, zonale Geschwindigkeit, meridonale Geschwindigkeit, Geopotenzial, relative Feuchtigkeit in der jeweiligen vertikalen Ebene; $TCWV$: Gesamter Wasserdampf der Vertikalen. Diese Tabelle und ihre Beschreibung ist aus [46] übernommen.

3.4. Datensatz

Diese Arbeit verwendet, wie in der Forschungsgrundlage beschrieben, ein Subset des ECMWF reanalysis v5 (ERA5)-Datensatzes mit einer zeitlichen Auflösung von 6 Stunden. Der vollständige ERA5-Datensatz bietet eine räumliche Auflösung von $0.25° \times 0.25°$ und eine stündliche zeitliche Auflösung. Insgesamt enthält er über 300 Parameter, die aus 46 vertikalen Ebenen aufgezeichnet werden.[19, 27, 12] Aus diesen wurden in [46] 20 Variablen ausgewählt, die in Tabelle 3.1 aufgeführt sind.

Um auf den identischen Daten wie das Ursprungsmodell zu arbeiten, wurden die Jahre 1979-2018 des Datensatzes aus dem Copernicus Climate Data Store bezogen. Hierbei werden die Daten, durch ein Interpolations-Schema von Copernicus, von einem Gauß-Gitter auf ein reguläres Euklidisches-Gitter umgerechnet, wodurch sie als zweidimensionales Feld mit $721h \times 1440w$ Pixeln repräsentiert werden [31]. Jedes Jahr umfasst aufgrund des $6h$ Subsamplings insgesamt 1460 Samples. Diese wurden wie folgt in Trainings-, Validierungs- und Test-Daten unterteilt:

- **Training**: Daten aus den Jahren 1979-2015 (54020 Sample)
- **Validierung**: Daten aus den Jahren 2016-2017 (2920 Sample)
- **Test**: Daten aus dem Jahr 2018 (1460 Sample)

3.5. Überlegungen zur Modellparallelisierung

Im Rahmen dieser Arbeit wurde zunächst der Quellcode des Originalpapers eingehend analysiert, um die zugrunde liegende Modellarchitektur zu verstehen.

3.5.1. Strategien zur Modellparallelisierung

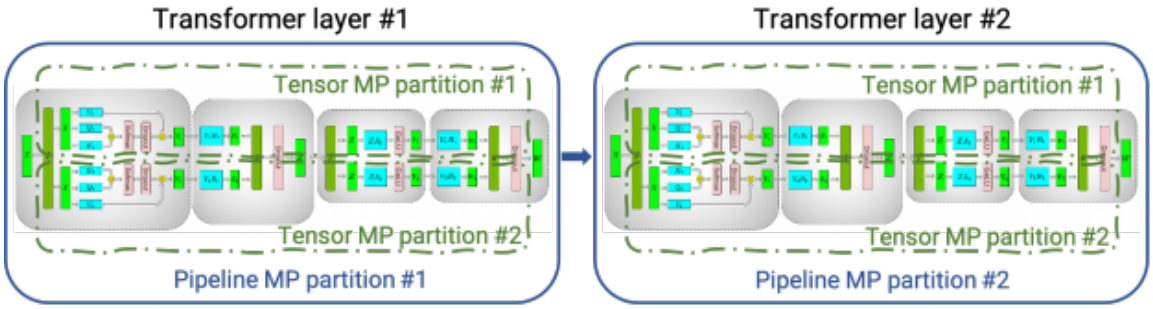
Basierend auf den Untersuchungen wurden zwei mögliche Strategien zur Parallelisierung des Modells identifiziert. Analog zum Ansatz von Megatron-LM kann die Parallelisierung über die Diagonalblöcke der Fourier-Layer erfolgen, wie im Originalpaper beschrieben. Alternativ kann eine künstliche Erweiterung der Modellkapazität durch eine Vergrößerung der Hidden-Dimension vorgenommen werden.

Megatron-LM. Megatron-LM ist ein Transformermodell für Anwendungen in der natürlichen Sprachverarbeitung. Es implementiert verschiedene Parallelisierungsstrategien, wie in Abbildung 3.2a zu erkennen ist, darunter Modellparallelität der Attention-Heads innerhalb des Encoders sowie der MLP-Schichten des Decoders und Pipelining zur Aufteilung der Transformerschichten.[53]

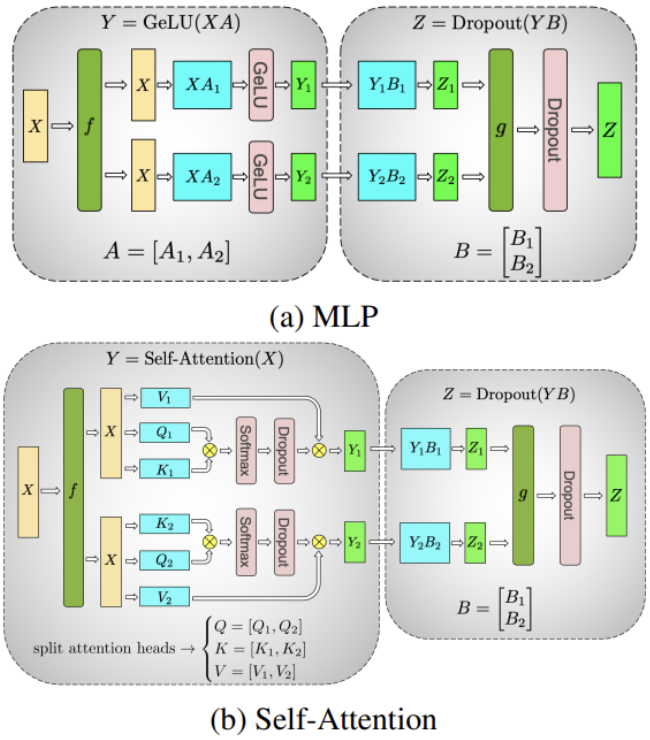
Im Kontext dieser Arbeit, eben durch den Fokus auf Modellparallelismus, wird das Pipelining durch eine Aufteilung des gesamten Encoders ersetzt.

Analyse des Modells Mit den in Abschnitt 3.1 beschriebenen Einstellungen, ergeben sich folgende Eigenschaften des Inputs der Fourier-Schichten des AFNO: $X \in \mathbb{R}^{b\times 90\times 180\times 768}$ bzw. $\mathcal{X} = \mathbb{C}^{b\times 90\times 180\times bl\times 768/bl}$ nach der Fourier-Transformation. Dabei bezeichnen b die Batch-Size und bl die Anzahl der Diagonalblöcke.

Eine modellparallele Umsetzung, wie sie im Originalpaper beschrieben wird, würde eine Aufteilung des fourier-transformierten Input-Tensors entlang der dritten Dimension bl erfordern, wobei die Diagonalblöcke gleichmäßig über die verfügbaren GPUs verteilt werden. Im weiteren werden diese GPU als modellparallele Gruppe (MP-Gruppe) bezeichnet. Bei der standardmäßigen Einstellung von $bl = 8$ ergibt sich eine maximale MP-Size, also ein maximaler Grad an Modellparallelität, von 8. Dies resultiert in einer Gruppierung der GPUs in datenparallele Einheiten mit je acht modellparallelen GPUs: $GPU_{DP} = \frac{GPU_{ges}}{GPU_{MP}}$



(a) Grobgranulare Aufteilung des Parallelismus in Megatron-LM.



(b) Vergrößerte Ansicht des MLP und der Self-Attention

Abb. 3.2.: Abbildungen übernommen aus [53]. Die Abbildungen zeigen die Umsetzung des Parallelismus in Megaton-LM. Abbildung A ist hierbei die grobgranulare Struktur, in der die Schichten des Transformers durch Pipelining und die einzelnen En- und Decoder über Tensorparallelismus verteilt werden. Abbildung B zeigt eine vergrößerte Ansicht der Encoder und Decoder. Weiterhin sind darin die Kommunikationsfunktionen `f` und `g` zu erkennen.

Zusätzlich zur Aufteilung entlang der Diagonalblöcke bl wurden folgende Alternativen untersucht:

1. **Aufteilung entlang der Batch-Dimension** b: Eine Aufteilung entlang der Batch-Dimension führt zu einem datenparallelen Ansatz, bei dem einzelne Batches unabhängig auf verschiedene GPUs verteilt werden. Dies ist der Standardfall für FourCastNet des Originalpapers.

2. **Aufteilung entlang der räumlichen Dimensionen** h **oder** w: Die Aufteilung entlang der räumlichen Dimensionen (90 bzw. 180 Elemente) würde zu einer sogenannten Domänenparallelität führen. Dieser Ansatz kann in CNNs angewandt werden. Aufgrund der vergleichsweise geringen Dimensionen in den Eingabedaten ist diese Methode jedoch weniger effizient und könnte zusätzliche Kommunikationskosten verursachen. Die Klassifikation als Modellparallelität ist in der Literatur umstritten [4].

3. **Aufteilung entlang der Hidden-Dimension vor der Fourier-Transformation**: Eine weitere Möglichkeit besteht darin, die Hidden-Dimension (768 Elemente) aufzuteilen. Diese Methode wurde in dieser Arbeit für die zweite Parallelisierungsstrategie, basierend auf Megatron, implementiert.

Eine dritte Parallelisierungsstrategie besteht in der Vergrößerung der Hidden-Dimension durch eine Erweiterung der Modellkapazität. Dies könnte durch die Einführung eines Single-Layer-Perzeptrons erreicht werden, das Input-Daten der Dimension $hidden_dim$ akzeptiert und Ausgabedaten der Dimension $MP \cdot hidden_dim$ generiert. Hierdurch würde die Hidden-Dimension des Modells um den Faktor MP skaliert, sodass jede GPU in der modellparallelen Konfiguration eine identische Datenmenge wie das ursprüngliche Modell verarbeitet. Aufgrund des zeitlich begrenzten Rahmens dieser Thesis konnte dieser Ansatz nicht mehr umgesetzt werden.

3.5.2. Lokale und globale Parallelisierungsansätze

Wie in Abbildung 3.3 dargestellt, kann das Modell an drei zentralen Stellen parallelisiert werden. Diese werden im Folgenden als lokale Parallelisierung (Mikro-Parallelisierung) und globale Parallelisierung (Makro-Parallelisierung) bezeichnet, wobei die Begriffe die Granularität der Parallelisierungsstrategie widerspiegeln.

Lokale Parallelisierung. Die lokale Parallelisierung, dargestellt in Abbildung 3.3, bezieht sich auf die Aufteilung von Blöcken innerhalb des AFNO-Layers, sowie eine potenzielle eigenständige Parallelisierung des MLP (Multi-Layer-Perceptron). Wie in der vorigen Sektion beschrieben, kann dies auf zwei Arten erfolgen. Zum einen über die Aufteilung der Hidden-Dimension vor der Fourier-Transformation und zum anderen über die Aufteilung der Diagonalblöcke nach der Fourier-Transformation. Diese Ansätze resultieren in unterschiedlichen Architekturen, die jeweils auf verschieden segmentierten Daten arbeiten.

Gegeben einem Tensor $A \in \mathbb{R}^{b \times h \times w \times hidden_dim}$, ergeben sich nach der Fourier- Transformation die folgenden Datenrepräsentationen:

- **Teilen der Diagonalblöcke**: $\mathcal{A} \in \mathbb{C}^{b \times h \times w \times bl/P \times hiddendim/bl}$,
- **Teilen der Hidden-dimension**: $\mathcal{A} \in \mathbb{C}^{b \times h \times w \times bl \times hidden_dim/(P \cdot bl)}$,

wobei P die Anzahl der modellparallelen GPUs darstellt.

Globale Parallelisierung. Die globale Parallelisierung betrifft die Aufteilung auf der Ebene des gesamten Modells, aufgezeigt in Abbildung 3.3 links.

Die Kommunikation erfolgt hierbei einmal beim Eintritt in die iterativen Blöcke durch Scatter-Operationen, um die Daten aufzuteilen und ein weiteres Mal beim Austritt aus der Encoder-Blocksequenz durch eine Allgather-Operation, wobei der Tensor rekombiniert wird. Dies entspricht einer Anpassung des Ansatzes von Megatron-LM ohne Pipelining. Diese Strategie minimiert die Kommunikation zwischen den GPUs während der Berechnung innerhalb der Blocksequenz, da die Daten nur einmal verteilt und synchronisiert werden müssen. Allerdings führt diese Strategie dazu, dass das Channel-Mixing nicht mehr identisch für alle Tokens ist, was vom Originalansatz abweicht.

3.5.3. Kommunikationsaufwand und Implementierungsdetails

Der Kommunikationsaufwand der beschriebenen Strategien hängt maßgeblich von den eingesetzten Funktionen und deren Implementierung im Framework ab. Hierbei müssen für das Training, analog zu Megatron-LM, Funktionen eingeführt werden die die Kommunikation im Forward- und Backwardpass übernehmen.

Im Forwardpass erfolgt die Tensorverteilung, durch die Scatter-Operation (Kommunikationsmuster der Operation ist One-to-All) und die Rekombination des Tensors durch eine Allgather-Operation die eine Kombination aus Gather (All-to-One) und Broadcast (One-to-All) darstellt. Der worst-case Kommunikationsaufwand für den Forwardpass beträgt $3(P-1)$ Kommunikationen pro Funktionspaar (Scatter/Allgather). Unter der Annahme einer effizienten Tree-basierten Implementierung (z. B. durch MPI oder Torch) ergibt sich jedoch ein Aufwand von $2log(P) + P - 1$, siehe Abbildung 3.4. Für den Backwardpass müssen die inversen Operationen des Forwardpasses verwendet werden. Das bedeutet Scatter wird zu Allgather, und Allgather wird zu Scatter. Nach dem Backwardpass folgt die Gradientensynchronisation über die DP-Gruppen (datenparallelen Gruppen). Hierbei wird ein Allreduce eingesetzt, welches, analog zu Allgather, die Kombination aus einer Reduce-Operation und einem Broadcast ist.

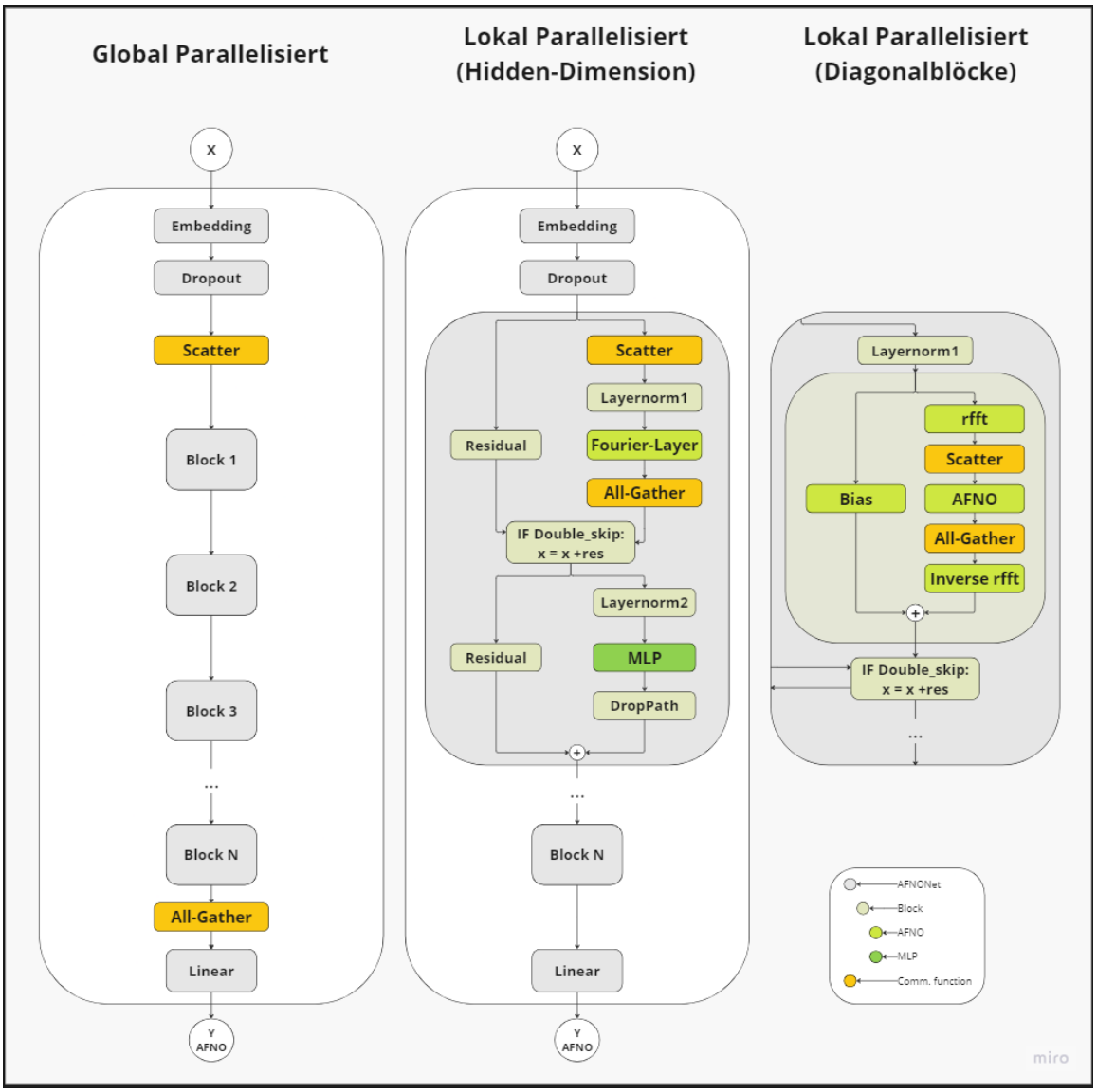


Abb. 3.3.: Visualisierung der Architekturen mit eingefügten Kommunikationsschichten (orange hervorgehoben). Auf der linken Seite ist die globale Parallelisierungsstrategie zu erkennen, bei der lediglich 2 Kommunikationsoperationen pro Forward und Backwardpass hinzukommen. Mittig sowie auf der rechten Seite sind die lokalen oder feingranularen Strategien abgebildet. Diese unterscheiden sich lediglich durch die Position sowie durch die Dimensionalität des zu kommunizierenden Tensors.

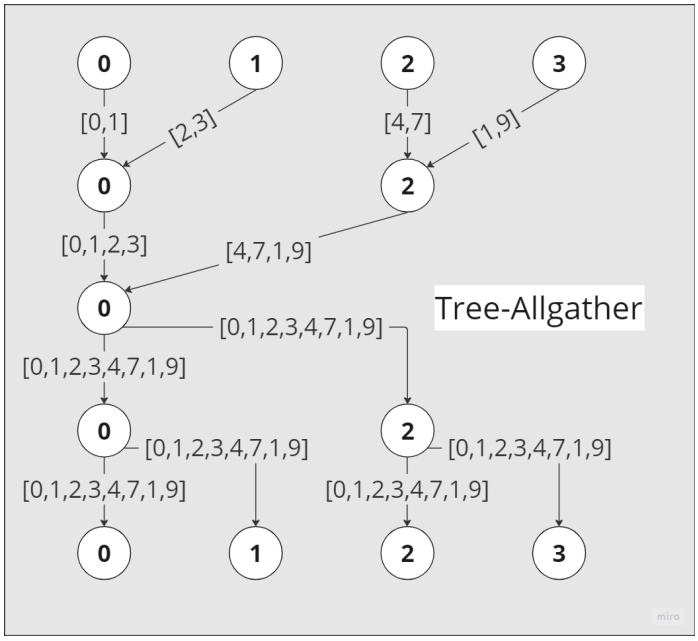


Abb. 3.4.: Eigene Darstellung eines Tree-Allgathers

Geisterneuronen Falls das zwei-Layer-MLP der AFNO-Blöcke, im Fall der lokalen Parallelisierung, über mehrere GPUs verteilt wird, müssen entweder andere Funktionen genutzt [4] oder sogenannte Geisterneuronen eingeführt werden, um Abhängigkeiten zwischen benachbarten Knoten lokal zu berechnen und Kommunikationsaufwand zu minimieren. Hierbei entstehen in diesem Fall $4 \cdot P$ Geisterneuronen. Die Kommunikation zwischen den Geisterneuronen besteht aus folgenden Teilen [56]:

- **Broadcast zwischen den Schichten**: Ein All-to-All Broadcast mit $P(P-1)$ Kommunikationen, um die Vollvernetztheit der Schichten zu gewährleisten.
- **Austausch zwischen benachbarten Neuronen**: Zwei One-to-One Kommunikationen, Send und Recive, zwischen benachbarten Neuronen. Für zwei Schichten mit je zwei Nachbarn pro Prozess ergibt sich: $8P$
- **Gather des Outputs**: Um den Tensor auf dem Root-Prozess der MP-Gruppe zu rekombinieren.

Im Backwardpass wird die Broadcast-Operation durch ein Allreduce ersetzt, was eine Kombination aus Reduktion und Synchronisation darstellt.

Kommunikationshäufigkeit der Parallelisierungsansätze. Folgend wird die Kommunikaitonskomplexität (Häufigkeit der Kommunikationsoperationen) für die verschiedenen Parallelisierungsansätze aus den oben dargestellten Eigenschaften hergeleitet. Dabei werden die Bestandteile des Forward- und Backwardpasses sowie die Gradientensynchronisation berücksichtigt. Zur Beschreibung der Kommunikationskosten seien P der Anzahl der beteiligten Prozesse und $Depth$ die Tiefe des Modells.

1. **Gradientensynchronisation**: Die Kommunikation der DP-Gruppen erfolgt durch eine Allreduce-Operation mit einem Kommunikationsaufwand von $2log(P_{DP})$.
2. **Lokale Parallelisierung ohne MLP**: In diesem Ansatz erfolgt die Parallelisierung ausschließlich innerhalb der Fourier-Layer.
   - **Forwardpass**: $Depth \cdot (P - 1 + 2log(P))$
   - **Backwardpass**: $Depth \cdot (2log(P) + P - 1)$
   - **Gesamt**: $2 \cdot Depth \cdot (P - 1 + 2log(P))$
3. **Lokale Parallelisierung mit MLP**: Dieser Ansatz ergänzt die Parallelisierung um das MLP mit Geisterneuronen.
   - **Forwardpass**:
     $Depth \cdot (P-1+2log(P)+P \cdot (P-1)+8 \cdot P+P-1) = Depth \cdot (P^2+9P-2+2log(P))$
   - **Backwardpass**:
     $Depth \cdot (2log(P) + P - 1 + 2log(P) + 8 \cdot P + P - 1) = Depth \cdot (10P - 2 + 4log(P))$
   - **Gesamt**: $Depth \cdot (P^2 + 19P - 4 + 6log(P))$
4. **Globale Parallelisierung**: Dieser Ansatz nach dem Vorbild von Megatron-LM reduziert die Kommunikation auf ein Minimum, indem Scatter- und Allgather-Operationen optimiert werden:
   - **Forwardpass**: $P - 1 + 2log(P)$
   - **Backwardpass**: $2log(P) + P - 1$
   - **Gesamt**: $2P - 2 + 4log(P)$

Tabelle 3.2 stellt die Ansätze nochmals gegenüber. Hierbei wird ersichtlich, dass die Einführung der Geisterneuronen durch den All-to-All Broadcast zu quadratisch steigender Kommunikation in P führt.

Ansatz	Kommunikationskomplexität					
Lokal ohne MLP	$Depth \cdot ($		$2P$	-2	$+4log(P))$	$+2log(P_{DP})$
Lokal mit MLP	$Depth \cdot ($	P^2	$+19P$	-4	$+6log(P))$	$+2log(P_{DP})$
Global			$2P$	-2	$+4log(P)$	$+2log(P_{DP})$

Tabelle 3.2.: Zusammenfassung der Kommunikationskomplexitäten der verschiedenen Ansätze

4. Implementation

Dieses Kapitel beschreibt die Implementierung der Experimente zur Evaluation der Skalierbarkeit modellparalleler AFNOs. Dabei werden sowohl die Anpassungen des Trainingsprozesses als auch die Implementierungsdetails modellparalleler Varianten nach Unterabschnitt 3.5.2 dargestellt. Zunächst werden die Skalierungsexperimente und deren Konfigurationen erläutert, gefolgt von notwendigen Modifikationen des Dataloaders, der Kommunikation für die Gradientensynchronisation und des Checkpointings.

4.1. Experimente

Wie in Abschnitt 3.1 beschrieben, werden die Experimente auf bis zu 64 GPUs ausgeführt. Aufgrund der Beschränkung des Originalmodels auf acht Diagonalblöcke ergibt sich ein Parallelisierungsgrad MP (zukünftig als `MP_Size` referenziert) von höchstens acht. Die Experimente wurden daher auf einer variablen Anzahl von GPUs N_{GPU} (8,16,32 und 64) durchgeführt, um die Skalierbarkeit unter verschiedenen Konfigurationen zu analysieren. Die Skalierbarkeit des Modells wurde durch die Regulierung der lokalen Batch-Size S_l untersucht. Diese Entscheidung ist begründet durch die Definitionen der Skalierbarkeiten. Durch eine lokale Batch-Size von 1 wird mehr Rechenkapazität, bis zu acht-fach, für das Lösen des gleichgroß bleibenden Problems (1 Batch pro Modell) verwendet, was der starken Skalierung entspricht. Gleichfalls wird, durch eine lokale Batch-Size gleich der MP-Size, ein größeres Problem (1 Batch pro GPU der MP-Gruppe) gelöst. Die Größe des Problems steigt proportional zu der aufgewendeten Rechenkapazität, was mit der Definition der schwachen Skalierung korrespondiert.

- **Starke Skalierung**: $S_g = \frac{N_{GPU} \cdot S_l}{MP}$ mit $S_l = 1$
- **Schwache Skalierung**: $S_g = \frac{N_{GPU} \cdot S_l}{MP}$ mit $S_l = MP$

S_g stellt hierbei die globale Batch-Size des Experiments dar. Weiterhin wurden Zeitlimits gesetzt die dem 1,5-fachen der Zeit entsprechen, die die Baseline benötigte.

4.2. Anpassung des Trainingscodes

Die Durchführung der Experimente erforderte Anpassungen des Trainingscodes.

Hierzu gehört der Dataloader, bei dem sichergestellt werden musste, dass innerhalb einer MP-Gruppe identische Trainingsdaten auf allen beteiligten GPU vorliegen. Dies wurde durch die Konfiguration der Variablen `world_size` und `rank` realisiert, wobei `world_size` die Gesamtzahl paralleler Prozesse und `rank` den Rang des ausführenden parallelen Prozesses beschreibt. Durch eine Integer-Division von `world_size` durch `rank` und den Term $world_rank - world_rank\%MP$ wurde festgelegt, dass `world_size` die Anzahl paralleler Modelle und `rank` den Rang des Root-Prozesses der MP-Gruppe wiederspiegelt.

Weiterhin musste die Kommunikation der Knoten organisiert werden. Die Initialisierung erfolgt durch eine Hilfsfunktion der HeAT-Bibliothek (Helmholz AI Tools), welche die Konfiguration von `torch.distributed` auf die spezifischen Anforderungen von Horeka anpasst. Anschließend wurde die Kommunikation in zwei Schemata unterteilt:

- **Datenparallele Kommunikation**: Dieses Schema synchronisiert die Gradienten zwischen GPUs mit identischen lokalen Rängen innerhalb einer MP-Gruppe, über alle MP-Gruppe hinweg ($0 \ldots n$, wobei n dem Grad der Parallelität entspricht). Der folgende Pseudocode illustriert die Zuordnung der Kommunikationsgruppen:

  for i in range(mp_size):
  if $world_rank\%mp_size == i$:
  Teil der Kommunikationsgruppe

- **Modellparallele Kommunikation**: Dieses Schema organisiert die Kommunikation aller lokalen Ränge innerhalb einer MP-Gruppe. Der Pseudocode für die Gruppenzuordnung lautet:

  $src_rank = world_rank - world_rank\%mp_size$
  if $world_rank$ in ($src_rank : src_rank + mp_size$):
  Teil der Kommunikationsgruppe

Die Aufteilung der datenparallelen Kommunikation erfolgt in dieser Weise, begründet durch die Verteilung der Gewichte des Modells. Hierbei dürfen ausschließlich GPUs mit identischen lokalen Rängen miteinander kommunizieren.

Zusätzlich musste die datenparallele Kommunikation des Loggings (dt. zur Protokollierung) angepasst werden. Beim Logging werden die Daten aller GPUs verwendet, um Desynchronisierungen zu vermeiden. Dazu werden die Loss-Werte zunächst durch den Grad der Parallelität geteilt, da diese Werte innerhalb einer MP-Gruppe identisch sind. Anschließend erfolgt eine Mittelung mittels Allreduce.

Schließlich wurde auch das Checkpointing, d. h. das Speichern und Laden von Modellparametern, an das neue Kommunikationsschema angepasst. Anstelle des Root-Prozesses mit Rang 0, welcher sämtliche Modellgewichte in einer Datei speichert oder lädt, generieren nun die Prozesse der Ränge 0 bis MP separate Dateien mit den Teilgewichten.

4.3. Implementation des Modellparallelismusses

In Abbildung 3.3 sind die unterschiedlichen Architekturen und deren Implementierungen dargestellt. Die hinzugefügten Kommunikationsschichten sind farblich in Orange hervorgehoben. Am linken Rand ist die Implementierung der globalen Parallelisierung abgebildet. In dieser Variante wird der Tensor beim Eintritt in die Blocksequenz entlang der Hidden-Dimension aufgeteilt. Dafür wird die benutzerdefinierte `Torch.autograd`-Funktion `ScatterGather` genutzt, die im Forward-Pass eine Scatter- und im Backward-Pass eine Allgather-Operation durchführt.

Der Ablauf dieser Kommunikationsfunktion ist in Pseudocode 1 dargestellt, wobei die Operationen `scatter` und `allgather` aus Algorithmus 2 als `comm_func` integriert werden. Um Kommunikationsprobleme zu vermeiden, wird der Tensor zunächst durch `x.reshape(3,0,1,2)` logisch neu geordnet und anschließend mit `x.contiguous()` im GPU-Speicher zusammenhängend abgelegt. Diese Schritte sind notwendig, da torch.distributed keine fehlerfreie Kommunikation über nicht zusammenhängende Dimensionen ermöglicht. Zudem verbessert dies die Effizienz der Datenübertragung. Nach der Anpassung wird der Tensor entlang der Hidden-Dimension aufgeteilt. Die resultierenden Teil-Tensoren werden in einer Liste gespeichert, die anschließend mithilfe der `scatter`-Funktion an die anderen GPUs innerhalb der MP-Gruppe verteilt wird. Auf den Ziel-GPUs erfolgt eine Rückumwandlung des Tensors in die ursprüngliche Form sowie eine Neuanordnung im Speicher, um Kontiguität sicherzustellen. Beim Verlassen der Blocksequenz wird die benutzerdefinierte `Torch.autograd`-Funktion `GatherScatter` verwendet, die im Forward-Pass eine Allgather- und im Backward-Pass eine Scatter-Operation ausführt. Nach der `allgather`-Operation wird eine Liste von Teil-Tensoren generiert, die mit `torch.cat(list)` entlang der verteilten Dimension zusammengeführt werden. Um die Funktionsfähigkeit des Ansatzes zu gewährleisten, muss die Anzahl der Neuronen im MLP entsprechend angepasst werden.

Die mittlere Abbildung aus Abbildung 3.3 zeigt die Implementierung des lokal parallelisierten Modells entlang der Hidden-Dimension. Dieses Verfahren folgt demselben Prinzip wie die globale Parallelisierung, wird jedoch blockweise durchgeführt, insbesondere bei Eintritt in und nach dem AFNO-Layer. Im Gegensatz zur globalen Parallelisierung bleibt die Hidden-Dimension-Größe des MLP unverändert.

Rechts ist die Parallelisierung entlang der Diagonalblöcke dargestellt. Die Scatter-Operation wird nach der Transformation in den Fourierraum ausgeführt, während die Allgather-Operation direkt vor der Rücktransformation erfolgt.

Algorithm 1 Hilfsfunktion zur Kommunikation von Tensoren durch Speichermanipulation

tensorCommunication(x,$comm_func$):
  Neuanordnung des Tensors x derart, dass die zu verteilende Dimension == 0
  Kontiguität durch `x.contiguous()` sicherstellen
  Kommunikation durch übergebene Funktion $x = comm_func(x)$
  Neuanordnung des erhaltenen Tensors, sodass die verteilte Dimension == -1
  Kontiguität durch `x.contiguous()` sicherstellen
  Return x

Algorithm 2 Kommunikationsfunktionen und deren Datenverarbeitung

scatter(x, src):
Aufteilen des Tensors entlang der zu verteilenden Dimension
in mp_size Chunks, Rückgabe als Liste $list$
Verteilen der Liste vom lokalen Root-Prozess an alle Prozesse der
Kommunikationsgruppe: $x = scatter(list, src = root)$

allgather(x):
Sammlung von Teil-Tensoren in einer Liste auf allen Rängen
der Kommunikationsgruppe: $list = allgather(x)$
Zusammenführung der Liste entlang der verteilten Dimension

allreduce(x, op):
Sammlung und Verarbeitung der Tensoren mittels der vorgegebenen
Operation op (z. B. `AVG`, `MIN`, `MAX`, `SUM`) und Übermittlung des verarbeiteten
Tensors an alle Ränge der Kommunikationsgruppe: $x = allreduce(x, \texttt{AVG})$

5. Evaluation

Das Evaluationskapitel beginnt mit einer Analyse der Baseline-Tests, die durchgeführt wurden, um den Einfluss der Anzahl verwendeter GPUs auf die Modellleistung und die Effizienz der Trainingszeit zu untersuchen. Anschließend werden die implementierten Modellarchitekturen und ihre Leistung in Bezug auf Skalierbarkeit, Overhead durch Kommunikationszeit und Speichermanipulation sowie Auswirkungen auf den Loss verglichen.

5.1. Baseline

Diese Sektion bezieht sich auf Tabelle 5.1. Die Baselinetests sind notwendig für die Skalierungsuntersuchung.

Hier stellt sich heraus, dass das Training auf 64 GPU mit etwa 4,5h schnell, jedoch ineffizient im Bezug zur GPU-Nutzung ist. Weiterhin hat das auf 8 GPU trainierte Modell einen 18,5% reduzierten Loss gegenüber dem Ursprungsmodell. Dies kann sich auf Large Mini-Batch Effekte zurückführen lassen.[23] Die Batch-Size aller Modelle ist 1, dennoch unterscheiden sich die globalen Batch-Sizes durch das Training unterschiedlich vieler parallelen Modelle, wodurch sich eine globale Batch-Size von 8, 16, 32 oder 64 ergibt obwohl der mini-batch weiterhin bei einem Sample liegt.

Außerdem ist der Durchsatz der Modelle unterschiedlich, dieser ist aus räumlichen Limitationen des Dokuments nicht in der Tabelle gelistet. Wie in Abschnitt 3.4 beschrieben besteht der Trainingsdatensatz aus 54020 Samplen die auf die Datenparallelen Modelle aufgeteilt werden. Hieraus entstehen folgende Splits:

- **8 GPU**: 6753 bzw. 6752 Sample
- **16 GPU**: 3377 bzw. 3376 Sample
- **32 GPU**: 1689 bzw. 1688 Sample
- **64 GPU**: 845 bzw. 844 Sample

diese sind ungleich verteilt da die Samples nicht ganz durch die GPU geteilt werden können, damit erhalten in jeder Aufteilung je 4 GPU einen Batch mehr.

#GPU	Valid Loss	ΔLoss	Laufzeit	ΔLaufzeit	in Prozent	GPU Zeit Gesamt
64	0,092134	-	04:23:04	-	-	11-16:36:16
32	0,082644	-10,30%	07:34:03	+ 03:10:59	+72,60%	10-02:09:36
16	0,078743	-14,53%	12:55:55	+ 08:32:51	+194,95%	08-14:54:40
8	0,075090	-18,50%	23:41:20	+ 19:18:16	+440,30%	07-21:30:40

Tabelle 5.1.: Vergleich der Baseline Experimente mit dem Ursprungsmodell. Verglichen wird über die Anzahl an verwendeten GPUs anhand des Validierungsfehlers, der Laufzeit sowie der Gesamtnutzung von GPU Ressourcen.

Daraus ergibt sich mit Batch-Size 1 ein Durchsatz der Modelle von

- **8 GPU**: $\frac{6752 Batches/Epoch \times 50 Epoch}{85278 s} = 3,958 Sample/s$
- **16 GPU**: $\frac{3376 Batches/Epoch \times 50 Epoch}{46553 s} = 3,626 Sample/s$
- **32 GPU**: $\frac{1688 Batches/Epoch \times 50 Epoch}{27241 s} = 3,098 Sample/s$
- **64 GPU**: $\frac{844 Batches/Epoch \times 50 Epoch}{15782 s} = 2,674 Sample/s$

Da es sich beim AFNONet um das Backbone des FourCastNet handelt und man dieses Backbone durch den Neural Operator lediglich einmal trainieren muss würde es sich lohnen mehr Realtime zu verwenden um zum einen weniger GPU Gesamtnutzung und zum anderen einen besseren Loss zu erreichen. Hierbei trainiert dieses zwar etwa 19,5 Stunden länger, spart jedoch 3 Tage und 5h GPU nutzung, hat einen 48% höheren Throughput und 18,5% reduzierten Loss als das mit 64 GPU trainierte Modell.

5.2. MP-Modelle

Die Evaluation der modellparallelen Ansätze konzentriert sich auf die Forschungsfrage zur Skalierbarkeit sowie den Erhalt der Genauigkeit des Ursprungsmodells. Neben der starken und schwachen Skalierbarkeit, der Effizienz sowie dem Durchsatz des Modells wird der Overhead (Kommunikationsaufwand mit Datenvorverarbeitung) bewertet.

Aufgrund der großen Anzahl potenzieller Konfigurationen (3 Architekturen * 4 GPU-Anzahlen * 3 MP-Sizes * 2 Skalierungsstrategien * 3 Untersuchungsdimensionen) werden im Folgenden nur repräsentative Ergebnisse dargestellt.

So wird der Vergleich der Architekturen, mit wenigen Ausnahmen, hauptsächlich auf die Ergebnisse bei Nutzung von 64 GPUs beschränkt, da dies die Anzahl im Originalpaper ist.

Durch die zeitlichen Limitationen und in Abschnitt 5.3 beschriebenen Hürden, konnten die lokalen Architekturen (wegen fehlenden Bugfixes) nur bedingt getestet werden. Daher werden Auswertungen auf den globalen Parallelisierungsansatz bezogen oder aus vorliegenden Daten der lokalen Ansätze Approximiert. Bei letzteren traten Gradientendesynchronisationen auf, welche ebenfalls im Abschnitt 5.3 aufgegriffen werden.

5.2.1. Skalierbarkeit

in Abbildung 5.1 sind die Laufzeiten und Durchsätze der verschiedenen Modelle aufgeführt. Hierbei werden die Modelle zum einen nach Architektur und zum anderen nach Skalierungsuntersuchung mit MP-Size unterteilt. Weiterhin sind die zu verarbeitenden Sample einer Epoche je nach Einstellung aufgeführt. Für jedes Modell wird die durchschnittliche Zeit pro Epoche sowie der daraus errechnete Durchsatz, einer einzelnen GPU im Modell, dargestellt. Die Werte `OOM` und `NaN` bedeuten, dass bei dem Modell ein Out of Memory Error aufgetreten ist und Modell somit keine Ergebnisse erzielen konnte.

Aus den Tabellen ist zu entnehmen, dass die lokalen Ansätze in etwa die doppelte Laufzeit des globalen Ansatzes aufweisen, was sich auf die Kommunikationskomplexität zurückführen lassen kann. Weiterhin ist zu erkennen, dass sich die MP-Size von 8 durchweg negativ auf die Laufzeit der Modelle auswirkt, begründet durch die hierbei erforderliche Internode-Kommunikation. Zuletzt haben die Modelle der Durchläufe in denen auf starke Skalierung getestet wurde einen höheren Durchsatz als bei Tests auf schwache Skalierbarkeit. Dies kann unter anderem auf kleinere Tensoren sowie erhöhter Rechenzeit gegenüber der Kommunikationszeit zurückzuführen sein.

In Abbildung 5.2 werden die Werte aus Abbildung 5.1 nochmals in den Bezug der modellinternen Vergleiche gestellt. Weiterhin werden die Werte der Baseline mit aufgeführt um Vergleiche ziehen und verdeutlichen zu können.

In Abbildung 5.2a lässt sich erkennen, dass der globale Ansatz mit einer MP-Size von 2 und einer lokalen Batch-Size von 2, unter außnahme der verwendung von 16 GPUs, schneller als das Baseline Modell ist. Hierbei zeigt das Modell einen vergleichbaren Durchsatz zur Baseline, mit der Besonderheit, dass es bei Nutzung von 8 GPUs 10% mehr Durchsatz aufweist. Dies ist die einzige Konfiguration, die schneller als die Baseline ist. Zudem ist hervorzuheben, dass der globale Parallelisierungsansatz, im Durchlauf zur Untersuchung der starken Skalierbarkeit unter MP-Sizes 4 und 8, mehr als den doppelten Durchsatz gegenüber der Baseline aufweist.

Wie bereits zu Abbildung 5.1 erwähnt, sind die lokalen Ansätze, zusehen in Abbildung 5.2c und Abbildung 5.2b, wie zu erwarten langsamer, begründet durch die erhöhten Kommunikationen. Hierbei ist interessant, dass das Scattern im Fourrier-Raum zwischen 0,7 und 16,8% inperformanter ist, jedoch bei der Nutzung von 32 GPUs und einer lokalen Batch-Size von 1, 3,6% performanter ist. Diese reduzierte Performanz könnte durch die erhöhte Fragmentierung des Tensors gegeben der zusätzlichen Dimension im Fourier-Raum erklärt werden.

Abbildung 5.3 zeigt auf, dass selbst beim Baseline Modell ansich lediglich bis zu einem Einsatz von 16 GPU von starker Skalierbarkeit gesprochen werden kann. Der globale Ansatz weist lediglich bei einer MP-Size von 2 und 8 GPUs einen Speedup von 0,9151 auf, jedoch liegt die Skalierbarkeit dieser Konfiguration bereits ab 16 GPUs mit einem Speedup von 1,5158 bei unter 75%. Die höchste Skalierbarkeit den der Ansatz bei MP-Size 4 und 8 aufweisen kann liegt bei 54,89 bzw. 29,42%.

Tabelle zum Vergleich der Laufzeit und des Durchsatzes der Architekturen (64 GPU)								
64 GPU		Modell:	Global		Lokal (Hidden)		Lokal (Diagonal)	
Skalierung	MP	Samples	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz
Stark	2	1688	362,70	4,654	592,69	2,848	609,53	2,769
Stark	4	3376	608,44	5,549	1088,02	3,103	1130,73	2,986
Stark	8	6752	1036,73	6,513	2675,90	2,523	2758,89	2,447
Schwach	2	844	311,36	2,711	460,91	1,831	478,40	1,764
Schwach	4	844	464,21	1,818	616,84	1,368	682,67	1,236
Schwach	8	844	657,00	1,285	1718,94	0,491	OOM	NAN

(a) Vergleich der Laufzeit und des Durchsatzes der Architekturen (64 GPU)

Tabelle zum Vergleich der Laufzeit und des Durchsatzes der Architekturen (32 GPU)								
32 GPU		Modell:	Global		Lokal (Hidden)		Lokal (Diagonal)	
Skalierung	MP	Samples	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz
Stark	2	3376	633,71	5,327	1044,14	3,233	1023,94	3,297
Stark	4	6752	1101,14	6,132	2159,44	3,127	2082,27	3,243
Stark	8	13505	2037,28	6,629	5078,65	2,659	5178,90	2,608
Schwach	2	1688	537,59	3,140	823,53	2,050	823,64	2,049
Schwach	4	1688	633,71	2,664	1093,42	1,544	1237,24	1,364
Schwach	8	1688	1564,21	1,079	3144,02	0,537	OOM	NAN

(b) Vergleich der Laufzeit und des Durchsatzes der Architekturen (32 GPU)

Tabelle zum Vergleich der Laufzeit und des Durchsatzes der Architekturen (16 GPU)								
16 GPU		Modell:	Global		Lokal (Hidden)		Lokal (Diagonal)	
Skalierung	MP	Samples	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz
Stark	2	6752	1125,15	6,001	1781,31	3,790	1823,98	3,702
Stark	4	13505	1855,76	7,277	3338,34	4,045	3563,72	3,790
Stark	8	27010	3428,84	7,877	8035,97	3,361	8654,58	3,121
Schwach	2	3376	978,99	3,448	1411,07	2,393	1512,06	2,233
Schwach	4	3376	1472,86	2,292	1944,24	1,736	2270,39	1,487
Schwach	8	3376	2682,27	1,259	5938,84	0,568	OOM	NAN

(c) Tabelle zum Vergleich der Laufzeit und des Durchsatzes der Architekturen (16 GPU)

Tabelle zum Vergleich der Laufzeit und des Durchsatzes der Architekturen (8 GPU)								
8 GPU		Modell:	Global		Lokal (Hidden)		Lokal (Diagonal)	
Skalierung	MP	Samples	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz	Zeit/Ep. (s)	Durchsatz
Stark	2	13505	1863,83	7,246	2844,11	4,748	3030,16	4,457
Stark	4	27010	3107,19	8,693	5443,61	4,962	5891,95	4,584
Stark	8	54020	5796,85	9,319	14058,96	3,842	15590,50	3,465
Schwach	2	6752	1532,20	4,407	2480,30	2,722	2583,75	2,613
Schwach	4	6752	2549,37	2,648	3560,88	1,896	4056,11	1,665
Schwach	8	6752	5154,22	1,310	11353,14	0,595	OOM	NAN

(d) Tabelle zum Vergleich der Laufzeit und des Durchsatzes der Architekturen (8 GPU)

Abb. 5.1.: Aufführung der durchschnittlichen Zeit in einer Epoche sowie dem Sampleduchsatz pro GPU, für 8, 16, 32 und 64 GPU, der verschiedenen Architekturen

Global		Durchschnittliche Rechenzeit pro Epoche (s)				Durchsatz pro GPU (Sample/s)			
Skalierung	MP	8 GPU	16 GPU	32 GPU	64 GPU	8 GPU	16 GPU	32 GPU	64 GPU
Stark	2	1863,83	1125,15	633,71	362,70	7,246	6,001	5,327	4,654
Stark	4	3107,19	1855,76	1101,14	608,44	8,693	7,277	6,132	5,549
Stark	8	5796,85	3428,84	2037,28	1036,73	9,319	7,877	6,629	6,513
Schwach	2	1532,20	978,99	537,59	311,36	4,407	3,448	3,140	2,711
Schwach	4	2549,37	1472,86	633,71	464,21	2,648	2,292	2,664	1,818
Schwach	8	5154,22	2682,27	1564,21	657,00	1,310	1,259	1,079	1,285
Baseline		1705,56	931,06	544,82	315,65	3,959	3,626	3,098	2,674

(a) Durchschnittliche Rechenzeit und Durchsatz aller Experiment Konfigurationen der global parallelisierten Architektur.

Lokal (Hidden)		Durchschnittliche Rechenzeit pro Epoche (s)				Durchsatz pro GPU (Sample/s)			
Skalierung	MP	8 GPU	16 GPU	32 GPU	64 GPU	8 GPU	16 GPU	32 GPU	64 GPU
Stark	2	2844,11	1781,31	1044,14	592,69	4,748	3,790	3,233	2,848
Stark	4	5443,61	3338,34	2159,44	1088,02	4,962	4,045	3,127	3,103
Stark	8	14058,96	8035,97	5078,65	2675,90	3,842	3,361	2,659	2,523
Schwach	2	2480,30	1411,07	823,53	460,91	2,722	2,393	2,050	1,831
Schwach	4	3560,88	1944,24	1093,42	616,84	1,896	1,736	1,544	1,368
Schwach	8	11353,14	5938,84	3144,02	1718,94	0,595	0,568	0,537	0,491
Baseline		1705,56	931,06	544,82	315,65	3,959	3,626	3,098	2,674

(b) Durchschnittliche Rechenzeit und Durchsatz aller Experiment Konfigurationen der, über die Hidden-Dimension, lokal parallelisierten Architektur.

Lokal (Diag)		Durchschnittliche Rechenzeit pro Epoche (s)				Durchsatz pro GPU (Sample/s)			
Skalierung	MP	8 GPU	16 GPU	32 GPU	64 GPU	8 GPU	16 GPU	32 GPU	64 GPU
Stark	2	3030,16	1823,98	1023,94	609,53	4,457	3,702	3,297	2,769
Stark	4	5891,95	3563,72	2082,27	1130,73	4,584	3,790	3,243	2,986
Stark	8	15590,50	8654,58	5178,90	2758,89	3,465	3,121	2,608	2,447
Schwach	2	2583,75	1512,06	823,64	478,40	2,613	2,233	2,049	1,764
Schwach	4	4056,11	2270,39	1237,24	682,67	1,665	1,487	1,364	1,236
Schwach	8	NaN	NaN	NaN	NaN	OOM	OOM	OOM	OOM
Baseline		1705,56	931,06	544,82	315,65	3,959	3,626	3,098	2,674

(c) Durchschnittliche Rechenzeit und Durchsatz aller Experiment Konfigurationen der, über die Diagonalblöcke, lokal parallelisierten Architektur.

Abb. 5.2.: Tabellen zur Darstellung der durchschnittlich benötigten Rechenzeit pro Epoche sowie dem Sampleduch- satz pro GPU, für 8, 16, 32 und 64 GPU, der verschiedenen Architekturen inklusive Baseline und Experimentkonfigurationen.

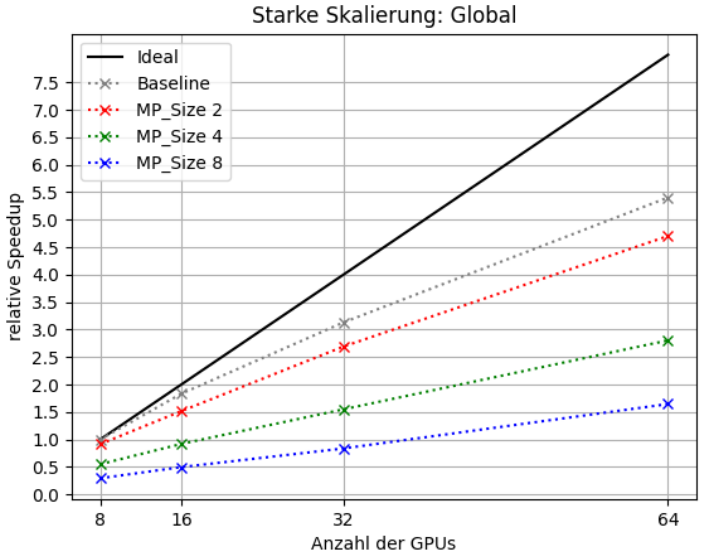


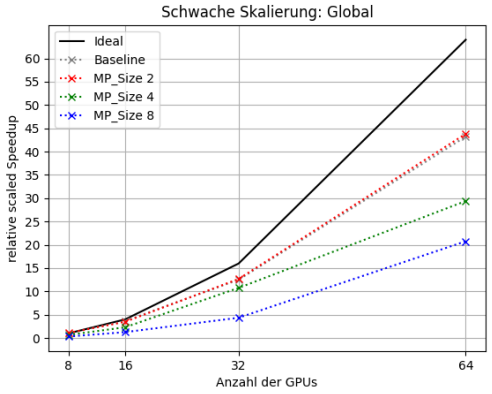
Abb. 5.3.: Diagramm zur Darstellung der relativen starken Skalierbarkeit des globalen Ansatzes. Neben der Ideallinie (100% Skalierbarkeit) werden die verschiedenen MP-Sizes in rot, grün und blau aufgeführt. Messwerte sind hier mit einem Kreuz gekennzeichnet, wobei die gepunkteten Linien lediglich der Visualisierung und Projektion dienen.

Im Gegensatz zeigt der globale Ansatz bei MP-Size von 2 bei der Nutzung von 8 GPU einen relativen skalierten Speedup von 1,1131 auf, siehe Abbildung 5.4a. Dieser ist nicht auf Effekte zurückzuführen sondern durch die Approximation von T_1 ($T_1 \approx T_{P_{min}}$) zu erklären und ist ebenfalls in Abbildung 5.2a zu erkennen. Hierbei muss angenommen werden, dass $T_{P_{min}} > P_{min} \cdot T_1$ und somit keine Hyperskalierbarkeit vorliegt.

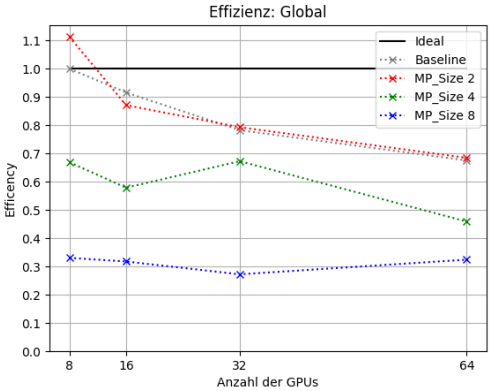
Auch in dieser Abbildung ist zu erkennen, dass die Kommunikation über Knotengrenzen hinweg die Skalierbarkeit maßgeblich einschränkt. Dies wird besonders deutlich wenn die Speedups der MP-Sizes 2 und 4 im Vergleich mit dem Speedup der Architektur mit MP-Size 8 betrachtet werden.

Die schwache Skalierbarkeit ist durch die Effizienz beschrieben und kann für die Architekturen der globalen Parallelisierung aus Abbildung 5.4b entnommen werden. Hier bildet sich, wie bereits erwähnt, ab, dass lediglich die Architektur mit MP-Size 2 effizienter als das Ursprungsmodell ist und, außer eben dieser Ausnahme, alle auf der globalen Parallelisierung basierenden Modelle bereits ab 8 GPU ineffizient sind. Bemerkenswert ist jedoch dass die Effizienz des Modells der MP-Size 4 nicht stetig sinkt und bei 32 GPU mit 67,28% sogar geringfügig höher liegt als bei 8 GPU mit 66,90%

Wie in Abbildung 5.5 zu erkennen liegt bei den lokal parallelisierten Ansätzen keine Skalierbarkeit vor. Der Speedup der Ansätze liegt in allen Konfigurationen unter 20-60% des Idealwerts. Dies kann jedoch auch auf die bereits erwähnte unausgereifte Implementierung der Ansätze zurückzuführen sein. Ähnlich bilden sich auch die Metriken zur schwachen Skalierung in Abbildung 5.6 und Abbildung 5.7 ab.

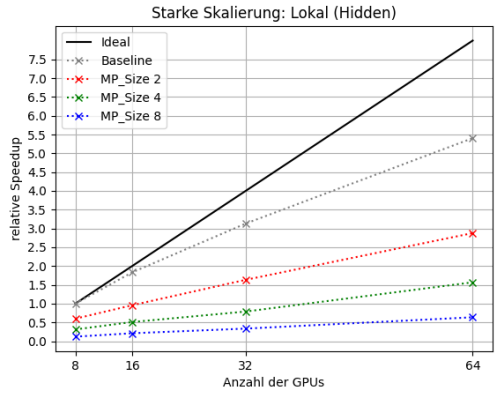


(a) Vergleich der relativen schwachen Skalierbarkeit.

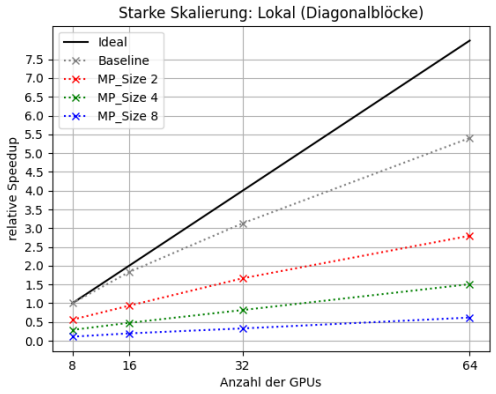


(b) Vergleich der relativen Effizienz.

Abb. 5.4.: Diagramme zur Veranschaulichung der Metriken der schwachen Skalierung aus den Experimenten auf der globalen Parallelisierungsstrategie. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3.

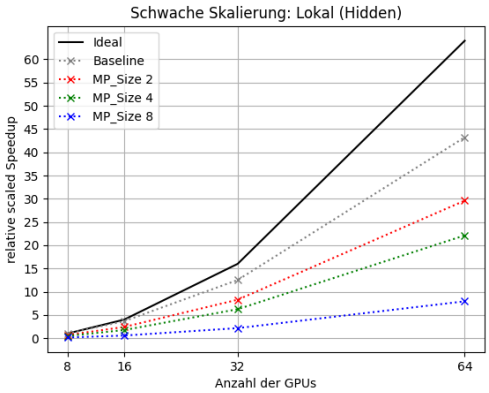


(a) Relative starke Skalierbarkeit bei Verteilung der Hidden-Dimension.

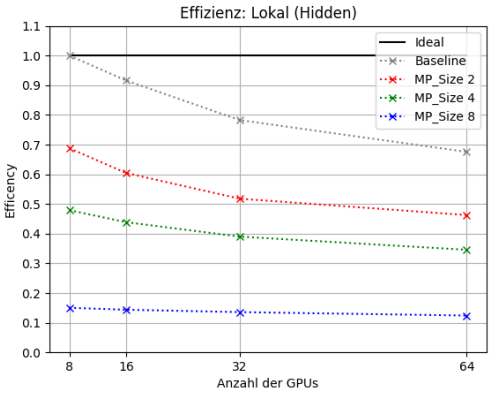


(b) Relative starke Skalierbarkeit bei Verteilung der Diagonalblöcke

Abb. 5.5.: Diagramm zur Darstellung der relativen starken Skalierbarkeit der lokalen Ansätze. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3.

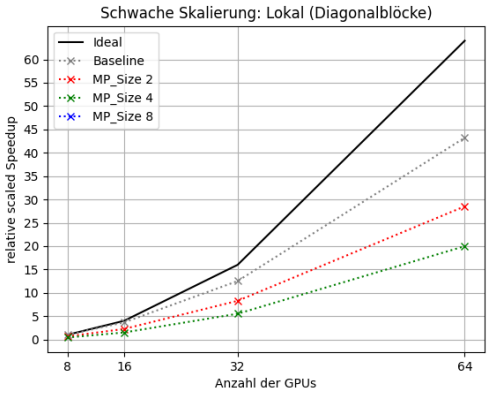


(a) Vergleich der relativen schwachen Skalierbarkeit.

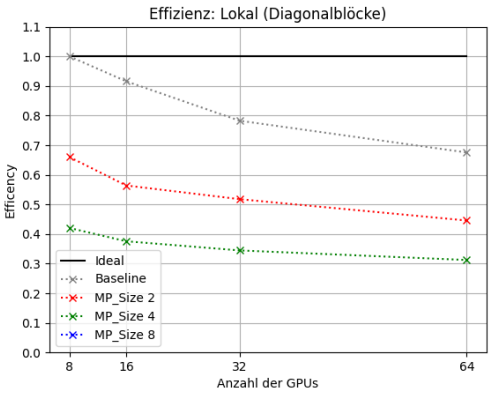


(b) Vergleich der relativen Effizienz.

Abb. 5.6.: Diagramme zur Veranschaulichung der Metriken der schwachen Skalierung aus den Experimenten der lokalen Parallelisierungsstrategie der Verteilung über die Hidden-Dimension. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3.



(a) Vergleich der relativen schwachen Skalierbarkeit.



(b) Vergleich der relativen Effizienz.

Abb. 5.7.: Diagramme zur Veranschaulichung der Metriken der schwachen Skalierung aus den Experimenten der lokalen Parallelisierungsstrategie der Verteilung über die Diagonalblöcke. Hierbei werden die Architekturen unterschiedlicher MP-Sizes mit der Baseline verglichen. Das Layout und Design ist analog zu Abbildung 5.3.

5.2.2. Overhead

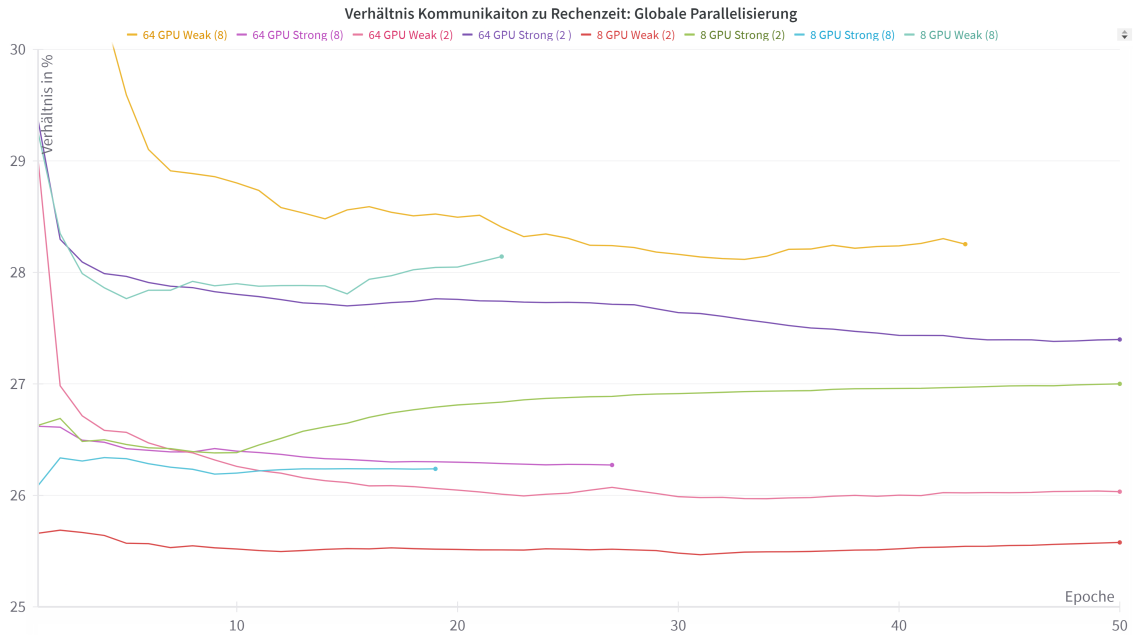
Für den Overhead wird der Kommunikationsaufwand sowie die Zeit zur Speicherumsortierung gegenüber dem Rechenaufwand dargestellt. Hierbei wird jedoch nicht Kommunikaiton gegenüber der Gesamtzeit einer Epoche sondern nur gegenüber der Zeit die das Modell für Berechnungen benötigt gestellt. Abbildung 5.8a und Abbildung 5.8b zeigen auf, dass das Training der global parallelisierten Modelle mit Batch-Size von 1, wie zu erwarten, weniger Overhead aufweisen als bei einem Training mit einer Batch-Size von MP-Size. Die Ausnahme stellt erneut Modell global_8GPU_weak_mp2, also das global Parallelisierte Model mit dem Prallelisierungsgrad von 2 mit 8 genutzen GPUs und lokaler Batch-Size von 2. Dieses weist mit 25,5% den geringsten Overhead in der Kommunikation auf. Weiterhin wird ersichtlich, dass sich der Overhead für diesen Ansatz zwischen 26 und 27% bei lokaler Batch-Size von 1 und 27,3 bis 29% bei Batch-Size von MP-Size bewegt. Im Gegensatz fällt die Datenvorverarbeitung mit 0,425 bis 0,44% bzw 0,49 bis 0,53% wenig ins Gewicht. Modelle die vor Epoche 50 stoppen haben die gesetzte Zeitgrenze der Experimente erreicht.

5.2.3. Loss

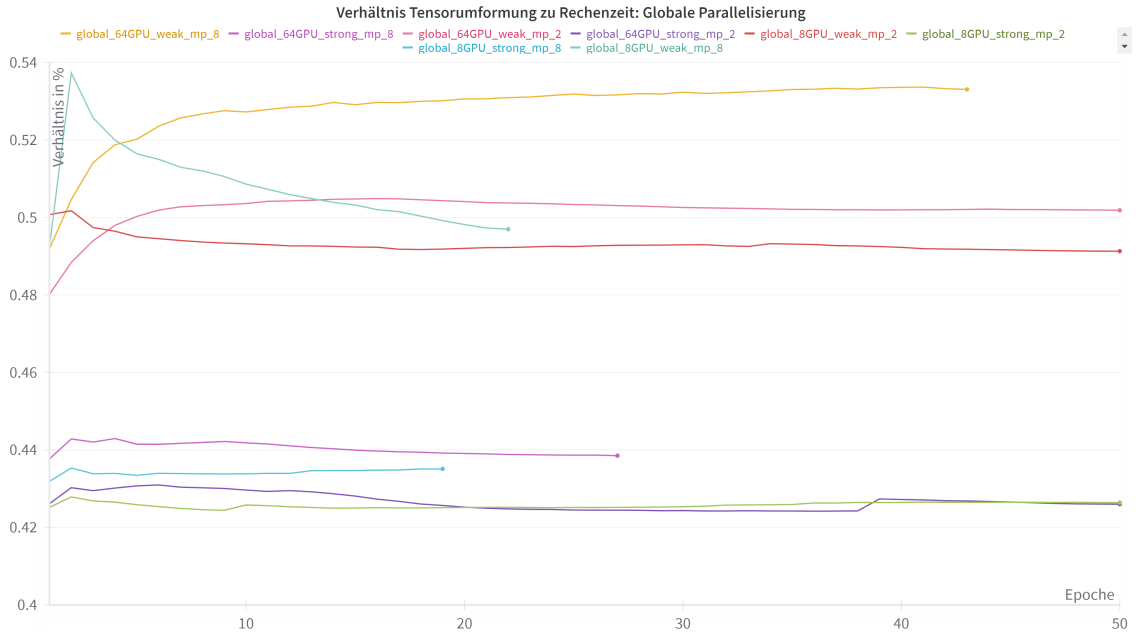
Das Forschungsziel des Erhalts der Genauigkeit, oder deren Verbesserung gegenüber dem Ursprungsmodell, ist in Abbildung 5.9a und Abbildung 5.9b evaluiert. Abbildung 5.9a ist nach durchschnittlicher Laufzeit pro Epoche sortiert und soll eventuellen Abwägungen zum Kompromiss zwischen Genauigkeit und Geschwindigkeit dienen Die Epochen dieser Tabellen sind nicht arbiträr gewählt, sondern um Werte bestimmter Ansätze und deren Potenzial aufzeigen zu können vor der Gradientendesynchronisation durch unausgereifte Implementation. Hierbei verschlechtert sich der Loss und bleibt auf einem Plateau. Diese Runs werden mit `Desync` gekennzeichnet. Weiterhin bedeutet `DNF`, dass as Experiment entweder durch erreichen des Zeitlimits oder aber duch einen Fehler anderer Art abgebrochen wurde, vergleiche Abbildung 5.10 grau und dunkel lila als `DNF` und Orange als `Desync`. Weiterhin ist die Validierung der lokal Parallelisierten Architekturen Problembehaftet wodurch hier lediglich der Trainingsloss verglichen werden kann.

Den geringsten Loss weisen die lokal parallelisierten Ansätze, mit MP-Size von 2 und 4, sowie der globale Ansatz, mit MP-Size 4 und 8, bei Batch-Size 1 auf. Diese sind jedoch etwa 2-mal bzw. etwa 3,5-mal langsamer. Die globale Parallelisierung mit Batch-Size 1 und MP-Size 2 ist knapp 20% langsamer als die Baseline mit vergleichbarem Loss. Im Gegensatz zu den anderen Metriken ist das global parallelisierte Modell mit MP-Size 2 und lokaler Batch-Size 2 hier nicht besser als das Baseline Modell. Mit 9% schlechterem Loss nach Epoche 3 und 17% bei Abschluss des Trainings in Epoche 50 ist der Vorteil des Modells der 2,4% geringeren Laufzeit zu vernachlässigen.

In Abbildung 5.9b sind die Ergebnisse aus Abbildung 5.9a erneut aufgeführt, jedoch sortiert nach Skalierungsansatz, Architektur und MP-Size. Hierdurch ist für Durchläufe mit Batch-Size von 1 eine positive Korrelation von Loss und MP-Size und eine negative Korellation für Durchläufe mit Batch-Size von MP-Size erkennbar. Die Ausnahme stellen die lokalen Ansätze mit MP-Size 8 und Batch-Size 1 dar. Außerdem wird die Instabilität der Lokalen Ansätze durch die zeitlich eingeschränkte Implementation nochmals deutlich.



(a) Verhältnis von Zeit in Kommunikationsoperationen zu Rechenzeit



(b) Verhältnis von Zeit zur Datenvorverarbeitung zur Rechenzeit

Abb. 5.8.: Diagramme zur Veranschaulichung des Overheads der Modelle der verschiedenen Experimente, erstellt in W&B. Hierbei werden diese in Kommunikations und Datenvorverarbeitungs-Overhead aufgeteilt und ins Verhältnis gegenüber der Rechenzeit gestellt. Dies geschieht für die Verwendung von 8 bzw. 64 GPUs auf dem globalen Ansatz mit MP-Sizes 2 und 8.

Experimenteinstellung			*Experimente nach Laufzeit pro Epoche (64 GPU)*					
Granularität	Skalierung	MP	Laufzeit pro Ep.	Loss Ep. 3	Loss Ep. 13	Loss Ep. 26	Loss Ep. 39	Loss Ep. 50
			(in s)					
Lokal (Diag)	Stark	8	2758,89	0,16281082	0,16098450	DNF	DNF	DNF
Lokal (Hidden)	Stark	8	2675,90	0,18618593	0,18492991	DNF	DNF	DNF
Lokal (Hidden)	Schwach	8	1718,94	0,20234755	0,19594905	0,19607803	DNF	DNF
Lokal (Diag)	Stark	4	1130,73	0,13330513	Desync	Desync	Desync	DNF
Lokal (Hidden)	Stark	4	1088,02	0,13018312	0,09053735	0,081385702	0,080248967	DNF
Global	Stark	8	1036,73	0,14418407	0,09881841	0,08866691	DNF	DNF
Lokal (Diag)	Schwach	4	682,67	0,20170715	0,13444850	Desync	Desync	Desync
Global	Schwach	8	657,00	0,22789511	0,17210448	0,14060083	0,13247691	DNF
Lokal (Hidden)	Schwach	4	616,84	0,20147683	Desync	Desync	Desync	Desync
Lokal (Diag)	Stark	2	609,53	0,17109285	Desync	Desync	Desync	Desync
Global	Stark	4	608,44	0,16032241	0,09664498	0,08717390	0,08587676	DNF
Lokal (Hidden)	Stark	2	592,69	0,16689123	0,10891010	0,09237646	0,08815311	0,086738698
Lokal (Diag)	Schwach	2	478,40	0,20268510	0,12616551	0,10473438	0,09960387	0,09760127
Global	Schwach	4	464,21	0,21714473	0,15904385	0,12305865	0,11361651	0,11117823
Lokal (Hidden)	Schwach	2	460,91	0,19786498	0,13451301	0,10978340	0,10290828	0,10087367
Global	Stark	2	362,70	0,18122715	0,11197904	0,09455625	0,08986796	0,08850516
Baseline	Keine	1	315,65	0,19212738	0,12451756	0,09624255	0,08882159	0,08729839
Global	Schwach	2	311,36	0,20966856	0,14138380	0,11230031	0,10455101	0,10243785

(a) Loss der Durchläufe sortiert nach Laufzeit

Experimenteinstellung			*Experimente nach Skalierung, (64 GPU)*					
Granularität	Skalierung	MP	Laufzeit pro Ep. i	Loss Ep 3	Loss Ep 13	Loss Ep 26	Loss Ep 39	Loss Ep 50
			(in s)					
Baseline	Keine	1	315,65	0,19212738	0,12451756	0,09624255	0,08882159	0,08729839
Global	Schwach	2	311,36	0,20966856	0,14138380	0,11230031	0,10455101	0,10243785
Global	Schwach	4	464,21	0,21714473	0,15904385	0,12305865	0,11361651	0,11117823
Global	Schwach	8	657,00	0,22789511	0,17210448	0,14060083	0,13247691	DNF
Lokal (Diag)	Schwach	2	478,40	0,20268510	0,12616551	0,104734376	0,099603869	0,097601272
Lokal (Diag)	Schwach	4	682,67	0,20170715	0,13444850	Desync	Desync	Desync
Lokal (Hidden)	Schwach	2	460,91	0,19786498	0,13451301	0,10978340	0,10290828	0,100873671
Lokal (Hidden)	Schwach	4	616,84	0,20147683	Desync	Desync	Desync	Desync
Lokal (Hidden)	Schwach	8	1718,94	0,20234755	0,19594905	0,19607803	DNF	DNF
Global	Stark	2	362,70	0,18122715	0,11197904	0,09455625	0,08986796	0,08850516
Global	Stark	4	608,44	0,16032241	0,09664498	0,08717390	0,08587676	DNF
Global	Stark	8	1036,73	0,14418407	0,09881841	0,08866691	DNF	DNF
Lokal (Diag)	Stark	2	609,53	0,17109285	Desync	Desync	Desync	Desync
Lokal (Diag)	Stark	4	1130,73	0,13330513	Desync	Desync	Desync	DNF
Lokal (Diag)	Stark	8	2758,89	0,16281082	0,16098450	DNF	DNF	DNF
Lokal (Hidden)	Stark	2	592,69	0,16689123	0,10891010	0,09237646	0,08815311	0,08673870
Lokal (Hidden)	Stark	4	1088,02	0,13018312	0,09053735	0,08138570	0,08024897	DNF
Lokal (Hidden)	Stark	8	2675,90	0,18618593	0,18492991	DNF	DNF	DNF

(b) Loss der Durchläufe hirarchisch sortiert nach Skalierungsanforderung, Parallelisierungsgranularität und MP-Size.

Abb. 5.9.: Tabellarische Abbildungen der Ergebnisse aller Experimente auf 64 GPUS.

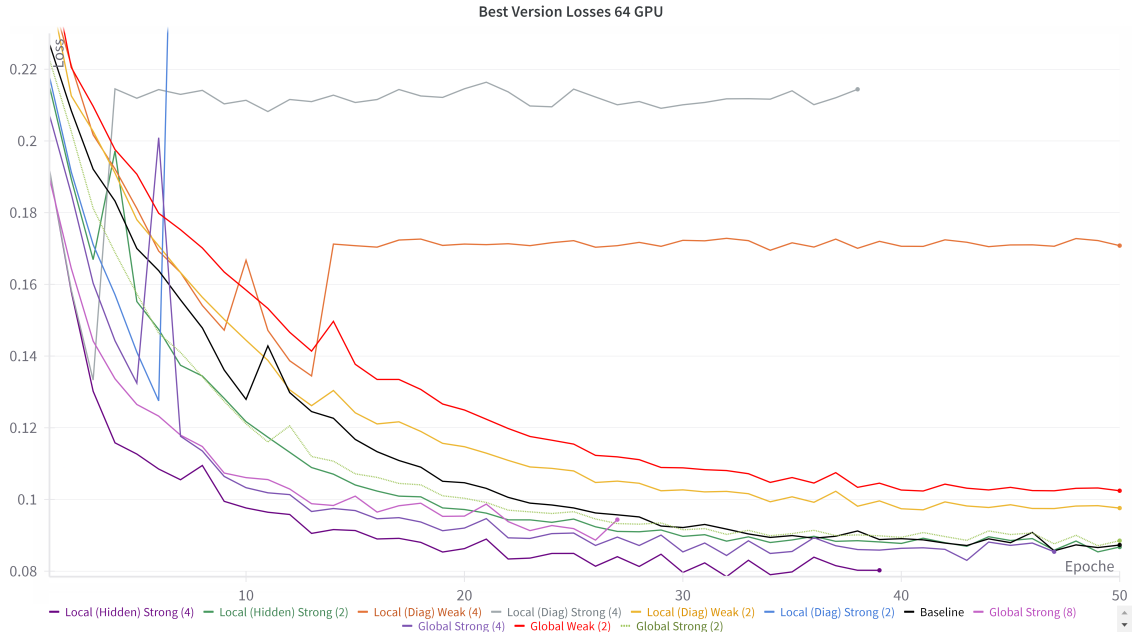


Abb. 5.10.: Verlauf des Loss entlang der Epochen des Trainings. In dieser Abbildung wurden aus allen Experimenten mit 64 GPUs, zehn Modelle mit den besten Losswerten bis Epoche 5 ausgewählt und den Trainingsverlauf in diesem Diagram dargestellt.

5.3. Herausforderungen und Verworfene Ansätze

In diesem Abschnitt werden Herausforderungen, die während der Entwicklung auftraten und gefundene Lösungen, sofern vorhanden, erläutert, sowie verworfene Ansätze und deren Gründe dargelegt.

5.3.1. Verworfene Ansätze

Erhöhen der Modellkapazität durch Vergrößerung der Hidden-Dimension. Ein Ansatz zur Erweiterung des Modells bestand darin, die Hidden-Dimension auf das Vielfache von `MP-Size` zu vergrößern (siehe Abbildung A.12). Die Idee war ein Linear-Layer als Blow-Up-Layer einzuführen, wie in Unterabschnitt 3.5.1 beschrieben. Dieses hätte die Hidden-Dimesion so erhöht, dass jeder Teil des verteilten Modells nach der Verteilung des Tensors die selbe Größe wie das ursprüngliche Modell besitzt, jedoch auf unterschiedlichen Output-Neuronen des eingefügten Layers arbeitet. Potenzial für zusätzliche Anpassungen, wie das Einfügen von Aktivierungsfunktionen oder Dropout, wurde ebenfalls in Betracht gezogen.

Das Blow-Up-Layer hätte sich flexibel positionieren lassen, wodurch sowohl globale, lokale Parallelisierung entlang der Hidden-Dimension als auch die Parallelisierung der Diagonalblöcke möglich gewesen wären. Hierbei hätten die Allgather-Operationen durch Allreduce-Operationen ausgetauscht werden müssen. Aufgrund der begrenzten Zeit der Thesis und den hohen zeitlichen Aufwänden der Experimente wurde dieser Ansatz jedoch nicht weiter verfolgt.

Modell aus NVidia Modulus. Das AFNO-Modell aus der Modulus-Bibliothek wurde untersucht (siehe Abbildung A.13). Die Untersuchungen der implementierten Modelle von Modulus bestätigten Überlegungen und Implementierungen dieser Thesis. Modulus verwendet jedoch unterschiedliche Architekturen für nicht verteilte und verteilte Szenarien (Abbildung A.13, links bzw. rechts), welche sich beide von der Originalimplementation unterscheiden.

Die nicht verteilte Implementation verzichtet in den Sequenziellenblöcken auf Dropout im Fourierraum sowie nach dem MLP und nutzt GeLU als Aktivierungsfunktion. Die verteilte Variante unterscheidet sich dazu signifikant. Dazu gehören architektonische Anpassungen wie die Reformierung des Tensors bereits bei verlassen der Embedded-Layer, analog zur Erkenntnis dieser Thesis, und dazugehörige Anpassungen in der Berechnung, der Austausch der Linear-Layer durch Convolutional-Layer im MLP und Head und die Verwendung von Softshrink und GeLU anstelle von ReLU. Durch die Verwendung von Convolutional-Layern werden Geisterneuronen überflüssig, da Faltungen und Matrixmultiplikationen rechnerisch äquivalent sind [4]. Weiterhin wird im Fourier-Raum auf einem einzigen Tensor für real- und komplex-Teil mit neuen Funktionen zur Berechnung gearbeitet.

Die Kommunikationsstrategie im Modulus-Modell ähnelt meinen Überlegungen, zeigt jedoch einige Abweichungen, siehe hierzu Abbildung 3.3 und Abbildung A.13. So, ist als erster Unterschied, im Embedding-Layer eine Option die es ermöglicht AFNONets sequentiel zu nutzen. Die nächste Kommunikation entspricht der Globalen Parallelisierung, jedoch wird nach dem Scatter, beim Eintritt in das AFNO-Layer erneut gescattert. Sollte nach dem AFNO-Layer die Blocksequenz verlassen werden so wird vor der Doubble-Skip-Verbindung ein Gather ausgeführt, im Gegensatz zur Allgather-Funktion der Lokalen Ansätze. Unter Ausnahme des ersten der sequenziellen Blöcke wird beim Eintritt in das MLP ein Gather ausgeführt. Darauf folgt ein Broadcast vor dem ersten Conv-Layer, sodass alle Daten auf allen GPU vorliegen, hierdurch ist das weight-sharing über das MLP gewährleistet. Vor dem Drop-Layer wird immer ein Reduce und nach dem Drop-Layer des MLP wird, mit Ausnahme des letzten Blocks der Sequenz, ein Scatter ausgeführt. Im letzten Block der Sequenz wird nach dem Droppath ein Gather und ein Broadcast verwendet, damit alle Teile der MP-Gruppe die selben Daten in den Head geben.

Das Modulus-Modell konnte jedoch nicht direkt mit dem Ansatz dieser Arbeit verglichen werden, da es zahlreiche Probleme gab, die in der nächsten Sektion detailliert beschrieben werden.

`torch.distributed.tensor.parallel.` Ein weiterer Ansatz war die Verwendung von torch.distributed.tensor.parallel, das die Erstellung eines Parallelisierungsplans ermöglicht, vgl. Abbildung 5.11. Hierbei wird spezifiziert, wie Module gesharded werden, beispielsweise zeilen- oder spaltenweise. Der Ansatz scheiterte jedoch an der fehlenden Unterstützung für die Funktion `rfft2`, die für die Transformation in den Fourrier-Raum genutzt wird. Ohne diese Funktion konnte die Parallelisierung nicht umgesetzt werden.

```
self.model = parallelize_module(AFNONet(params).to(self.device),
                                tp_mesh,
                                {
                                    "patch_embed": RowwiseParallel(
                                        input_layouts=Replicate(),
                                        output_layouts=Shard(1),
                                    ),
                                    "pose_embed": RowwiseParallel(),
                                    "norm": SequenceParallel(),
                                    "head": ColwiseParallel(
                                        input_layouts=Shard(3),
                                        output_layouts=Replicate()
                                    ),
                                })
for layer_id, block in enumerate(self.model.blocks):
    layer_tp_plan = {
        "norm1": SequenceParallel(),
        "filter": PrepareModuleInput(
            input_layouts=(Shard(3),),
            desired_input_layouts=(Replicate(),),
        ),
        "filter.w1": ColwiseParallel(),
        "filter.w2": RowwiseParallel(output_layouts=Shard(3)),
        "norm2": SequenceParallel(),
        "mlp": PrepareModuleInput(
            input_layouts=(Shard(3),),
            desired_input_layouts=(Replicate(),),
        ),
        "mlp.fc1": ColwiseParallel(),
        "mlp.fc2": RowwiseParallel(output_layouts=Shard(3)),
    }
```

Abb. 5.11.: Beispielhafter Parallelisierungsplan des AFNONet für torch.distributed tensor.parallel.

`mpi4py`. Der letzte verworfene Ansatz ist die Nutzung von `mpi4py`. `mpi4py` ist eine Python-Bibliothek welche einen Wrapper für die Programmierung von parallelen Programmen mit MPI darstellt. Obwohl die Bibliothek grundsätzlich geeignet ist, wurde schließlich `torch.distributed` verwendet. Die Gründe für diese Entscheidung werden in der nächsten Sektion erläutert.

5.3.2. Aufgetretene Probleme

Während der Entwicklung und Implementierung traten diverse Herausforderungen auf, die hier systematisch dargestellt werden. Dazu zählen sowohl technische Hürden bei der Initialisierung und Organisation der Kommunikationsinfrastruktur als auch spezifische Probleme mit verwendeten Frameworks und Bibliotheken. Die jeweiligen Lösungsansätze werden, soweit vorhanden, ebenfalls erläutert.

Initialisierung und Organisation von Kommunikatoren. Die Initialisierung und Organisation von Kommunikatoren war mit Schwierigkeiten verbunden, da hierzu neben den internen Vorgänge von Horeka, Slurm, die zuverwendenden Module und deren Einstellungen, auch die Anforderungen der Python-Bibliotheken bekannt sein müssen.

Für die Erstellung eines funktionstüchtigen virtuellen Python-Environments war es zuerst notwendig, einen GPU-beschleunigten Knoten auf Horeka zu buchen, um CUDA-Bibliotheken und CUDA-aware MPI zu installieren. Dies ist wichtig, da es ansonsten zu Kommunikationsproblemen an den Schnittstellen zwischen Infiniband und NVLink kam. Diese Probleme führten dazu, dass die Ranks nur innerhalb eines Knotens und mit den Ranks an der Infiniband-Schnittstelle (z. B. 0, 1, 2, 3, 4, 8, 12, 16, …) korrekt kommunizieren konnten. Inkompatibilitäten zwischen MPI und NCCL wurden durch eine Mitteilung von NVIDIA bestätigt (siehe Anhang Abbildung A.16). Auf dem Horeka-Knoten musste dann zunächst das korrekte OpenMPI-Modul (Version 4.1) geladen werden, da OpenMPI 5.0 zum Zeitpunkt dieser Arbeit Probleme mit CUDA-awareness aufwies und häufig Fehler wie `mpi4py.MPI.Exception: MPI_ERR_INTERN: internal error` verursachte. Zusätzlich war die Wahl der Python-Version kritisch, da einige der notwendigen Pakete nur unter bestimmten Versionen miteinander kompatibel waren.

Für die Ausführung eines Slurm-Jobs (Run eines Experiments durch, den Workloadmanager für Clustersysteme, Slurm) mit dem Befehl `srun` musste als MPI API PMIx anstelle von PMI2 gewählt werden, da es sonst zu Fehlern wie `PMI2 Initialization failed error 14` kam.

Ein weiteres Problem war das Verständnis der notwendigen Konfigurationen für Stores des Clusters (URLs, IP-Adressen, Ports etc.). Modelle wie das Modulus-Modell von NVIDIA konnten nicht verwendet werden, da deren Stores, in einem Modul namens `DistributedManager`, hardcoded und speziell auf NVIDIA-Cluster abgestimmt waren (siehe Anhang Abbildung A.17). Generell kam es durch den DistributedManager zu vielen Problemen, die darauf zurückzuführen sind, dass dieser für den Cluster von Nvidia erstellt wurde.

Weiterhin hat das Checkpointing des Modulus-Tainings auf Horeka nicht funktioniert. Dies führte dazu, dass das Training nach der ersten Epoche in einen Fehlerzustand überging und still blockierte, bis die maximale Laufzeit (*wallclock time*) erreicht wurde.

Als Lösung wurde versucht, den DistributedManager des Moduls zu umgehen und neu zu implementieren Trotz erheblicher Bemühungen, die NVIDIA-spezifischen Implementierungen anzupassen, gelang es dann nicht, die korrekte Gradientenpropagation sicherzustellen, was dazu führte, dass das Modell keine sinnvollen Ergebnisse lieferte und dieser Ansatz verworfen wurde.

`torch.distributed.` Die Initialisierung von `torch.distributed` kann auf verschiedene Arten erfolgen: mittels den Umgebungsvariablen (MASTER_IP und MASTER_Port), über TCP-Verbindungen des Root-Rangs, durch eine gemeinsam genutzte Datei, auf die jeder Knoten zugreifen muss, um Informationen zu schreiben, oder explizit durch Key-Value-Store, was wissen über den Cluster voraussetzt. Jedoch traten bei allen Methoden erhebliche Probleme auf. Umgebungsvariablen und TCP konnten im Vorfeld nicht korrekt gesetzt werden, da diese im Voraus nicht bekannt sind, was die Verwendung dieser Initialisierungsarten verhinderte. Die Initialisierung über eine gemeinsame Datei funktionierte aus unbekannten Gründen ebenfalls nicht zuverlässig. Schließlich wurde ein Code-Snippet aus der HeAT-Bibliothek bereitgestellt, welches die Initialisierung von `torch.distributed` über Slurm ohne weitere Komplikationen ermöglichte.

Ein weiteres Problem betraf die fehlerhafte Verarbeitung von Tensoren mit nichtkontigen Dimensionen. Wie in vorherigen Kapiteln beschrieben, führte dies dazu, dass Daten an falschen Speicherpositionen landeten, z. B. wurden Daten der Dimensionen 1 und 2 in Dimension 3 abgelegt, siehe Abbildung A.14 Dies liegt vermutlich daran, dass `torch.distributed` die Tensoren intern in Numpy-Arrays umwandelt und diese anschließend ohne Berücksichtigung von Offsets aufteilt und verschickt. Im Gegensatz dazu behandelt HeAT diese Tensoren korrekt.

Das Problem wurde schließlich durch die Implementierung einer Umformung und anschließenden Sortierung des Speichers, wie in Abschnitt 4.3 beschrieben, gelöst.

Gradientendesynchronisation. Nach der Behebung der oben genannten Probleme trat ein weiteres Phänomen auf: Der Loss-Wert wurde nach etwa 5 bis 50 Epochen (abhängig von der Modellgröße und der Anzahl der GPUs) zu `NaN`. Dieses Problem wurde auf eine Gradienten-Desynchronisation zurückgeführt, die bei asynchroner Kommunikation auftreten kann.

Ein Beispiel hierfür ist, wenn ausschließlich Rank 0 die Validierung übernimmt, während andere Ränge bereits mit der nächsten Epoche beginnen. Um dies zu vermeiden, müssen Synchronisationsbarrieren an geeigneten Stellen gesetzt werden. Diese verhindern, dass Ränge fortfahren, bis alle Ränge die Barriere erreicht haben. Im Gegensatz zu `torch.distributed`, bei dem Kommunikation standardmäßig asynchron erfolgt, ist Kommunikation in MPI implizit synchron (außer es werden explizit asynchrone Funktionen genutzt).

Warum wurde MPI nicht verwendet? Der ursprüngliche Plan war, MPI mit `mpi4py` zu verwenden. Zum Beginn dieser Arbeit war jedoch die Version 4.0.0 von `mpi4py` noch nicht veröffentlicht, die einige der Probleme mit Buffern hätte verringern können. Zudem stellte sich die Nutzung von MPI als herausfordernd dar:

1. **Zwei Programmieransätze**:
   - **Funktionen in Lowercase**: Hier können Python-Objekte direkt übergeben werden, und der Wrapper kümmert sich um die Pufferübertragung, Offsets und variable Größen. Allerdings treten dieselben Probleme wie bei torch.distributed auf, da intern Numpy-Arrays verwendet werden.
   - **Funktionen in Uppercase**: Hier müssen Sende- und Empfangs-Puffer mit Offsets, Nummer der zu übertragenden Elemente und Datentyp selbst angelegt werden. Erst in mpi4py version 5 konnten modifizierte Speicherrepräsentationen und Pointer als Puffer genutzt werden.
2. **Leistung**: MPI ist signifikant langsamer als `torch.distributed`. Beispielsweise wurde für eine `allgather`-Operation innerhalb eines Knotens eine Zeit von 2,36 Sekunden mit MPI gemessen, während `torch.distributed` dieselbe Operation in 0,00033 Sekunden ausführen konnte. Mithilfe der HeAT-Bibliothek konnten die Zeiten für `scatter` und `gather` auf 0,13 bzw. 0,18 Sekunden reduziert werden. Durch den zusätzlichen Reformatierungstrick sanken diese Werte schließlich auf 0,0289 und 0,033 Sekunden (siehe Abbildung A.14). Diese Werte sind weitaus besser als die Werte des Barebone MPI jedoch ein hundertfaches langsamer als `torch.distributed`.

torch.distributed.DataDistributedParallel() und MPI. Ein weiteres Problem ergab sich durch die Verwendung des `torch.distributed.DataDistributedParallel (DDP)`-Wrappers in Kombination mit MPI. Während die Kommunikation im datenparallelen Fall mit DDP asynchron erfolgt, ist sie in MPI standardmäßig synchron. Diese Diskrepanz führte zu zusätzlichen Herausforderungen und war letztlich der Grund, weshalb die Nutzung von MPI vollständig aufgegeben wurde.

Darüber hinaus synchronisierte DDP standardmäßig die Gradienten über alle GPUs hinweg, anstatt nur die relevanten Teile der Modellparallelisierung. Dies war in der spezifischen Implementierung problematisch, da dadurch unerwünschte Berechnungen und Kommunikationsübertragungen stattfanden, die das Modell ineffizient machten.

Um dieses Problem zu lösen, war es notwendig, eine eigene Implementierung der datenparallelen Kommunikation zu entwickeln, wie bereits in den vorherigen Abschnitten beschrieben. Diese Anpassung ermöglichte eine gezielte Synchronisation ausschließlich zwischen den relevanten Teilen der Modellparallelisierung, wodurch die Effizienz und Konsistenz des Trainings verbessert werden konnte.

6. Fazit

In dieser Masterarbeit wurden modellparallele Architekturen für den Adaptive Fourier Neural Operator entwickelt und hinsichtlich ihrer Skalierbarkeit und Effizienz untersucht. Ziel war es, das Ursprungsmodell um die Möglichkeit des modellparallelen Trainings zu erweitern und dabei die Genauigkeit des Modells beizubehalten oder idealerweise zu verbessern. Zudem wurde der durch die Parallelisierung entstehende Overhead analysiert und die Laufzeiten zwischen dem ursprünglichen und den modellparallelen Architekturen verglichen. Hierzu mussten viele Hürden überwunden und Ansätze verworfen werden.

Die Ergebnisse der Arbeit zeigen, dass die entwickelten modellparallelen Architekturen unter den gegebenen Testbedingungen keine signifikante bis keine Skalierbarkeit für den Anwendungsfall der Wettervorhersage aufwiesen. Ein interessanter Fund war hierbei, dass das Ursprungsmodell ebenfalls keine signifikatne Skalierbarkeit aufweist. Ein einzelnes Modell (globaler Ansatz, MP-Size von 2, Batch-Size von 2) konnte ein besseres Skalierungsverhalten als das Ursprungsmodell aufweisen, jedoch bei 17% höheren Loss-Werten. Mehrere Durchläufe erreichten bessere Loss-Werte als die Baseline jedoch bei zwei bis dreieinhalb-facher Laufzeit. Diese besseren Losswerte, die bei Untersuchungen starker Skalierung beobachtet wurden, lassen sich vermutlich auf eine Veränderung der Batch-Size zurückführen. Ein weiterführender Test könnte zeigen, ob das Baseline-Modell mit einer größeren Batch-Size ähnliche Ergebnisse erzielen kann. Einschränkungen wie Out-of-Memory-Fehler bei mehr als fünf Samples pro GPU begrenzten hierbei jedoch die Skalierbarkeituntersuchung.

Ein weiterer zentraler Befund dieser Arbeit ist das Potenzial lokal parallelisierter Modelle. Diese zeigten im Vergleich zu globalen Ansätzen eine bessere Leistung, insbesondere im Hinblick auf die Loss-Werte. Dies deutet darauf hin, dass Faktoren wie Weight-Sharing und häufigerer Datenaustausch eine entscheidende Rolle für die Modellgenauigkeit spielen.

Gleichzeitig erwies sich die datenparallele Ausführung als die effizienteste Lösung für Stand-Alone-Anwendungen des AFNO. Sollte der AFNO wie ursprünglich als Backbone in FourCastNet eingesetzt werden, könnte ein Training auf weniger GPUs sinnvoll sein, um einen Kompromiss zwischen Gesamttrainingszeit und Validierungsloss zu finden. Dies ist insbesondere deshalb relevant, da der AFNO aufgrund seiner Eigenschaften, Operationen zwischen Banachräumen zu lernen, lediglich einmal trainiert werden muss.

Die Ergebnisse dieser Arbeit verdeutlichen die Herausforderungen und Möglichkeiten modellparalleler Ansätze. Während datenparallele Strategien aktuell die effizienteste Lösung für den Einsatz auf Wetterdaten darstellen, könnten zukünftige Arbeiten die untersuchten Architekturen weiter optimieren und ihr Potenzial für andere Anwendungsfälle weiter ausschöpfen.

7. Ausblick

Basierend auf den Erkenntnissen dieser Arbeit ergeben sich mehrere vielversprechende Ansätze für zukünftige Untersuchungen.

Eine Möglichkeit besteht darin, die Modelle auf Knoten mit acht GPUs zu betreiben. Dadurch könnten Verzögerungen vermieden werden, die durch die Verwendung inhomogener Kommunikationstechnologien innerhalb des Modells durch die Gegebenheiten des Rechenclusters entstehen. Lediglich die Gradienten-Synchronisation bleibt weiterhin ein potenzieller limitierender Faktor, welcher jedoch bei reinem Datenparallelismus auch besteht.

Ein weiterer Ansatz wäre die vollständige Implementierung und Evaluation lokal parallelisierter Modelle. Eine vollständige Implementierung könnte Aufschluss darüber geben, ob diese eine bessere Alternative zu datenparallelen Ansätzen darstellen können. Anschließend könnten diese mit dem modellparallelen Modell aus Modulus verglichen werden, um deren Vor- und Nachteile im Hinblick auf Performance und Effizienz zu analysieren. Ebenso wäre es interessant, dabei einen direkten Vergleich zwischen den Modulus-Modellen und dem ursprünglichen AFNO-Modell durchzuführen.

Unter dem Gesichtspunkt verworfener Ansätze wie die Untersuchung von Modulus, bietet die Implementation, des ebenfalls verworfenen Ansatzes, der Erweiterung der Modellkapazität durch eine Vergrößerung der Hidden-Dimension eine spannende Perspektive. Die Untersuchung der Auswirkungen dieser Anpassung könnte neue Optimierungsmöglichkeiten aufzeigen und die Leistungsfähigkeit des Modells weiter steigern.

Weitere Optimierungsmöglichkeiten und interessante Untersuchungen liegen in einer Hyperparameter-Optimierung der modellparallelen Architekturen. Die Anpassung von Parametern wie der Tiefe des Modells, der Anzahl an Diagonalblöcken oder der Größe der Patches könnte Aufschluss darüber geben, wie die Modelle weiter verbessert und effizienter gestaltet werden können.

Aufbauend auf den Erkenntnissen dieser Arbeit wäre es außerdem interessant, Spherical Harmonics Neural Operators (SNO) aus FourCastNetv2 zu untersuchen. Diese arbeiten im Gegensatz zu AFNO nicht auf kartesischen sondern spherischen Koordinaten, wodurch die Gegebenheiten des Planeten besser repräsentiert und modelliert werden können.[9] Dabei könnte insbesondere geprüft werden, ob dieser Neural Operator eine bessere Skalierbarkeit und Performance als AFNO aufweist.

Literatur

[1] Gene M. Amdahl. „Validity of the single processor approach to achieving large scale computing capabilities“. en. In: *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*. Atlantic City, New Jersey: ACM Press, 1967, S. 483. DOI: 10.1145/1465482.1465560. URL: http://portal.acm.org/citation.cfm?doid=1465482.1465560 (besucht am 29. 12. 2024).

[2] Jason Ansel u. a. „PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation“. In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, Apr. 2024. DOI: 10.1145/3620665.3640366. URL: https://pytorch.org/assets/pytorch2-2.pdf.

[3] Banach, Stefan. *Theorie Des Operations Lineaires*. fr. 1932. URL: http://archive.org/details/in.ernet.dli.2015.493033 (besucht am 28. 12. 2024).

[4] Tal Ben-Nun und Torsten Hoefler. *Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis*. arXiv:1802.09941 [cs]. Sep. 2018. DOI: 10.48550/arXiv.1802.09941. URL: http://arxiv.org/abs/1802.09941 (besucht am 02. 01. 2025).

[5] Kaifeng Bi u. a. „Accurate medium-range global weather forecasting with 3D neural networks“. In: *Nature* 619.7970 (2023), S. 533–538.

[6] Kaifeng Bi u. a. *Pangu-Weather: A 3D High-Resolution Model for Fast and Accurate Global Weather Forecast*. arXiv:2211.02556 [physics]. Nov. 2022. DOI: 10.48550/arXiv.2211.02556. URL: http://arxiv.org/abs/2211.02556 (besucht am 25. 12. 2024).

[7] Massimo Bonavita. „On Some Limitations of Current Machine Learning Weather Prediction Models“. en. In: *Geophysical Research Letters* 51.12 (Juni 2024), e2023GL107377. ISSN: 0094-8276, 1944-8007. DOI: 10.1029/2023GL107377. URL: https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2023GL107377 (besucht am 25. 12. 2024).

[8] Massimo Bonavita. *On some limitations of data-driven weather forecasting models*. arXiv:2309.08473 [stat]. Nov. 2023. DOI: 10.48550/arXiv.2309.08473. URL: http://arxiv.org/abs/2309.08473 (besucht am 25. 12. 2024).

[9] Boris Bonev u. a. *Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere*. arXiv:2306.03838 [cs]. Juni 2023. DOI: 10.48550/arXiv.2306.03838. URL: http://arxiv.org/abs/2306.03838 (besucht am 02. 01. 2025).

[10] Alexandre Joel Chorin und Jerrold E. Marsden. *A mathematical introduction to fluid mechanics*. 3rd ed. Texts in applied mathematics 4. New York: Springer, 1998. ISBN: 978-0-387-97918-2.

[11] Modulus Contributors. *NVIDIA Modulus: An open-source framework for physics-based deep learning in science and engineering*. Feb. 2023. URL: https://github.com/NVIDIA/modulus (besucht am 02. 01. 2025).

[12] Copernicus Climate Change Service. *Complete ERA5 global atmospheric reanalysis*. 2023. DOI: 10.24381/CDS.143582CF. URL: https://cds.climate.copernicus.eu/doi/10.24381/cds.143582cf (besucht am 29. 12. 2024).

[13] G. Cybenko. „Approximation by superpositions of a sigmoidal function“. en. In: *Mathematics of Control, Signals, and Systems* 2.4 (Dez. 1989), S. 303–314. ISSN: 0932-4194, 1435-568X. DOI: 10.1007/BF02551274. URL: http://link.springer.com/10.1007/BF02551274 (besucht am 11. 12. 2024).

[14] Lisandro Dalcin. *MPI for Python (mpi4py)*. Mai 2016. URL: https://mpi4py.readthedocs.io/en/latest/index.html (besucht am 02. 01. 2025).

[15] D. P. Dee u. a. „Toward a Consistent Reanalysis of the Climate System“. en. In: *Bulletin of the American Meteorological Society* 95.8 (Aug. 2014), S. 1235–1248. ISSN: 0003-0007, 1520-0477. DOI: 10.1175/BAMS-D-13-00043.1. URL: http://journals.ametsoc.org/doi/10.1175/BAMS-D-13-00043.1 (besucht am 25. 12. 2024).

[16] Jacob Devlin u. a. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv:1810.04805 [cs]. Mai 2019. DOI: 10.48550/arXiv.1810.04805. URL: http://arxiv.org/abs/1810.04805 (besucht am 25. 12. 2024).

[17] Alexey Dosovitskiy u. a. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929 [cs]. Juni 2021. DOI: 10.48550/arXiv.2010.11929. URL: http://arxiv.org/abs/2010.11929 (besucht am 11. 12. 2024).

[18] Peter D. Dueben und Peter Bauer. „Challenges and design choices for global weather and climate models based on machine learning“. en. In: *Geoscientific Model Development* 11.10 (Okt. 2018), S. 3999–4009. ISSN: 1991-9603. DOI: 10.5194/gmd-11-3999-2018. URL: https://gmd.copernicus.org/articles/11/3999/2018/ (besucht am 25. 12. 2024).

[19] ECMWF. *ERA5 Data Documentation*. URL: https://confluence.ecmwf.int/display/CKB/ERA5%3A+data+documentation#heading-Parameterlistings.

[20] Jeffrey L Elman. „Finding structure in time“. In: *Cognitive science* 14.2 (1990). Publisher: Wiley Online Library, S. 179–211.

[21] Alireza Farokhi. *Array vs List — Compare Array and List performance in C#*. en. Juni 2022. URL: https://alirezafarokhi.medium.com/array-vs-list-compare-array-and-list-performance-in-c-722316603c8c (besucht am 01. 01. 2025).

[22] *Global Forecast System (GFS)*. en. Aug. 2020. URL: https://www.ncei.noaa.gov/products/weather-climate-models/global-forecast (besucht am 25. 12. 2024).

[23] Priya Goyal u. a. *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour*. arXiv:1706.02677 [cs]. Apr. 2018. DOI: 10.48550/arXiv.1706.02677. URL: http://arxiv.org/abs/1706.02677 (besucht am 02. 01. 2025).

[24] John Guibas u. a. *Adaptive Fourier Neural Operators: Efficient Token Mixers for Transformers*. arXiv:2111.13587 [cs]. März 2022. DOI: 10.48550/arXiv.2111.13587. URL: http://arxiv.org/abs/2111.13587 (besucht am 11. 12. 2024).

[25] John L. Gustafson. „Reevaluating Amdahl's law". en. In: *Communications of the ACM* 31.5 (Mai 1988), S. 532–533. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/42411.42415. URL: https://dl.acm.org/doi/10.1145/42411.42415 (besucht am 29. 12. 2024).

[26] *Hermitian matrix - Encyclopedia of Mathematics*. Adaptiert aus dem Buch Encyclopedia of Mathematics, ISBN 1402006098, Michiel Hazewinkel, 1994. URL: https://encyclopediaofmath.org/index.php?title=Hermitian_matrix (besucht am 28. 12. 2024).

[27] Hans Hersbach u. a. „The ERA5 global reanalysis". en. In: *Quarterly Journal of the Royal Meteorological Society* 146.730 (Juli 2020), S. 1999–2049. ISSN: 0035-9009, 1477-870X. DOI: 10.1002/qj.3803. URL: https://rmets.onlinelibrary.wiley.com/doi/10.1002/qj.3803 (besucht am 25. 12. 2024).

[28] Mark D. Hill. „What is scalability?" In: *SIGARCH Comput. Archit. News* 18.4 (Dez. 1990), S. 18–21. ISSN: 0163-5964. DOI: 10.1145/121973.121975. URL: https://dl.acm.org/doi/10.1145/121973.121975 (besucht am 20. 12. 2024).

[29] Sepp Hochreiter und Jürgen Schmidhuber. „Long Short-Term Memory". en. In: *Neural Computation* 9.8 (Nov. 1997), S. 1735–1780. ISSN: 0899-7667, 1530-888X. DOI: 10.1162/neco.1997.9.8.1735. URL: https://direct.mit.edu/neco/article/9/8/1735-1780/6109 (besucht am 11. 12. 2024).

[30] Frank Krysiak. *Ist die künstliche Intelligenz eine Klimasünderin, Herr Krysiak?* de. Wissenschaftsmagazin. Feb. 2020. URL: https://www.unibas.ch/de/Aktuell/Uni-Nova/Uni-Nova-136/Uni-Nova-136-Klimasuenderin-Kuenstliche-Intelligenz-Krysiak.html (besucht am 02. 01. 2025).

[31] Thorsten Kurth u. a. „FourCastNet: Accelerating Global High-Resolution Weather Forecasting Using Adaptive Fourier Neural Operators". en. In: *Proceedings of the Platform for Advanced Scientific Computing Conference*. Davos Switzerland: ACM, Juni 2023, S. 1–11. ISBN: 979-8-4007-0190-0. DOI: 10.1145/3592979.3593412. URL: https://dl.acm.org/doi/10.1145/3592979.3593412 (besucht am 11. 12. 2024).

[32] Remi Lam u. a. *GraphCast: Learning skillful medium-range global weather forecasting*. Version Number: 2. 2022. DOI: 10.48550/ARXIV.2212.12794. URL: https://arxiv.org/abs/2212.12794 (besucht am 25. 12. 2024).

[33] Remi Lam u. a. „Learning skillful medium-range global weather forecasting". en. In: *Science* 382.6677 (Dez. 2023), S. 1416–1421. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.adi2336. URL: https://www.science.org/doi/10.1126/science.adi2336 (besucht am 11. 12. 2024).

[34] Y. LeCun u. a. „Backpropagation Applied to Handwritten Zip Code Recognition". In: *Neural Computation* 1.4 (1989), S. 541–551.

[35] Y. Lecun u. a. „Gradient-based learning applied to document recognition“. In: *Proceedings of the IEEE* 86.11 (Nov. 1998), S. 2278–2324. ISSN: 00189219. DOI: 10.1109/5.726791. URL: http://ieeexplore.ieee.org/document/726791/ (besucht am 11. 12. 2024).

[36] Yann LeCun, Yoshua Bengio und Geoffrey Hinton. „Deep learning“. en. In: *Nature* 521.7553 (Mai 2015), S. 436–444. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/nature14539. URL: https://www.nature.com/articles/nature14539 (besucht am 11. 12. 2024).

[37] Zongyi Li u. a. *Fourier Neural Operator for Parametric Partial Differential Equations*. arXiv:2010.08895 [cs]. Mai 2021. DOI: 10.48550/arXiv.2010.08895. URL: http://arxiv.org/abs/2010.08895 (besucht am 11. 12. 2024).

[38] Zongyi Li u. a. *Neural Operator: Graph Kernel Network for Partial Differential Equations*. arXiv:2003.03485 [cs]. März 2020. DOI: 10.48550/arXiv.2003.03485. URL: http://arxiv.org/abs/2003.03485 (besucht am 15. 12. 2024).

[39] Lu Lu, Pengzhan Jin und George Em Karniadakis. „DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators“. In: *Nature Machine Intelligence* 3.3 (März 2021). arXiv:1910.03193 [cs], S. 218–229. ISSN: 2522-5839. DOI: 10.1038/s42256-021-00302-5. URL: http://arxiv.org/abs/1910.03193 (besucht am 15. 12. 2024).

[40] Peter J. Lynch. *The emergence of numerical weather prediction: Richardson's dream*. eng. Cambridge: Cambridge university press, 2006. ISBN: 978-0-521-85729-1.

[41] Mark Mantel. *Zigtausende H100-GPUs: Google baut Top-2-Supercomputer*. Nachrichten-Website. Mai 2023. URL: https://www.heise.de/news/Google-steigt-ins-Exascale-Rennen-ein-KI-Training-mit-zigtausenden-Nvidia-GPUs-9019317.html.

[42] Clare D. McGillem und George R. Cooper. *Continuous and discrete signal and system analysis*. eng. 2. ed. Holt, Rinehart and Winston series in electrical engineering, electronics, and systems. New York: Holt, Rinehart und Winston, 1984. ISBN: 978-0-03-061703-4.

[43] Leonardo Olivetti und Gabriele Messori. „Do data-driven models beat numerical models in forecasting weather extremes? A comparison of IFS HRES, Pangu-Weather, and GraphCast“. en. In: *Geoscientific Model Development* 17.21 (Nov. 2024), S. 7915–7962. ISSN: 1991-9603. DOI: 10.5194/gmd-17-7915-2024. URL: https://gmd.copernicus.org/articles/17/7915/2024/ (besucht am 25. 12. 2024).

[44] OpenAI. *ChatGPT: Optimizing Language Models for Dialogue*. 2023. URL: https://openai.com/chatgpt.

[45] Jaideep Pathak und Morteza Mardani. *FourCastNet*. original-date: 2023-01-11T17:04:29Z. Jan. 2022. URL: https://github.com/NVlabs/FourCastNet (besucht am 02. 01. 2025).

[46] Jaideep Pathak u. a. *FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators*. arXiv:2202.11214 [physics]. Feb. 2022. DOI: 10.48550/arXiv.2202.11214. URL: http://arxiv.org/abs/2202.11214 (besucht am 11. 12. 2024).

[47] Arnold Reusken. „Numerical Methods for the Navier-Stokes Equations". In: *RWTH, Aachen* (2012).

[48] F. Rosenblatt. „The perceptron: A probabilistic model for information storage and organization in the brain." en. In: *Psychological Review* 65.6 (1958), S. 386–408. ISSN: 1939-1471, 0033-295X. DOI: 10.1037/h0042519. URL: https://doi.apa.org/doi/10.1037/h0042519 (besucht am 11. 12. 2024).

[49] David E. Rumelhart, Geoffrey E. Hinton und Ronald J. Williams. „Learning representations by back-propagating errors". en. In: *Nature* 323.6088 (Okt. 1986), S. 533–536. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/323533a0. URL: https://www.nature.com/articles/323533a0 (besucht am 25. 12. 2024).

[50] Itay Safran und Ohad Shamir. *Depth-Width Tradeoffs in Approximating Natural Functions with Neural Networks*. arXiv:1610.09887 [cs]. Mai 2020. DOI: 10.48550/arXiv.1610.09887. URL: http://arxiv.org/abs/1610.09887 (besucht am 28. 12. 2024).

[51] Suranjana Saha u. a. „The NCEP Climate Forecast System Reanalysis". en. In: *Bulletin of the American Meteorological Society* 91.8 (Aug. 2010), S. 1015–1058. ISSN: 0003-0007, 1520-0477. DOI: 10.1175/2010BAMS3001.1. URL: https://journals.ametsoc.org/doi/10.1175/2010BAMS3001.1 (besucht am 25. 12. 2024).

[52] KIT SCC SCS. *NHR@KIT User Documentation*. 2023. URL: https://www.nhr.kit.edu/userdocs/horeka/batch/ (besucht am 02. 01. 2025).

[53] Mohammad Shoeybi u. a. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053 [cs]. März 2020. DOI: 10.48550/arXiv.1909.08053. URL: http://arxiv.org/abs/1909.08053 (besucht am 11. 12. 2024).

[54] Robert Tibshirani. „Regression Shrinkage and Selection Via the Lasso". en. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58.1 (Jan. 1996), S. 267–288. ISSN: 1369-7412, 1467-9868. DOI: 10.1111/j.2517-6161.1996.tb02080.x. URL: https://academic.oup.com/jrsssb/article/58/1/267/7027929 (besucht am 28. 12. 2024).

[55] Ashish Vaswani u. a. *Attention Is All You Need*. Version Number: 7. 2017. DOI: 10.48550/ARXIV.1706.03762. URL: https://arxiv.org/abs/1706.03762 (besucht am 11. 12. 2024).

[56] Cong Wang u. a. „Scalable massively parallel computing using continuous-time data representation in nanoscale crossbar array". en. In: *Nature Nanotechnology* 16.10 (Okt. 2021), S. 1079–1085. ISSN: 1748-3387, 1748-3395. DOI: 10.1038/s41565-021-00943-y. URL: https://www.nature.com/articles/s41565-021-00943-y (besucht am 02. 01. 2025).

[57] Yuting Wu und Wei Xue. „Data-Driven Weather Forecasting and Climate Modeling from the Perspective of Development“. en. In: *Atmosphere* 15.6 (Juni 2024), S. 689. ISSN: 2073-4433. DOI: 10.3390/atmos15060689. URL: https://www.mdpi.com/2073-4433/15/6/689 (besucht am 25. 12. 2024).

[58] Lingxi Xie. *Pangu-Weather*. original-date: 2023-01-11T17:04:29Z. Jan. 2023. URL: https://github.com/198808xc/Pangu-Weather (besucht am 02. 01. 2025).

Abkürzungsverzeichnis

AFNO Adaptive Fourier Neural Operator.

CNN Convolutional Neural Network.

DDP Distributed Dataparallel.

DNN Deep Neural Network.

ECMWF European Centre for Medium-Range Weather Forecasts.

ERA5 ECMWF reanalysis v5.

FFT Fast Fourier Transformation.

FNO Fourier Neural Operator.

FourCastNet Fourier Forecasting Neural Network.

KI Künstliche Intelligenz.

LSTM Long-Short Term Memory.

MLP Multilayer Perceptron.

MP-Gruppe modellparallele Gruppe.

MPI Message Passing Interface.

NN Neural Network.

PDE Partial Differential Equation.

RNN Recurrent Neural Network.

A. Anhang

Als letztes möchte ich auf die Predictionbilder eingehen was meiner Meinung nach nicht in den Hauptteil dieser Thesis gepasst hätte. Hier kann man bei allen die Patches erkennen, bei manchen sehen diese sehr viel pixliger aus als bei anderen, dies korreliert mit der MP-Size. Jedoch können bei allen Bildern Batchgrenzen erkannt werden, dies liegt womöglich daran, dass das AFNO nur das Backbonenetz des Fourcastnets ist ist.

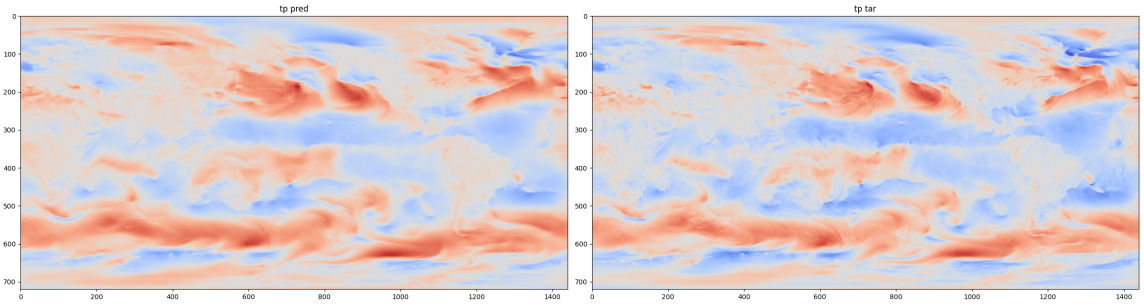


Abb. A.1.: Baseline Output bei 64 GPU in Epoche 50.

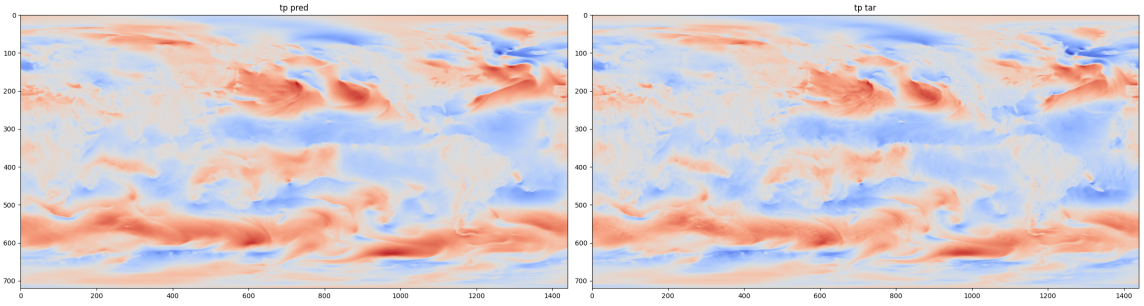


Abb. A.2.: Output Baseline 8GPU Ep50

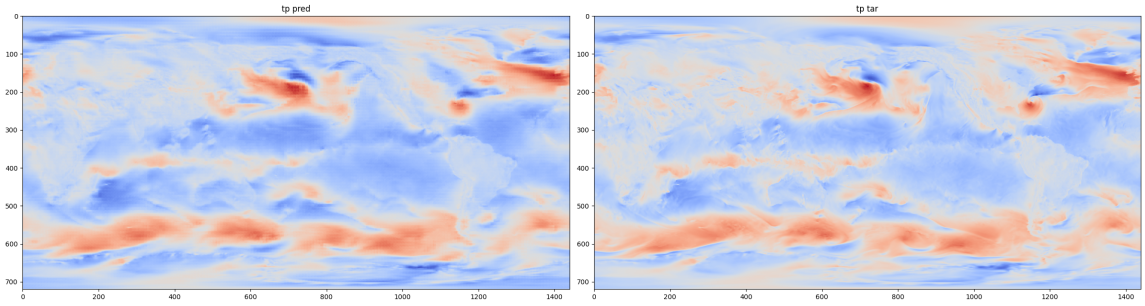


Abb. A.3.: Output bei 64 GPU Global Weak MP8 in Epoche 50.

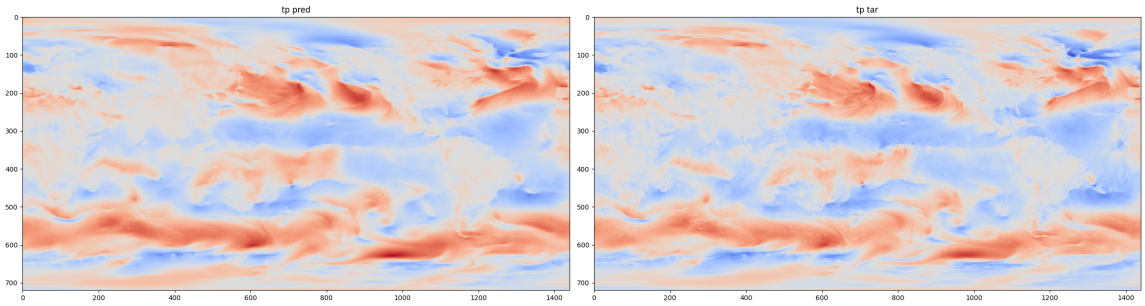


Abb. A.4.: Output bei 64 GPU Global Strong MP8 in Epoche 50.

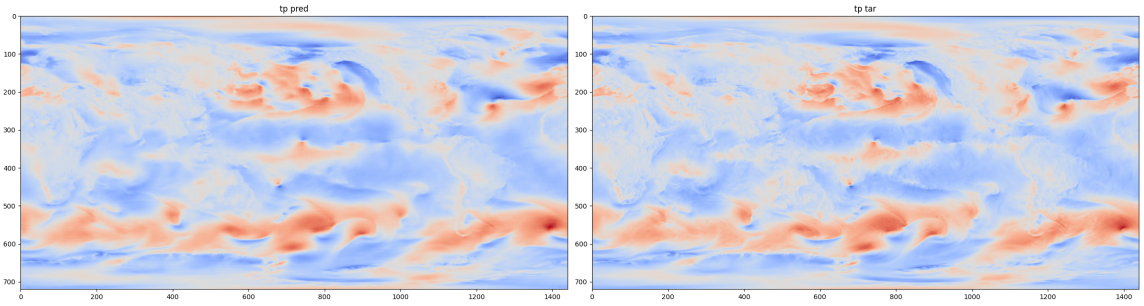


Abb. A.5.: Output bei 64 GPU Global Weak MP2 in Epoche 50.

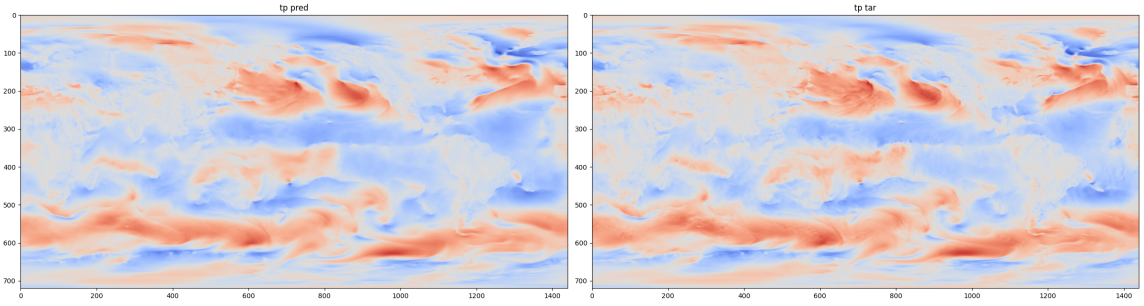


Abb. A.6.: Output bei 64 GPU Global Strong MP2 in Epoche 50.

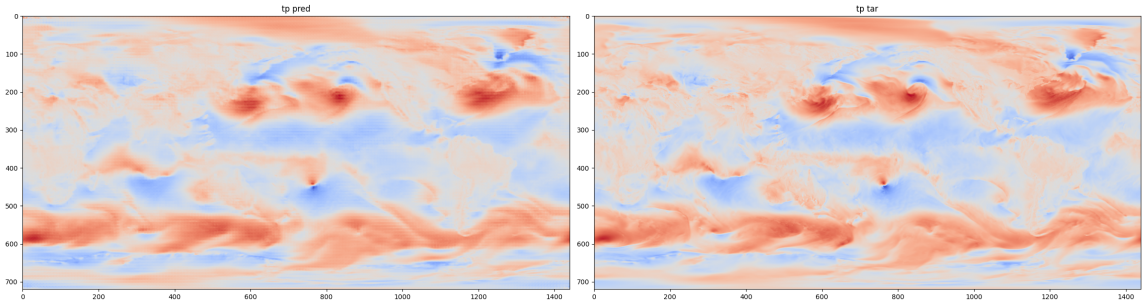


Abb. A.7.: Output bei 64 GPU Hidden Strong MP8 in Epoche 40.

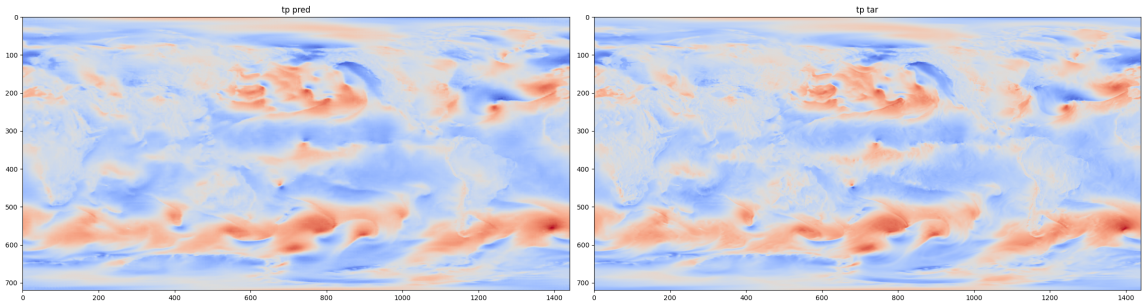


Abb. A.8.: Output bei 64 GPU Diagonal Weak MP2 in Epoche 40.

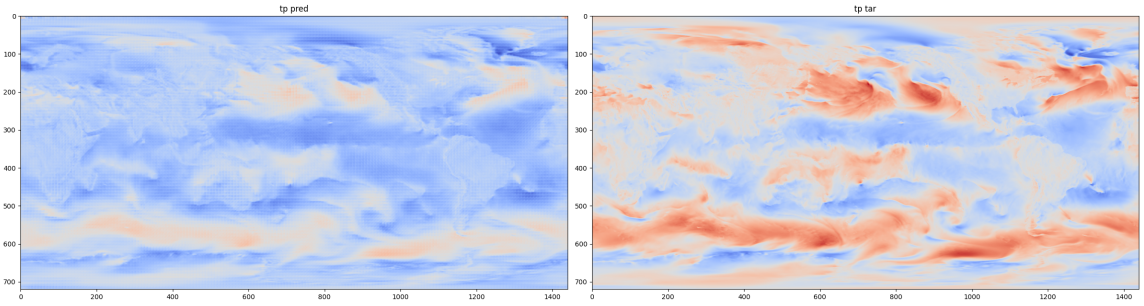


Abb. A.9.: Output bei 64 GPU Diagonal Strong MP2 in Epoche 40.

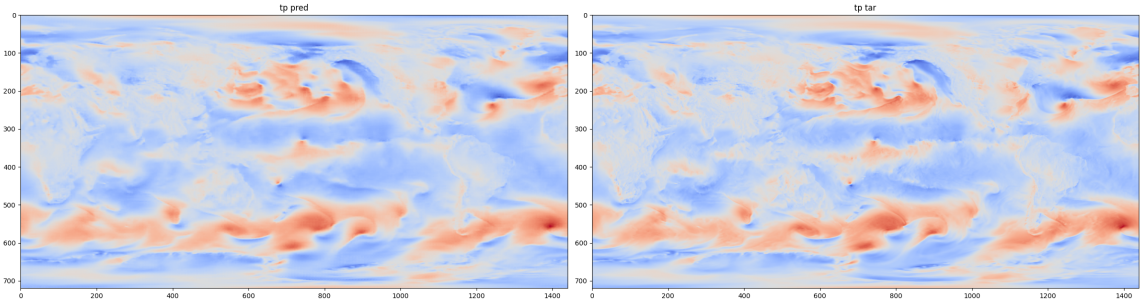


Abb. A.10.: Output bei 64 GPU Hidden Weak MP2 in Epoche 50.

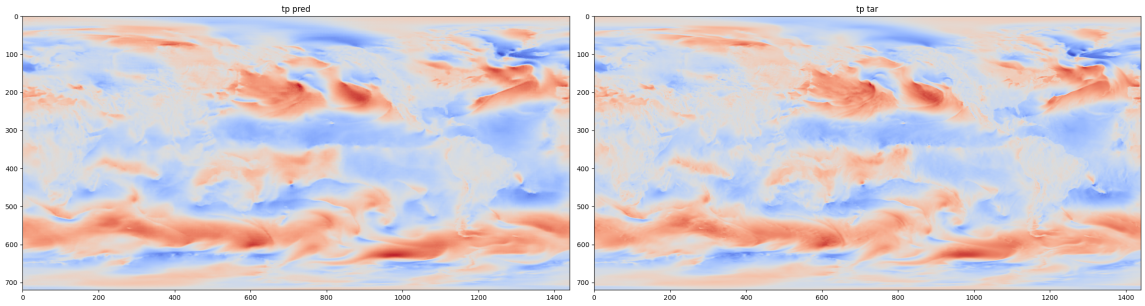


Abb. A.11.: Output bei 64 GPU Hidden Strong MP2 in Epoche 50.

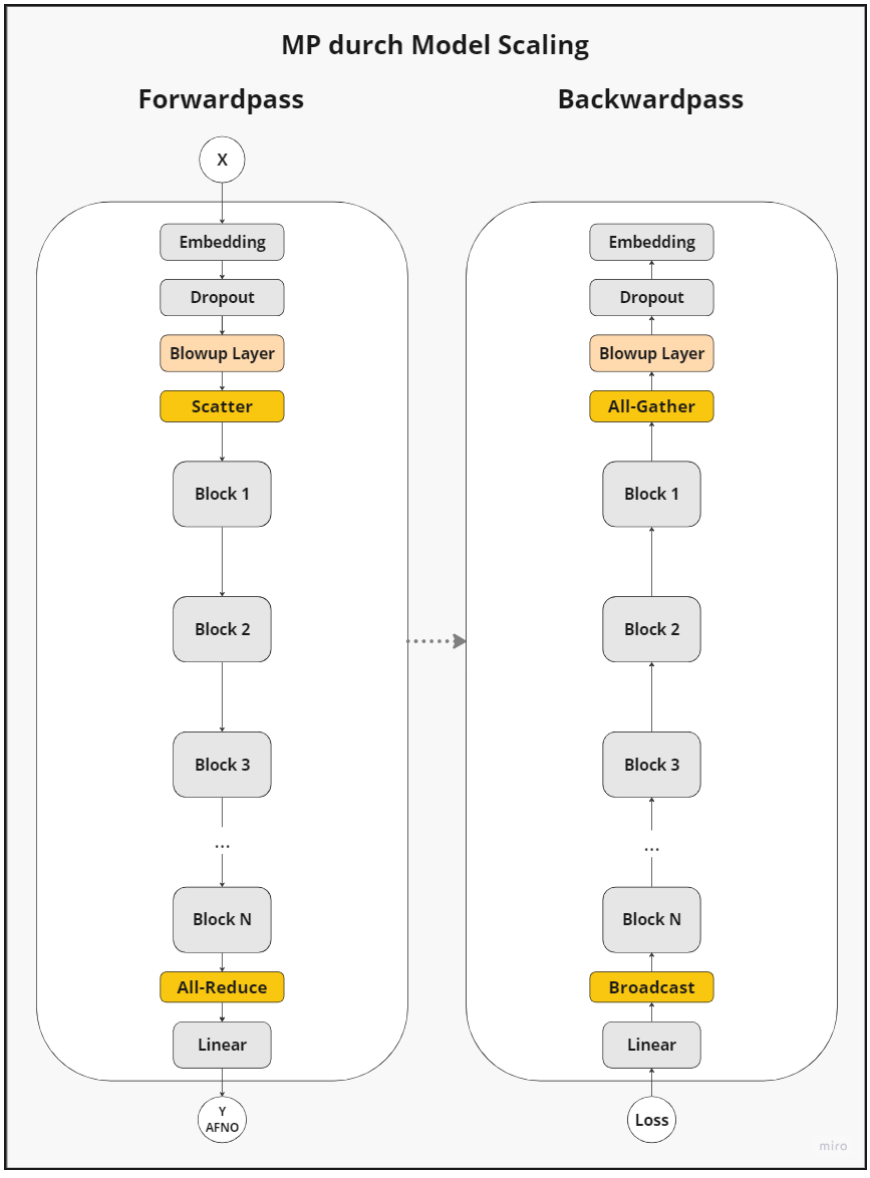


Abb. A.12.: Mogliche Implementation eines Block up Layers für den verworfen Ansatz

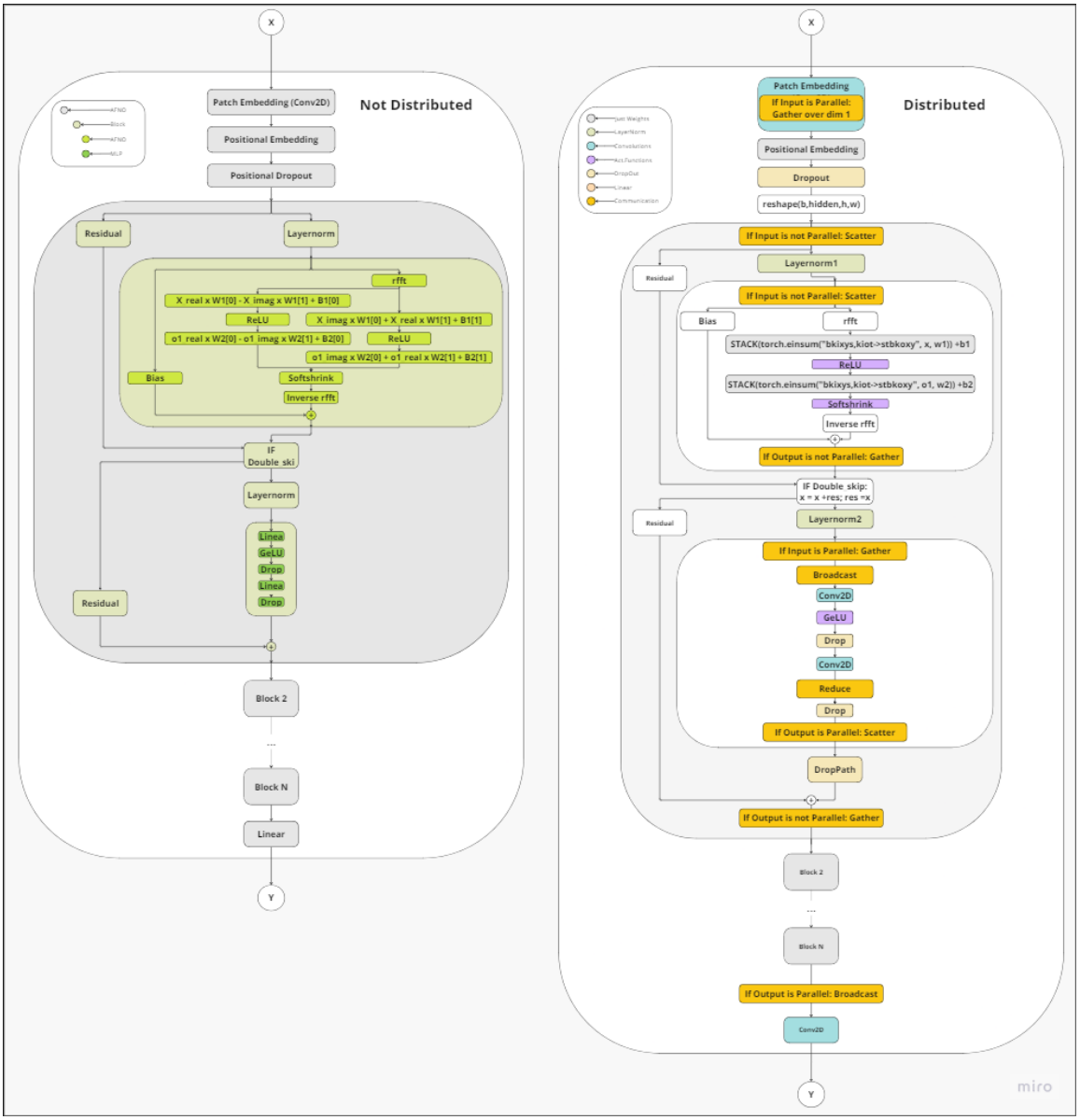


Abb. A.13.: Darstellung der Modulusmodelle nach vorbild von Abbildung 2.1

```
Time to create tensor: 0.0272982120513916 rank: 1
Time to create tensor: 0.02634906768798828 rank: 0
TorchDist Scatter time: 0.037339210510253906 rank: 0
TorchDist Scatter time: 0.07191777229309082 rank: 1
TorchDist Gather time: 0.0003330707550048828 rank: 0
TorchDist Gather time: 0.00029921531677246094 rank: 1
HEAT Scatter time: 0.13103461265563965 rank: 0
HEAT Scatter time: 0.1279287338256836 rank: 1
HEAT Gather time: 0.18460965156555176 rank: 0
HEAT Gather time: 0.18750405311584473 rank: 1
Rank: 0 TorchDist: False HEAT: True Both: False
input: tensor([[[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.]],

        [[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.]],

        [[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.]]], device='cuda:0')
outTorch: tensor([[[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 0., 0., 0.],
         [0., 0., 0.,  ..., 1., 1., 1.]],

        [[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 0., 0., 0.],
         [0., 0., 0.,  ..., 1., 1., 1.]],

        [[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 0., 0., 0.],
         [0., 0., 0.,  ..., 1., 1., 1.]]], device='cuda:0')
outHeat: tensor([[[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.]],

        [[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.]],

        [[0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.],
         [0., 0., 0.,  ..., 1., 1., 1.]]], device='cuda:0')
Errors Torch: 3110400;
Errors Heat: 0,
```

Abb. A.14.: Fehler beim verteilen von nicht Kontigeren Dimensionen sowie Timings

```
NoMP Compute Time: 0.9610044956207275 Comm Time: 0.7417397499084473
2MP Compute Time: 3.096160411834717 Comm Time: 2108.4389123916626
4MP Compute Time: 9.044354677200317 Comm Time: 7545.264755010605
8MP Compute Time: 9.22384262084961 Comm Time: 8170.565279483795
```

Abb. A.15.: Barebone MPI Commtime pro epoche

Inter-GPU Communication with CUDA-aware MPI

Using NCCL to perform inter-GPU communication concurrently with CUDA-aware MPI may create deadlocks.

NCCL creates inter-device dependencies, meaning that after it has been launched, a NCCL kernel will wait (and potentially block the CUDA device) until all ranks in the communicator launch their NCCL kernel. CUDA-aware MPI may also create such dependencies between devices depending on the MPI implementation.

Using both MPI and NCCL to perform transfers between the same sets of CUDA devices concurrently is therefore not guaranteed to be safe.

Abb. A.16.: Nvidia zu Cuda-aware MPI mit NCCL

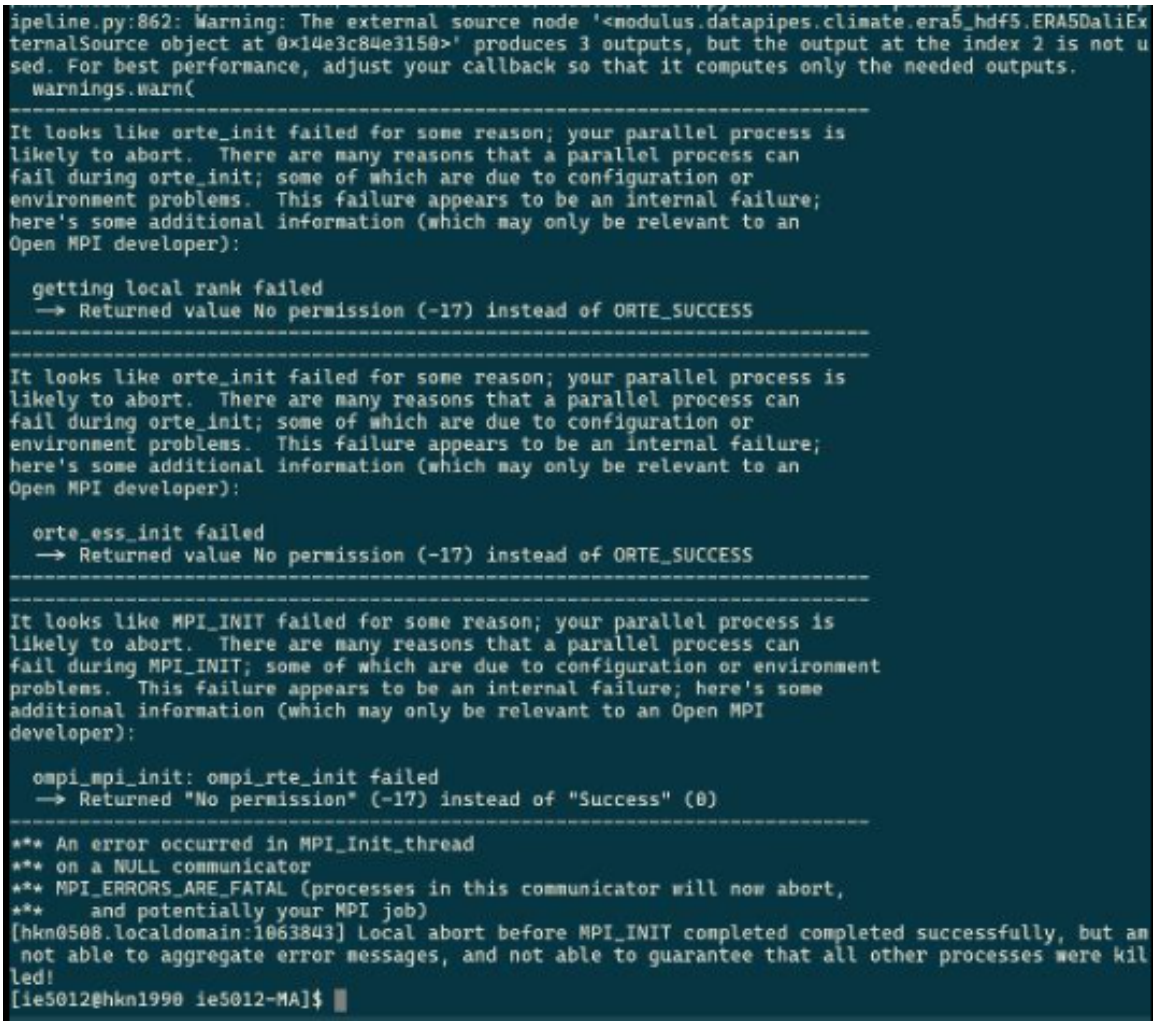


Abb. A.17.: Modulus Init Shinanigans